Stack Operativo Ottimizzato

Come integrare ChatGPT, Codex, Prism e GitHub per progettare, sviluppare e far evolvere una piattaforma di forecasting energetico

Maurizio Mormone

29 ottobre 2026
Versione 15

Abstract

Il progetto *Energy Forecasting Platform – AI Operational Stack* documenta la progettazione e l'evoluzione di una piattaforma per il forecasting energetico destinata a supportare un prosumer nella previsione integrata di consumi, produzione fotovoltaica e fabbisogno netto. L'obiettivo principale è trasformare dati storici, variabili meteorologiche e pattern temporali in informazioni operative utili alla pianificazione energetica, alla riduzione dell'incertezza decisionale e alla progressiva costruzione di una soluzione software governabile.

Il punto di partenza tecnico è un notebook monolitico, sviluppato come prototipo sperimentale per generare dati sintetici, preparare serie temporali, addestrare modelli LSTM sequence-to-sequence, produrre previsioni e calcolare il bilancio energetico tra consumo e produzione. Pur essendo efficace nella fase esplorativa, tale impostazione presenta limiti strutturali in termini di manutenibilità, riuso, testabilità e controllo dell'evoluzione. Il documento descrive quindi il percorso di refactoring necessario per trasformare il notebook in una piattaforma modulare, mantenendo continuità con la logica originaria ma separando progressivamente responsabilità, componenti e flussi operativi.

L'approccio adottato è quello dello Stack Operativo Ottimizzato, che integra progettazione architetturale, sviluppo assistito da AI, tracciabilità tramite GitHub e memoria documentale tramite Prism. ChatGPT supporta l'analisi, la formalizzazione dei requisiti, la definizione delle decisioni architetturali e la revisione concettuale; Codex agisce come agente di sviluppo incaricato di eseguire task operativi sul repository; GitHub governa Issue, branch, Pull Request, review e merge; Prism conserva la continuità documentale tra decisioni, avanzamento, modifiche significative e roadmap.

La soluzione è organizzata in Data Layer, Feature Engineering Layer, Forecasting Layer, Monitoring Layer, Energy Balance Layer e Visualization Layer. Questa suddivisione separa acquisizione e validazione, trasformazioni, modellazione predittiva, valutazione quantitativa, analisi energetica e rappresentazione visuale. La verifica automatica rimane una capacità trasversale pianificata attraverso la strategia di testing e la Issue #6.

Per il Visualization Layer, il materiale sorgente esaminato documenta sette API grafiche e una suite di test specifica. È stato comunicato l'esito di 18 test superati in 5,14 secondi con backend Matplotlib Agg. Questa evidenza riguarda esclusivamente il perimetro visuale e non implica il completamento della piattaforma, né verifica l'integrazione dei sorgenti nel repository o lo stato amministrativo delle Issue e delle Pull Request.

I benefici attesi riguardano la maggiore manutenibilità del codice, la scalabilità dell'architettura, il riuso dei componenti, la riduzione del rischio di regressioni e una migliore governance del progetto. Il documento mostra come la collaborazione uomo-AI, se inserita in un processo strutturato e revisionabile, possa accelerare la trasformazione di un prototipo analitico in una piattaforma software evolutiva, tracciabile e coerente con obiettivi tecnici e operativi.

1 Business Context

1.1 Introduzione al contesto energetico

Il settore energetico sta attraversando una trasformazione profonda, guidata dalla diffusione delle fonti rinnovabili, dall'aumento della volatilità dei prezzi, dalla crescita dell'elettrificazione dei consumi e dalla necessità di utilizzare l'energia in modo più consapevole. In questo scenario, la gestione dell'energia non può più essere considerata una funzione puramente amministrativa o consuntiva, basata solo sulla lettura delle bollette o sull'analisi storica dei consumi.

Le organizzazioni, le comunità energetiche e gli utenti dotati di impianti di produzione locale devono prendere decisioni sempre più frequenti e tempestive. Devono capire quando consumare, quando acquistare energia dalla rete, quando valorizzare la produzione fotovoltaica e come ridurre sprechi o inefficienze. La disponibilità di dati storici è importante, ma non sufficiente: per governare in modo efficace l'energia è necessario disporre di una vista prospettica.

Il progetto *Energy Forecasting Platform – AI Operational Stack* nasce in questo contesto. La sua finalità è supportare la costruzione di una piattaforma capace di prevedere fabbisogno energetico e produzione fotovoltaica, trasformando i dati in informazioni operative utili per il processo decisionale.

1.2 Il prosumer come cliente di riferimento

Il cliente di riferimento della soluzione è il *prosumer*, cioè un soggetto che non si limita a consumare energia, ma la produce anche, tipicamente attraverso un impianto fotovoltaico. Il prosumer può essere una piccola o media impresa, un edificio direzionale, una struttura pubblica, una comunità energetica, un sito industriale leggero o un'organizzazione con consumi elettrici rilevanti e capacità di generazione distribuita.

Per un prosumer, il rapporto con l'energia è più complesso rispetto a quello di un consumatore tradizionale. Non si tratta solo di acquistare energia al minor costo possibile, ma di bilanciare

consumo, autoproduzione, eventuale immissione in rete e strategie operative. La produzione fotovoltaica introduce opportunità, ma anche variabilità: dipende dalle condizioni meteorologiche, dalla stagione, dall'orario e dalle caratteristiche dell'impianto.

In questo contesto, il prosumer ha bisogno di strumenti che rendano leggibile il futuro energetico prossimo. Sapere in anticipo se nelle prossime ore o nei prossimi giorni ci sarà un eccesso di produzione, un picco di consumo o un fabbisogno da coprire tramite rete consente di prendere decisioni migliori e più tempestive.

1.3 Il problema operativo da risolvere

Il problema operativo principale è la mancanza di una previsione integrata e utilizzabile del bilancio energetico futuro. Molti soggetti dispongono di dati storici sui consumi e, in alcuni casi, di dati sulla produzione fotovoltaica. Tuttavia, questi dati sono spesso consultati a posteriori, quando l'opportunità decisionale è già passata.

Senza una previsione affidabile, le decisioni energetiche rimangono reattive. L'organizzazione interviene dopo aver osservato un'anomalia, un costo elevato o uno squilibrio tra produzione e consumo. Questo approccio limita la capacità di ottimizzare l'uso dell'energia, ridurre gli sprechi e pianificare azioni correttive.

La piattaforma intende affrontare questo problema trasformando dati storici, informazioni meteorologiche e pattern temporali in previsioni operative. Il valore non consiste solo nel produrre un numero, ma nel rendere quel numero utilizzabile per decidere come gestire l'energia in un orizzonte temporale prossimo.

1.4 Necessità di prevedere consumi e produzione fotovoltaica

La previsione dei consumi energetici e della produzione fotovoltaica risponde a due esigenze complementari. Da un lato, prevedere i consumi consente di stimare il fabbisogno futuro dell'organizzazione. Dall'altro, prevedere la produzione fotovoltaica consente di stimare quanta parte di tale fabbisogno potrà essere coperta dall'autoproduzione.

Queste due grandezze devono essere considerate insieme, perché il valore operativo emerge dal loro confronto. Un consumo elevato non è necessariamente critico se coincide con un'elevata produzione locale. Allo stesso modo, una produzione elevata può generare opportunità se supera il fabbisogno previsto e può essere valorizzata tramite autoconsumo, accumulo o immissione in rete.

Il forecasting energetico permette quindi di passare da una vista separata delle singole serie a una lettura integrata del bilancio futuro. Questa prospettiva è essenziale per un prosumer, perché il problema non è soltanto conoscere il consumo o la produzione, ma comprendere la relazione tra i due fenomeni nel tempo.

1.5 Impatto delle previsioni sulle decisioni energetiche

Le previsioni energetiche incidono direttamente sulla qualità delle decisioni operative. Una previsione attendibile consente di anticipare picchi di consumo, individuare finestre favorevoli di autoproduzione, stimare l'energia da acquistare dalla rete e valutare l'energia potenzialmente cedibile o non utilizzata.

Per un'organizzazione, questo significa poter pianificare azioni in modo più razionale. Alcuni carichi possono essere programmati in fasce orarie più convenienti, alcune attività possono essere coordinate con la disponibilità di energia fotovoltaica e alcune anomalie possono essere intercettate prima che producano impatti economici o operativi significativi.

Il forecasting non sostituisce la responsabilità decisionale delle persone, ma ne aumenta la capacità informativa. La previsione diventa uno strumento di supporto: riduce l'incertezza, rende visibili scenari futuri e permette di confrontare alternative operative con maggiore consapevolezza.

1.6 Stakeholder coinvolti

Il progetto coinvolge più stakeholder, ciascuno con esigenze e responsabilità differenti. La piattaforma deve quindi produrre valore non solo per chi sviluppa o gestisce il sistema, ma anche per chi utilizza le informazioni nel lavoro quotidiano.

L'**Energy Manager** è interessato alla qualità delle previsioni, al bilancio energetico e alla possibilità di individuare opportunità di ottimizzazione. Per questa figura, la piattaforma rappresenta uno strumento di analisi e governo dell'energia, utile per ridurre costi, migliorare autoconsumo e supportare strategie di sostenibilità.

Il **Responsabile Operativo** utilizza le previsioni per pianificare attività, carichi e processi in modo più coerente con la disponibilità energetica prevista. Il suo interesse principale è trasformare l'informazione previsionale in azioni pratiche, comprensibili e compatibili con i vincoli dell'organizzazione.

L'**IT Manager** è coinvolto perché la soluzione deve inserirsi in un contesto digitale governabile, tracciabile e manutenibile. Anche senza entrare nei dettagli tecnici, è importante che il progetto sia organizzato in modo chiaro, documentato e compatibile con processi di sviluppo controllati.

Gli **utilizzatori finali delle previsioni** sono le persone che consultano report, dashboard o indicazioni operative generate dalla piattaforma. Per loro il valore risiede nella chiarezza dell'informazione: le previsioni devono essere comprensibili, contestualizzate e orientate all'azione.

1.7 Obiettivi di business della soluzione

Gli obiettivi di business della soluzione sono orientati a migliorare la capacità dell'organizzazione di governare il proprio profilo energetico. Il primo obiettivo è aumentare la prevedibilità del fabbisogno, così da ridurre decisioni improvvisate e interventi puramente reattivi.

Un secondo obiettivo è valorizzare meglio la produzione fotovoltaica. La disponibilità di energia autoprodotta genera benefici solo se viene compresa e integrata nei processi decisionali. Prevedere la produzione consente di aumentare l'autoconsumo, ridurre l'energia acquistata e valutare con maggiore chiarezza le situazioni di surplus.

Un terzo obiettivo è creare una base informativa stabile per decisioni ricorrenti. La piattaforma deve aiutare l'organizzazione a costruire un metodo: osservare i dati, prevedere gli scenari, interpretare il bilancio energetico e decidere azioni coerenti.

1.8 Benefici attesi

I benefici attesi riguardano sia la dimensione economica sia quella organizzativa. Sul piano economico, una migliore previsione può contribuire a ridurre acquisti non pianificati, migliorare l'utilizzo dell'energia autoprodotta e rendere più consapevole la gestione dei picchi di consumo.

Sul piano operativo, la piattaforma può migliorare la capacità di pianificazione. Le decisioni non dipendono più soltanto dall'esperienza individuale o dall'analisi retrospettiva, ma possono essere supportate da indicatori previsionali aggiornati e leggibili.

Sul piano manageriale, il progetto favorisce una maggiore tracciabilità delle decisioni energetiche. Report e previsioni possono diventare parte di un processo ricorrente di monitoraggio, discussione e miglioramento. In questo modo, il forecasting energetico non rimane un esercizio tecnico, ma entra nel ciclo decisionale dell'organizzazione.

1.9 Motivazioni per lo Stack Operativo Ottimizzato

La definizione dello Stack Operativo Ottimizzato nasce dalla necessità di governare un progetto che unisce dominio energetico, analisi dati, sviluppo software e documentazione. Una piattaforma di forecasting non richiede solo un modello predittivo: richiede un processo capace di trasformare conoscenza, codice e decisioni in un sistema evolutivo.

ChatGPT, Codex, Prism e GitHub sono stati organizzati come strumenti complementari per sostenere questo percorso. La progettazione deve essere chiarita e documentata, lo sviluppo deve essere tracciabile, le decisioni devono essere motivate e il repository deve poter evolvere in modo controllato.

Lo Stack Operativo Ottimizzato risponde quindi a una motivazione di business oltre che tecnica: ridurre il rischio di dispersione, rendere più veloce il passaggio dall'idea all'implementazione e mantenere allineati obiettivi, documentazione e attività operative.

1.10 Collegamento tra forecasting energetico e processo decisionale

Il forecasting energetico crea valore quando diventa parte del processo decisionale. Una previsione isolata, non collegata ad azioni o responsabilità, rischia di rimanere un dato informativo. Al contrario, una previsione inserita in un flusso di lavoro consente di orientare scelte operative, valutazioni economiche e priorità manageriali.

Nel progetto, il collegamento tra forecasting e decisione avviene attraverso la trasformazione dei dati previsionali in informazioni comprensibili: consumo atteso, produzione attesa, energia da acquistare, energia potenzialmente immessa e indicatori sintetici. Queste informazioni permettono agli stakeholder di leggere il futuro energetico prossimo e di intervenire con maggiore anticipo.

La piattaforma si propone quindi come supporto alla decisione, non come sostituto del giudizio umano. Il suo compito è rendere l'incertezza più gestibile, offrire una base informativa coerente e abilitare un governo energetico più consapevole, misurabile e orientato al miglioramento continuo.

2 Requirements

2.1 Introduzione

Questo documento formalizza i requisiti della soluzione *Energy Forecasting Platform – AI Operational Stack*, derivati dall'analisi del problema cliente descritta nel Business Context. Il suo ruolo è tradurre il bisogno di governo energetico del prosumer in un insieme coerente di capacità attese, vincoli progettuali e qualità richieste alla soluzione.

All'interno dello Stack Operativo Ottimizzato, i requisiti costituiscono il punto di raccordo tra la comprensione del problema e le successive decisioni architetturali. Essi permettono di mantenere allineati obiettivi di business, progettazione documentale, backlog operativo e attività future di sviluppo, evitando che la soluzione venga guidata esclusivamente da scelte tecniche o da sperimentazioni isolate.

Il documento non descrive implementazioni software, non introduce ADR e non anticipa le attività dello Strato 3. La sua finalità è definire che cosa la soluzione deve rendere possibile, con quale livello di qualità atteso e secondo quali vincoli di progetto.

2.2 Obiettivi funzionali della soluzione

La piattaforma deve consentire la previsione dei consumi energetici del prosumer, in modo da rendere visibile il fabbisogno atteso su un orizzonte temporale utile alla pianificazione. Questa capacità è centrale perché permette di passare da una lettura consuntiva dei consumi a una vista prospettica, più adatta a supportare decisioni operative.

Accanto alla previsione dei consumi, la soluzione deve prevedere la produzione fotovoltaica. La

produzione locale introduce variabilità e opportunità: stimarla in anticipo consente di comprendere quanta parte del fabbisogno potrà essere coperta dall'autoproduzione e in quali momenti potranno verificarsi surplus o deficit energetici.

La piattaforma deve produrre forecast orari multi-serie, mantenendo distinte ma confrontabili le previsioni relative a consumo e produzione. La granularità oraria è necessaria per supportare decisioni operative concrete, mentre la gestione multi-serie permette di rappresentare correttamente la natura del problema energetico del prosumer.

L'orizzonte previsionale di riferimento è pari a 168 ore, corrispondenti a una settimana. Questo intervallo consente di combinare pianificazione giornaliera e visione settimanale, offrendo agli stakeholder una finestra temporale sufficientemente ampia per anticipare criticità, organizzare attività e valutare scenari.

La soluzione deve inoltre calcolare il bilancio energetico atteso, confrontando consumi previsti e produzione prevista. Da questo confronto devono derivare la stima dell'energia acquistata dalla rete e la stima dell'energia immessa in rete. Tali informazioni trasformano il forecast in uno strumento decisionale, perché rendono esplicito l'impatto operativo della relazione tra domanda e autoproduzione.

Infine, la piattaforma deve produrre KPI e report utilizzabili dagli stakeholder. Le previsioni non devono rimanere dati tecnici isolati, ma devono essere sintetizzate in indicatori leggibili, utili per Energy Manager, Responsabile Operativo, IT Manager e utilizzatori finali delle previsioni.

2.3 Requisiti relativi ai dati

Nella fase iniziale, la soluzione deve poter utilizzare dati sintetici. Questo requisito consente di sviluppare, validare e discutere il comportamento generale della piattaforma anche in assenza di dataset reali completi o immediatamente disponibili. I dati sintetici devono essere considerati uno strumento di avvio e sperimentazione, non il punto di arrivo del progetto.

La piattaforma deve essere predisposta alla successiva integrazione con dati reali. Il passaggio dai dati sintetici ai dati effettivi del prosumer è essenziale per trasformare la soluzione da prototipo controllato a strumento operativo. Tale evoluzione dovrà preservare la coerenza del processo e rendere possibile il confronto tra risultati sperimentali e risultati ottenuti su dati reali.

La soluzione deve gestire dati storici di consumo energetico e dati storici di produzione fotovoltaica. Queste due categorie informative rappresentano la base per comprendere i pattern del prosumer e per costruire previsioni utili. I dati storici devono essere trattati come patrimonio informativo del progetto, perché descrivono abitudini, cicli, anomalie e condizioni operative ricorrenti.

La piattaforma deve integrare dati meteorologici, poiché la produzione fotovoltaica e, in alcuni casi, anche i consumi possono essere influenzati dalle condizioni ambientali. Temperatura, irraggiamento, copertura nuvolosa o altre variabili meteo possono contribuire a migliorare la qualità delle previsioni e a rendere più interpretabili alcuni comportamenti energetici.

Un requisito fondamentale riguarda la qualità dei dati. La soluzione deve prevedere controlli sulla coerenza temporale, sulla presenza delle informazioni attese, sulla plausibilità dei valori e sulla continuità delle serie. La qualità del dato condiziona direttamente la qualità della previsione: dati incompleti, incoerenti o non tracciati possono compromettere l'affidabilità del sistema.

La gestione dei valori mancanti deve essere affrontata in modo esplicito. Il sistema deve poter riconoscere assenze informative, segnalarle e trattarle secondo regole coerenti con il contesto. La presenza di valori mancanti non deve generare risultati opachi o non interpretabili.

Infine, le sorgenti informative devono essere tracciabili. Deve essere possibile comprendere da dove proviene un dataset, a quale periodo si riferisce, con quale livello di affidabilità è stato acquisito e quali trasformazioni principali ha subito. La tracciabilità delle sorgenti è necessaria sia per ragioni operative sia per sostenere audit, manutenzione e miglioramento continuo.

2.4 Requisiti relativi ai modelli

La soluzione deve prevedere modelli distinti per consumi e produzione fotovoltaica. Questa separazione risponde alla diversa natura dei due fenomeni: i consumi dipendono da comportamenti, processi, calendario e abitudini operative, mentre la produzione fotovoltaica dipende in modo significativo dalle condizioni ambientali e dalle caratteristiche dell'impianto.

I modelli devono poter essere sottoposti a retraining periodico. Nel tempo, i pattern energetici possono cambiare a causa di nuove abitudini di consumo, modifiche organizzative, variazioni impiantistiche o condizioni esterne. Il retraining consente di mantenere le previsioni allineate al comportamento reale del sistema energetico osservato.

La piattaforma deve supportare il monitoraggio delle prestazioni previsionali. Non è sufficiente generare forecast: è necessario valutare nel tempo quanto le previsioni siano accurate, stabili e utili. Il monitoraggio permette di rilevare decadimenti di performance, anomalie e possibili necessità di aggiornamento.

I modelli devono essere versionati. Ogni versione deve poter essere collegata al periodo di addestramento, ai dati utilizzati, alle metriche ottenute e alle condizioni operative note. Il versionamento è indispensabile per confrontare modelli diversi, riprodurre risultati e comprendere l'evoluzione della soluzione.

La soluzione deve conservare lo storico delle metriche. Questo archivio permette di valutare le prestazioni nel tempo, confrontare esperimenti e supportare decisioni su mantenimento, aggiornamento o sostituzione dei modelli previsionali.

2.5 Requisiti operativi

La piattaforma deve prevedere l'esecuzione periodica delle pipeline previsionali. Il valore operativo della soluzione dipende dalla capacità di produrre informazioni aggiornate con regolarità, evitando

che le previsioni diventino rapidamente obsolete rispetto al contesto energetico reale.

La generazione delle previsioni deve essere automatizzabile. Gli stakeholder devono poter contare su forecast prodotti in modo ricorrente e controllato, senza dipendere da esecuzioni manuali occasionali o da attività non documentate. L'automazione riduce il rischio operativo e rende la soluzione più vicina a un utilizzo continuativo.

La piattaforma deve produrre report utilizzabili nel processo decisionale. I report devono sintetizzare i risultati previsionali e il bilancio energetico atteso, rendendo comprensibili le informazioni principali a figure con responsabilità diverse. Il report non deve essere un semplice output tecnico, ma uno strumento di comunicazione operativa.

La soluzione deve consentire l'archiviazione degli esperimenti. Ogni esecuzione significativa, ogni confronto tra modelli e ogni variazione rilevante delle ipotesi deve poter essere registrata. Questo requisito permette di mantenere memoria del percorso evolutivo del progetto e di comprendere perché certe scelte sono state mantenute o superate.

Infine, la piattaforma deve supportare la manutenzione evolutiva. Il progetto nasce da un notebook e dovrà evolvere progressivamente: i requisiti operativi devono quindi permettere miglioramenti successivi, integrazione di nuovi dati, revisione dei modelli e adattamento delle modalità di reporting.

2.6 Requisiti di qualità

L'affidabilità è un requisito centrale. La piattaforma deve produrre risultati coerenti, controllabili e sufficientemente stabili da poter essere utilizzati a supporto di decisioni energetiche. L'affidabilità non riguarda solo l'accuratezza del modello, ma anche la qualità del processo che porta dal dato alla previsione.

La manutenibilità è necessaria perché la soluzione dovrà evolvere nel tempo. Un sistema difficilmente manutenibile riduce il valore del progetto, aumenta il costo delle modifiche e rende più rischioso ogni intervento successivo. La documentazione, la chiarezza delle responsabilità e la tracciabilità delle decisioni contribuiscono direttamente a questo requisito.

La modularità deve consentire di separare responsabilità diverse senza confondere dati, previsioni, bilancio energetico e reporting. Anche senza entrare nei dettagli implementativi, il requisito di modularità stabilisce che la soluzione deve poter evolvere per parti, evitando dipendenze implicite o accoppiamenti non governati.

La tracciabilità deve permettere di collegare requisiti, decisioni, dataset, modelli, risultati e attività operative. In un progetto supportato da strumenti AI e da un repository GitHub, la tracciabilità è indispensabile per mantenere controllo e responsabilità sulle evoluzioni della soluzione.

La documentabilità indica la necessità di produrre e mantenere documentazione comprensibile. Il progetto deve poter essere letto, spiegato e trasferito; non deve dipendere esclusivamente dalla memoria individuale o da conoscenza implicita contenuta nel notebook originale.

L'estendibilità della piattaforma è un requisito di qualità orientato al futuro. La soluzione deve poter accogliere nuove sorgenti dati, nuovi scenari energetici, nuovi report e miglioramenti progressivi senza richiedere una riprogettazione completa.

2.7 Vincoli progettuali

Il repository GitHub rappresenta l'ambiente centrale di sviluppo. Tutte le evoluzioni rilevanti del progetto dovranno essere tracciate, versionate e rese verificabili attraverso il repository, che costituisce il punto di coordinamento tra documentazione, backlog e sviluppo futuro.

ChatGPT deve essere utilizzato come supporto analitico e architetturale. Il suo ruolo è aiutare a chiarire requisiti, strutturare decisioni, verificare coerenza documentale e supportare la formulazione di attività operative comprensibili.

Codex sarà utilizzato per implementazione e refactoring nelle fasi successive. In questo documento il suo ruolo è definito come vincolo di processo: le attività future dovranno essere descritte in modo sufficientemente chiaro da poter essere trasformate in modifiche controllate al repository.

Prism rappresenta il repository della conoscenza progettuale. Deve preservare contesto, decisioni, motivazioni, documenti e continuità tra analisi, progettazione e implementazione. Il valore di Prism è mantenere leggibile il percorso del progetto nel tempo.

Un vincolo fondamentale è l'approccio progressivo da notebook a soluzione modulare. Il notebook originale rappresenta il punto di partenza sperimentale; la soluzione finale dovrà emergere attraverso passaggi controllati, evitando riscritture improvvise e mantenendo la possibilità di confrontare i risultati con la baseline iniziale.

2.8 Requisiti non funzionali

Il codice che verrà prodotto nelle fasi successive dovrà essere leggibile. La leggibilità è un requisito non funzionale essenziale perché facilita review, manutenzione, troubleshooting e collaborazione tra strumenti e persone.

La separazione delle responsabilità deve guidare l'evoluzione della soluzione. Ogni parte del sistema dovrà avere un ruolo chiaro, così da ridurre ambiguità e rendere più semplice individuare dove intervenire in caso di modifica o problema.

La testabilità è necessaria per rendere affidabile il passaggio da notebook a piattaforma. Le funzioni e i comportamenti principali dovranno poter essere verificati in modo controllato, riducendo il rischio di regressioni durante refactoring e manutenzione evolutiva.

Il versionamento completo deve riguardare codice, documentazione, modelli, metriche e artefatti rilevanti. Senza versionamento, sarebbe difficile ricostruire il percorso progettuale, confrontare risultati o comprendere l'impatto di una modifica.

La soluzione deve supportare il troubleshooting. In caso di risultato anomalo, errore operativo o decadimento delle prestazioni, deve essere possibile analizzare il problema, ricostruire il flusso informativo e individuare la causa con un livello adeguato di evidenza.

Infine, la piattaforma deve supportare l'evoluzione futura. Il progetto non deve essere concepito come un prodotto chiuso, ma come una base progressiva su cui introdurre nuove capacità, nuovi dati, nuovi modelli e nuove modalità di fruizione delle previsioni.

2.9 Conclusioni

I requisiti formalizzati in questo documento costituiscono la base per le scelte architetturali documentate nello Strato 2. Essi derivano dal problema cliente e descrivono le capacità che la soluzione deve possedere per generare valore nel contesto del forecasting energetico per prosumer.

Questi requisiti guideranno anche le attività implementative successive, mantenendo un collegamento esplicito tra bisogno di business, progettazione, backlog e sviluppo. In questo modo lo Stack Operativo Ottimizzato potrà sostenere un'evoluzione coerente: dal notebook sperimentale a una soluzione modulare, documentata, tracciabile e orientata al processo decisionale.

3 Architettura logica preliminare

3.1 Scopo del documento

Questo documento formalizza l'architettura logica implicita nel notebook `Previsione Fabbisogno v05_all.ipynb`, considerandolo come implementazione iniziale di una soluzione di forecasting energetico per un prosumer. L'analisi adotta come riferimento metodologico il documento *Esempio pratico di uno Stack Operativo Ottimizzato v01*, che propone una progressione ordinata dagli strati di acquisizione e analisi, architettura e progettazione, implementazione modulare, validazione e successiva operativizzazione.

L'obiettivo non è modificare il codice esistente, ma rendere esplicite le responsabilità architetturali già presenti nel notebook e preparare una futura separazione in moduli Python indipendenti.

3.2 Sintesi della soluzione analizzata

Il notebook implementa un flusso end-to-end per la previsione oraria di consumo e produzione fotovoltaica sui successivi sette giorni. La soluzione parte dalla generazione di dati sintetici storici, costruisce variabili meteo e target energetici, prepara sequenze temporali per modelli LSTM sequence-to-sequence, addestra due modelli distinti per consumo e produzione, calcola metriche di valutazione, genera forecast futuri e deriva il bilancio energetico orario.

Dal punto di vista logico, il notebook contiene già tutti gli elementi di una pipeline applicativa completa, ma tali elementi sono ancora concentrati in un unico artefatto esecutivo. La responsabilità architetturale principale della fase successiva sarà trasformare questa pipeline monolitica in componenti separati, testabili e riutilizzabili.

3.3 Architettura logica implicita

L'architettura implicita può essere descritta come una pipeline a layer sequenziali:

1. acquisizione o generazione dei dati storici;
2. pulizia, normalizzazione e preparazione delle variabili;
3. costruzione delle sequenze encoder-decoder;
4. addestramento e valutazione dei modelli previsionali;
5. generazione delle previsioni future;
6. calcolo del bilancio energetico;
7. produzione di report, metriche e grafici.

Questa pipeline è coerente con lo Stack Operativo Ottimizzato perché separa progressivamente problema, architettura, implementazione, validazione e deliverable. Nel notebook, tuttavia, tali separazioni sono concettuali più che fisiche: funzioni, configurazioni, addestramento, persistenza, reportistica e visualizzazione convivono nello stesso documento.

3.4 Layer architetturali

3.4.1 Data Layer

Il Data Layer ha la responsabilità di costruire e rendere disponibili le serie temporali di base usate dal resto della pipeline. Nel notebook include:

- definizione dei parametri generali di orizzonte storico, frequenza, finestre temporali e split del dataset;
- generazione dell'indice temporale storico tramite `genera_date_range_storico`;
- generazione dei dati meteo sintetici tramite `genera_meteo_sintetico`;
- generazione del consumo sintetico tramite `genera_consumo_sintetico`;
- generazione della produzione fotovoltaica sintetica tramite `genera_produzione_sintetica`;

- eventuale caricamento alternativo da CSV per consumo e produzione;
- concatenazione del dataset storico complessivo con meteo, consumo e produzione;
- costruzione del dataset meteo futuro sui sette giorni di previsione.

Questo layer rappresenta il punto di ingresso della soluzione. In una futura architettura modulare dovrebbe isolare sorgenti dati sintetiche, sorgenti dati reali, validazione dello schema e persistenza dei dataset intermedi.

3.4.2 Feature Engineering Layer

Il Feature Engineering Layer trasforma i dati grezzi in strutture utilizzabili dai modelli previsionali. Nel notebook include:

- pulizia e imputazione tramite `pulisci_e_imputa`;
- suddivisione temporale in train, validation e test tramite `split_time_series`;
- analisi statistica dei dataset tramite `analisi_statistica`;
- selezione delle feature storiche per consumo e produzione;
- selezione delle feature meteo future per il decoder;
- scaling delle feature e dei target tramite `StandardScaler`;
- trasformazione inversa delle previsioni tramite `inverse_transform_array`;
- costruzione dei dataset sequenziali tramite `crea_sequence_dataset`;
- validazione delle dimensioni tramite `validate_shapes`.

Questo layer dipende dal Data Layer e produce tensori o array strutturati per il Forecasting Layer. La sua responsabilità deve rimanere distinta dalla scelta del modello: deve sapere come costruire sequenze e trasformazioni, non come addestrare una rete neurale.

3.4.3 Forecasting Layer

Il Forecasting Layer contiene la logica predittiva. Nel notebook include:

- definizione dell'architettura LSTM sequence-to-sequence tramite `build_seq2seq_model`;
- configurazione di input encoder, input decoder, LSTM, dropout e layer denso time-distributed;
- addestramento del modello di consumo;

- addestramento del modello di produzione;
- uso di callback come early stopping e checkpoint;
- salvataggio dei modelli e delle history di training;
- generazione delle previsioni su train, validation e test;
- previsione dei successivi sette giorni usando le ultime finestre storiche e il meteo futuro.

La soluzione prevede due modelli distinti, uno per il consumo e uno per la produzione. Il layer espone responsabilità di training e inference, evitando dipendenze da metriche, grafici o logica energetica.

3.4.4 Monitoring Layer

Il Monitoring Layer è un layer ufficiale e implementato attraverso `metrics.py` ed `evaluation.py`. Esso calcola MAE e RMSE e orchestra il confronto tra valori osservati e predetti già riportati alla scala originale. Non esegue training, prediction, visualizzazione o drift monitoring. L'assenza di una Issue numerata dedicata nel backlog storico riguarda esclusivamente la tracciabilità originaria e non riduce il ruolo architetturale del layer.

3.4.5 Energy Balance Layer

L'Energy Balance Layer traduce le previsioni in indicatori energetici operativi. Nel notebook include:

- costruzione del dataframe di forecast con consumo previsto e produzione prevista;
- calcolo di `bilancio` e `net_balance` come produzione meno consumo;
- separazione dei flussi originali `to_grid` e `to_buy`;
- calcolo dei totali e dei valori cumulativi sui sette giorni;
- classificazione testuale ed esportazione CSV, presenti nel prototipo ma esterne al contratto dei nuovi moduli.

L'architettura adotta i nomi canonici `net_balance`, `grid_export` e `grid_import`. Il layer deve dipendere solo dall'output fisico del Forecasting Layer e non dai dettagli di addestramento dei modelli. Non applica clipping automatico agli input, non classifica obbligatoriamente il caso di bilancio nullo e non gestisce file I/O.

3.4.6 Visualization Layer

Il Visualization Layer rende leggibili output, metriche e risultati già calcolati. Il materiale sorgente fornito per la v15 documenta un'implementazione di `plotting.py` con sette API pubbliche e una suite di test specifica. Questa evidenza riguarda il codice descritto nel materiale e non dimostra, da sola, l'integrazione nel branch ufficiale o la chiusura della Issue #5.

Le API rappresentano training history, una finestra reale–predetto, la distribuzione degli errori, MAE/RMSE già calcolati, forecast futuro congiunto, scambio orario con la rete e cumulativi dei flussi. Nel notebook le visualizzazioni di dataset e correlazioni restano esplorative e fuori dalla suite operativa iniziale; la stampa di riepiloghi resta esterna alla Issue #5. Le funzioni restituiscono oggetti Matplotlib in memoria e non assumono la responsabilità di calcolo, file I/O o pubblicazione.

Nel notebook originale il layer include:

- visualizzazione delle loss function tramite `plot_loss`;
- confronto tra valori reali e predetti tramite `plot_confronto_window`;
- visualizzazione della distribuzione degli errori tramite `plot_error_distribution`;
- grafico di consumo e produzione previsti;
- grafico di `grid_import` e `grid_export`;
- grafico dei relativi cumulativi;
- stampa di riepiloghi, presente nel notebook ma esterna alla Issue #5.

L'implementazione di `plotting.py` è presente nel materiale sorgente esaminato. La Issue #5 era descritta come aperta nella baseline v14; il suo stato amministrativo attuale e l'eventuale stato della Pull Request non sono verificabili dalle fonti disponibili. Il modulo resta opzionale rispetto all'esecuzione della pipeline numerica; eventuali dashboard rimangono estensioni future.

3.5 Dipendenze tra layer

Le dipendenze logiche individuate sono le seguenti:

In dettaglio:

- il Feature Engineering Layer dipende dagli output del Data Layer, in particolare dataset storico e meteo futuro;
- il Forecasting Layer dipende dalle sequenze, dagli scaler e dai target prodotti dal Feature Engineering Layer;

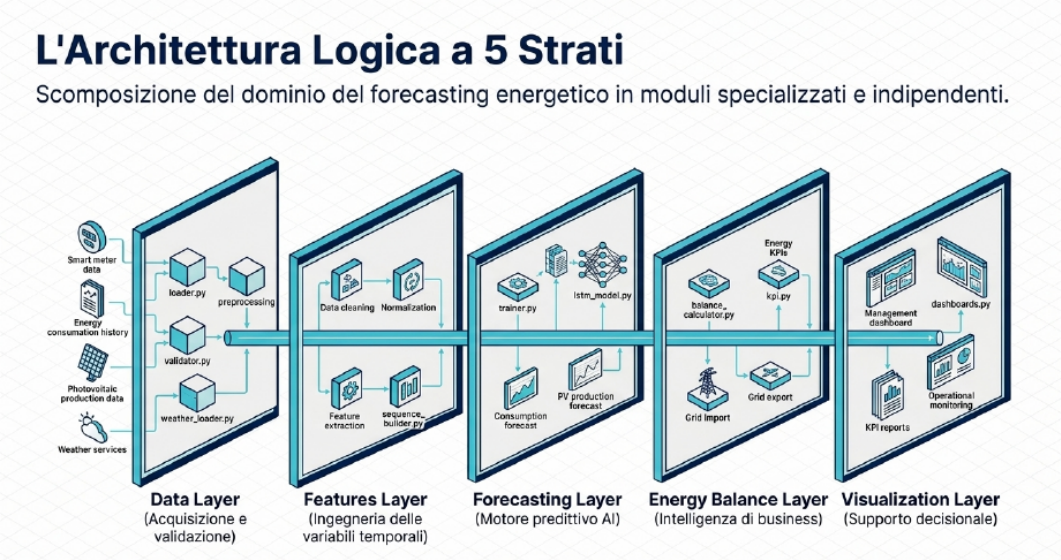


Figure 1: Schema storico a cinque strati, precedente al consolidamento del Monitoring Layer.

- il Monitoring Layer dipende da valori osservati e predetti già espressi nella scala originale;
- l'Energy Balance Layer dipende esclusivamente dalle previsioni finali di consumo e produzione;
- il Visualization Layer dipende da previsioni, metriche e bilanci già calcolati;
- il Data Layer non dovrebbe dipendere da nessun layer successivo;
- il Forecasting Layer non dovrebbe dipendere da visualizzazioni o reportistica;
- l'Energy Balance Layer non dovrebbe conoscere dettagli interni dei modelli LSTM.

Questa direzione delle dipendenze è coerente con una progettazione modulare a pipeline e riduce il rischio di accoppiamento tra notebook sperimentale e componenti riutilizzabili.

3.6 Componenti da estrarre in moduli Python

Le parti del notebook da estrarre successivamente in moduli indipendenti sono:

- `config`: costanti di finestra temporale, split, parametri fisici, parametri di training e percorsi di output;
- `data_generation`: funzioni per date range, meteo sintetico, consumo sintetico e produzione sintetica;
- `data_loading`: caricamento opzionale da CSV e normalizzazione degli schemi dati;
- `preprocessing`: pulizia, imputazione, split temporale e analisi statistica;

- `features`: selezione feature, scaling, inverse transform e costruzione sequenze encoder-decoder;
- `models`: definizione del modello LSTM sequence-to-sequence;
- `training`: addestramento, callback, checkpoint e salvataggio history;
- `forecasting`: definizione del modello, training e inference sui sette giorni futuri;
- `monitoring`: calcolo di MAE e RMSE e orchestrazione della valutazione su valori osservati e predetti;
- `energy_balance`: calcolo di `net_balance`, `grid_import`, `grid_export`, totali e cumulativi;
- `visualization`: grafici di loss, confronto, errore, forecast e cumulati; il materiale sorgente per `plotting.py` e i test specifici è ora disponibile, mentre integrazione nel repository e collegamento al notebook non sono verificati;
- `reporting`: esportazione CSV, logging e riepiloghi testuali.

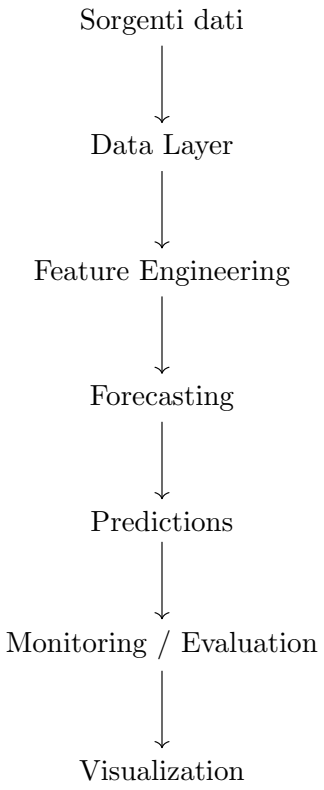
Questa estrazione dovrebbe avvenire per passi incrementali, evitando una riscrittura completa iniziale. La roadmap ha già formalizzato le fasi per dati, feature engineering, forecasting, monitoring ed energy balance; per il Visualization Layer i sorgenti e i test specifici sono presenti nel materiale fornito, mentre restano da verificare l'integrazione nel repository e l'uso effettivo dal notebook ottimizzato.

3.7 Proposta di architettura coerente con lo Stack Operativo Ottimizzato

Seguendo la metodologia dello Stack Operativo Ottimizzato, la soluzione dovrebbe evolvere secondo una progressione controllata:

1. **Acquisizione e analisi**: consolidare obiettivi, stakeholder, dati disponibili, ipotesi sintetiche e rischi legati al passaggio da dati simulati a dati reali.
2. **Architettura e progettazione**: stabilire confini tra layer, interfacce minime, formato dei dataset, formato delle previsioni e responsabilità dei moduli.
3. **Implementazione modulare**: estrarre gradualmente funzioni dal notebook in moduli Python, mantenendo il notebook come orchestratore o demo applicativa.
4. **Validazione e test**: introdurre test su generazione dati, trasformazioni, shape delle sequenze, metriche e calcolo del bilancio energetico.
5. **Operativizzazione**: predisporre logging, salvataggio modelli, configurazioni esterne, gestione input reali e produzione di output ripetibili.

L'architettura target distingue due percorsi a valle delle previsioni: la valutazione quantitativa e l'analisi energetica. Entrambi possono alimentare la rappresentazione visuale:



Dove le sorgenti dati possono essere sintetiche nella fase iniziale e reali nelle fasi successive. In parallelo, le predizioni alimentano l'Energy Balance Layer, che produce bilanci e KPI utilizzabili dal Visualization Layer e dall'Operational Analysis. Il notebook dovrebbe diventare progressivamente un documento di esperimento, mentre la logica di produzione dovrebbe risiedere in moduli versionati e testabili.

3.8 Rischi architetturali individuati

- **Accoppiamento elevato**: il notebook combina configurazione, dati, modelli, valutazione, bilancio e grafici.
- **Riproducibilità limitata**: la generazione casuale e l'uso di timestamp correnti richiedono una gestione esplicita di seed e date di riferimento.
- **Dipendenza dai dati sintetici**: la pipeline è utile per prototipazione, ma deve essere validata su dati reali con schemi e anomalie realistiche.
- **Persistenza non centralizzata**: modelli, history, log e CSV sono prodotti in punti diversi del flusso.
- **Interfacce implicite**: shape, colonne attese e finestre temporali sono comprensibili dal codice, ma non ancora formalizzate come contratti.

3.9 Deliverable architetturali preliminari

I deliverable consigliati per la sezione architetturale sono:

- mappa dei layer logici della soluzione;
- diagramma delle dipendenze tra layer;
- elenco delle responsabilità per layer;
- elenco dei moduli Python candidati all'estrazione;
- definizione preliminare degli input e output di ciascun layer;
- identificazione dei rischi architetturali da gestire nelle fasi successive;
- backlog tecnico per modularizzazione, test e operativizzazione.

3.10 Conclusione

Il notebook rappresenta una valida implementazione iniziale e dimostra l'intero ciclo funzionale della soluzione: dati, feature, modelli, forecast, bilancio e visualizzazione. La principale evoluzione architetturale consiste nel trasformare questa soluzione monolitica in una pipeline modulare, coerente con lo Stack Operativo Ottimizzato, in cui ogni layer abbia responsabilità esplicite, dipendenze controllate e interfacce testabili.

4 Architecture Decision Records

4.1 ADR-001: Evoluzione da notebook monolitico a soluzione modulare

4.1.1 Titolo

Evoluzione da notebook monolitico a soluzione modulare.

4.1.2 Stato

Accettata.

4.1.3 Data

9 giugno 2026.

4.1.4 Contesto

Il notebook di forecasting energetico rappresenta l'implementazione iniziale della soluzione e contiene, in un unico artefatto esecutivo, logiche eterogenee relative a caricamento e generazione dei dati, preprocessing, feature engineering, costruzione delle sequenze temporali, addestramento dei modelli, valutazione, calcolo del bilancio energetico e visualizzazione dei risultati.

Questa impostazione è coerente con una fase esplorativa e prototipale, perché consente di validare rapidamente il flusso end-to-end. Tuttavia, come evidenziato nel documento architetturale preliminare, la concentrazione di responsabilità rende progressivamente più difficile la manutenzione, il testing, il controllo delle regressioni e l'evoluzione della soluzione verso un'applicazione più robusta.

La metodologia dello Stack Operativo Ottimizzato suggerisce una progressione ordinata: analisi del problema, definizione dell'architettura logica, implementazione modulare, validazione e successiva operativizzazione. In questo contesto, il notebook deve essere considerato come punto di partenza sperimentale, non come forma finale della soluzione.

4.1.5 Problema

La soluzione deve evolvere senza perdere la capacità dimostrativa del notebook, ma separando le responsabilità applicative in componenti più chiari, testabili e riutilizzabili. Il problema principale è individuare un percorso architetturale che riduca l'accoppiamento senza introdurre una complessità prematura.

In particolare, occorre evitare che logiche appartenenti a layer differenti rimangano mescolate nello stesso file: Data, Feature Engineering, Forecasting, Monitoring, Energy Balance e Visualization devono evolvere con responsabilità distinte. Il testing mantiene una responsabilità trasversale di verifica e non costituisce un layer applicativo della pipeline.

4.1.6 Alternative considerate

A) Mantenere un unico notebook. Questa alternativa preserva la semplicità iniziale e mantiene un unico punto di esecuzione. Tuttavia, non risolve i problemi di manutenibilità, testing, riuso delle funzioni e controllo delle dipendenze tra responsabilità applicative.

B) Trasformare immediatamente il progetto in una architettura a microservizi. Questa alternativa massimizza la separazione fisica dei componenti, ma introduce complessità infrastrutturale, operativa e organizzativa non giustificata dalla maturità attuale della soluzione.

C) Creare una struttura modulare all'interno dello stesso repository GitHub. Questa alternativa consente di separare progressivamente le responsabilità in moduli Python indipendenti, mantenendo un unico repository e una gestione operativa semplice.

4.1.7 Decisione

Si decide di adottare l'alternativa C: creare una struttura modulare all'interno dello stesso repository GitHub.

La soluzione dovrà evolvere da notebook monolitico a repository modulare, mantenendo il notebook come strumento di analisi, demo o orchestrazione sperimentale, ma spostando progressivamente la logica applicativa in moduli dedicati.

4.1.8 Motivazioni

La decisione è motivata dai seguenti elementi:

- **Semplicità operativa**: un unico repository riduce la complessità di configurazione, versionamento e coordinamento.
- **Minore complessità gestionale**: la modularizzazione interna evita l'introduzione prematura di deployment distribuiti, API di comunicazione tra servizi e infrastruttura di orchestrazione.
- **Miglior manutenibilità**: la separazione tra dati, feature engineering, forecasting, bilancio energetico e visualizzazione rende più semplice comprendere e modificare la soluzione.
- **Maggiore testabilità**: funzioni e componenti isolati possono essere validati con test unitari e di integrazione mirati.
- **Evoluzione graduale**: la soluzione potrà eventualmente evolvere verso architetture più complesse solo quando requisiti, carico operativo e vincoli organizzativi lo renderanno necessario.
- **Coerenza con Codex**: una struttura modulare consente a Codex di intervenire su porzioni più circoscritte del codice, riducendo il rischio di modifiche non controllate.

4.1.9 Conseguenze

L'adozione di questa decisione comporta le seguenti conseguenze:

- introduzione progressiva della cartella `src`;
- separazione delle responsabilità applicative in moduli dedicati;
- estrazione delle funzioni di generazione e caricamento dati dal notebook;
- estrazione delle funzioni di preprocessing, scaling e costruzione sequenze;
- isolamento della logica di training, valutazione e inference dei modelli;

- separazione della logica di bilancio energetico dalla logica di forecasting;
- maggiore facilità di testing e validazione automatizzata;
- migliore integrazione con Codex, GitHub e futuri workflow di Continuous Integration.

La modularizzazione dovrà essere incrementale. Non è richiesto un refactoring completo immediato: le estrazioni dovranno seguire priorità tecniche e rischio di regressione.

4.1.10 Impatti futuri

Nel breve periodo, questa decisione abilita la costruzione di una struttura di repository più leggibile, con moduli candidati quali `data_generation`, `data_loading`, `preprocessing`, `features`, `models`, `training`, `evaluation`, `forecasting`, `energy_balance`, `visualization` e `reporting`.

Nel medio periodo, la separazione dei moduli permetterà di introdurre test automatici, validazione delle interfacce, configurazioni esterne e pipeline di esecuzione ripetibili. Nel lungo periodo, se emergeranno requisiti di scalabilità, integrazione enterprise o deployment distribuito, la modularizzazione interna costituirà una base ordinata per un'eventuale evoluzione verso servizi separati.

4.2 ADR-002: Separazione dei modelli di forecasting per consumi e produzione fotovoltaica

4.2.1 Titolo

Separazione dei modelli di forecasting per consumi e produzione fotovoltaica.

4.2.2 Stato

Accettata.

4.2.3 Data

9 giugno 2026.

4.2.4 Contesto

La soluzione di forecasting energetico deve produrre previsioni orarie relative a due grandezze distinte: consumi energetici e produzione fotovoltaica. Tali previsioni vengono successivamente

combinate per calcolare il bilancio energetico, identificando energia da acquistare dalla rete ed energia eventualmente da immettere.

Nel notebook iniziale sono già presenti due percorsi previsionali distinti: uno per il consumo e uno per la produzione. Entrambi utilizzano un approccio LSTM sequence-to-sequence, ma operano su fenomeni fisici e comportamentali differenti. Il consumo è influenzato da pattern di utilizzo, abitudini, cicli giornalieri e possibili fattori produttivi; la produzione fotovoltaica è invece fortemente legata a irraggiamento, condizioni meteorologiche, stagionalità e caratteristiche dell'impianto.

4.2.5 Problema

Occorre stabilire se la soluzione debba utilizzare un unico modello predittivo per stimare simultaneamente consumi e produzione, oppure due modelli separati, ciascuno specializzato su uno dei due fenomeni.

La decisione ha impatti su training, valutazione, manutenzione, interpretabilità e possibilità di miglioramento futuro. Un modello unico potrebbe semplificare l'orchestrazione, ma rischia di mescolare dinamiche differenti. Due modelli distinti richiedono maggiore disciplina nella gestione delle pipeline, ma consentono specializzazione e ottimizzazione indipendente.

4.2.6 Alternative considerate

A) Utilizzo di un unico modello predittivo. Questa alternativa prevede un solo modello incaricato di produrre sia la previsione dei consumi sia la previsione della produzione fotovoltaica. Riduce il numero di artefatti modellistici, ma introduce accoppiamento tra fenomeni con dinamiche differenti.

B) Utilizzo di due modelli distinti. Questa alternativa prevede un modello dedicato al consumo e un modello dedicato alla produzione fotovoltaica. Aumenta la separazione logica e permette ottimizzazioni indipendenti.

4.2.7 Decisione

Si decide di adottare l'alternativa B: utilizzare due modelli distinti, uno per il forecasting dei consumi energetici e uno per il forecasting della produzione fotovoltaica.

I due modelli potranno condividere alcuni componenti tecnici, come funzioni di preparazione sequenze, metriche, callback o utility di scaling, ma dovranno rimanere separati dal punto di vista del training, della valutazione e dell'inference.

4.2.8 Motivazioni

La decisione è motivata dai seguenti elementi:

- **Differenza tra i fenomeni**: i consumi dipendono principalmente da comportamenti umani, profili d'uso, cicli giornalieri e condizioni operative.
- **Dipendenza meteorologica della produzione**: la produzione fotovoltaica dipende prevalentemente da irraggiamento, temperatura, stagionalità e caratteristiche fisiche dell'impianto.
- **Dinamiche temporali differenti**: consumo e produzione possono avere pattern, rumore, variabilità e sensibilità alle feature non coincidenti.
- **Maggiore flessibilità evolutiva**: i due modelli possono evolvere con algoritmi, feature set, finestre temporali e iperparametri differenti.
- **Migliore valutazione**: metriche e analisi degli errori possono essere interpretate separatamente, evitando compensazioni artificiali tra target diversi.
- **Coerenza con il bilancio energetico**: il bilancio deve combinare due previsioni esplicite e interpretabili, non un output opaco derivato da un modello unico.

4.2.9 Conseguenze

L'adozione di questa decisione comporta le seguenti conseguenze:

- introduzione di due pipeline di training separate;
- gestione distinta dei dataset target per consumo e produzione;
- possibilità di ottimizzare indipendentemente architettura, feature e iperparametri dei due modelli;
- separazione delle metriche di valutazione per consumo e produzione;
- maggiore interpretabilità della soluzione complessiva;
- maggiore facilità di manutenzione e sostituzione di un singolo modello senza impattare necessariamente l'altro;
- necessità di orchestrare correttamente le due previsioni prima del calcolo del bilancio energetico.

Questa scelta introduce una leggera complessità aggiuntiva nella pipeline, ma tale complessità è giustificata dalla chiarezza concettuale e dalla maggiore robustezza evolutiva.

4.2.10 Impatti futuri

Nel breve periodo, la separazione dei modelli richiederà di formalizzare interfacce comuni per training, evaluation e inference, in modo che le due pipeline siano coerenti ma indipendenti. Sarà utile definire convenzioni comuni per input, output, metriche, logging e salvataggio degli artefatti.

Nel medio periodo, sarà possibile sperimentare modelli differenti per i due domini, ad esempio architetture più sensibili ai pattern comportamentali per i consumi e modelli più orientati alle variabili meteo per la produzione fotovoltaica. Nel lungo periodo, la separazione consentirà di sostituire o migliorare uno dei due motori previsionali senza riprogettare l'intera soluzione di bilancio energetico.

5 Data Flows

5.1 Introduzione ai flussi informativi

Un sistema di forecasting energetico non è costituito solamente dal modello predittivo. Il modello rappresenta una componente centrale della soluzione, ma il valore operativo nasce dall'intera pipeline che raccoglie, controlla, trasforma, interpreta e rende fruibili i dati. Nel contesto della piattaforma *Energy Forecasting Platform – AI Operational Stack*, il flusso informativo deve quindi essere progettato come una catena coerente di trasformazioni, in cui ogni passaggio aggiunge qualità, significato o utilità decisionale.

La soluzione è dedicata alla previsione oraria multi-serie dei consumi energetici e della produzione fotovoltaica di un prosumer. Questo implica che i dati grezzi provenienti da misure storiche, fonti meteorologiche e sorgenti operative non possano essere inviati direttamente al modello senza un processo preliminare di validazione e preparazione. La pipeline deve invece trasformare progressivamente i dati reali in feature utilizzabili, le feature in previsioni, le previsioni in bilanci energetici e i bilanci in report interpretabili.

Il flusso logico di riferimento è rappresentato nella Figura 1 - Flusso logico della pipeline di forecasting energetico.

Dati reali $\longrightarrow$ Preprocessing $\longrightarrow$ Forecast $\longrightarrow$ Bilancio energetico $\longrightarrow$ Report

Figura 1 - Flusso logico della pipeline di forecasting energetico

Questa rappresentazione sintetica mostra la direzione principale del flusso dati. Il consolidamento successivo formalizza inoltre il Monitoring Layer come responsabilità autonoma, mentre il testing rimane una capacità trasversale pianificata e non una fase del flusso applicativo.

5.2 Acquisizione dei dati reali

La prima fase della pipeline riguarda l'acquisizione dei dati reali. In una soluzione di forecasting energetico per un prosumer, le sorgenti informative principali sono le misure storiche di consumo, le misure storiche di produzione fotovoltaica e i dati meteorologici. Queste informazioni costituiscono la base empirica su cui il sistema apprende i pattern temporali e produce previsioni sulle ore successive.

Le misure storiche di consumo descrivono il comportamento energetico dell'utenza nel tempo. Possono riflettere abitudini giornaliere, cicli settimanali, stagionalità, profili produttivi o variazioni legate alla temperatura. Le misure storiche di produzione fotovoltaica descrivono invece la resa dell'impianto, influenzata soprattutto da irraggiamento, condizioni meteo, orientamento, capacità installata e caratteristiche tecniche del sistema. I dati meteorologici completano il quadro, fornendo variabili esplicative fondamentali per anticipare sia la produzione futura sia, in parte, il consumo.

Nella fase iniziale del progetto, il notebook utilizza dati sintetici e possibili caricamenti da file. In una versione progressivamente più operativa, gli input potranno provenire da file CSV, file Excel o future sorgenti API. I file CSV rappresentano una scelta semplice e facilmente tracciabile per prototipi e prime integrazioni. I file Excel possono essere utili quando i dati sono prodotti o validati manualmente da utenti di business. Le API diventeranno invece rilevanti quando la piattaforma dovrà integrarsi con sistemi esterni, servizi meteo, contatori intelligenti, inverter fotovoltaici o piattaforme energetiche.

Il Data Layer ha il compito di isolare questa complessità. Deve acquisire i dati dalle diverse sorgenti, uniformarne la struttura, controllare la presenza delle colonne attese e produrre dataset coerenti per gli strati successivi. In questo modo, il resto della pipeline non dipende direttamente dal formato originale delle sorgenti, ma lavora su rappresentazioni dati più stabili e controllate.

5.3 Fase di preprocessing

Dopo l'acquisizione, i dati devono essere preparati prima di poter essere utilizzati dai modelli previsionali. La fase di preprocessing è il punto in cui il dato grezzo viene trasformato in dato affidabile e modellabile. Questa fase è particolarmente importante perché errori, discontinuità o incoerenze nei dati possono compromettere la qualità del forecast anche in presenza di un modello predittivo sofisticato.

Il primo passaggio riguarda la validazione dei dati. La pipeline deve verificare che le serie temporali abbiano una frequenza coerente, che i timestamp siano ordinati, che le colonne richieste siano presenti e che i valori rientrino in intervalli plausibili. Una misura di consumo negativa, un valore di produzione fotovoltaica notturna anomalo o un timestamp duplicato sono esempi di condizioni che devono essere intercettate prima dell'addestramento o dell'inference.

La gestione dei valori mancanti è un secondo aspetto essenziale. Nelle serie energetiche reali possono verificarsi assenze dovute a problemi di misura, esportazioni incomplete o interruzioni nei sistemi di raccolta. Il preprocessing deve decidere se imputare tali valori, interpolarli, escludere finestre

temporali non affidabili o segnalare l'anomalia a valle. In modo analogo, la gestione delle anomalie deve distinguere tra eventi reali significativi e dati errati, evitando sia di eliminare informazione utile sia di introdurre rumore nel modello.

Una volta validati e ripuliti, i dati devono essere normalizzati. La normalizzazione consente ai modelli, in particolare alle reti neurali, di lavorare su scale numeriche più stabili. Nel caso del notebook iniziale, questa responsabilità è assolta tramite scaler applicati a feature e target. In una soluzione modulare, tali trasformazioni devono essere rese esplicite, persistibili e riutilizzabili anche in fase di inference.

Il preprocessing include inoltre la creazione delle feature temporali e la costruzione delle sequenze per il modello LSTM. Le feature temporali possono descrivere ora del giorno, giorno della settimana, stagionalità o pattern ricorrenti. La costruzione delle sequenze traduce invece la serie storica in finestre temporali di input e output, coerenti con l'approccio sequence-to-sequence. Nel caso specifico, la logica di riferimento utilizza una finestra storica di 168 ore e produce una previsione sulle successive 168 ore.

Il Feature Engineering Layer ha quindi il ruolo di trasformare i dataset prodotti dal Data Layer in strutture direttamente utilizzabili dal Forecasting Layer. La sua responsabilità non è scegliere il modello, ma costruire input affidabili, coerenti e ripetibili.

5.4 Fase di forecasting

La fase di forecasting utilizza le sequenze e le feature preparate nello strato precedente per generare le previsioni energetiche. In coerenza con le decisioni architetturali già formalizzate, la soluzione prevede due modelli distinti: un modello per i consumi energetici e un modello per la produzione fotovoltaica.

Il modello dei consumi ha il compito di stimare il fabbisogno energetico orario del prosumer. Tale previsione può dipendere da abitudini di utilizzo, cicli produttivi, condizioni climatiche e pattern storici ricorrenti. Il modello della produzione fotovoltaica, invece, deve stimare l'energia generata dall'impianto nelle ore future, con una dipendenza più marcata dalle condizioni meteorologiche e dall'irraggiamento.

La separazione tra i due modelli consente di rispettare la natura differente dei fenomeni osservati. I consumi e la produzione fotovoltaica non sono semplicemente due colonne dello stesso problema: rappresentano dinamiche diverse, con driver differenti e possibili strategie di ottimizzazione indipendenti. Questa impostazione rende più semplice interpretare gli errori, migliorare un modello senza modificare l'altro e introdurre in futuro algoritmi o feature specifiche per ciascun dominio.

L'output atteso della fase di forecasting è una previsione oraria sulle successive 168 ore, pari a sette giorni. Il Forecasting Layer ha il compito di orchestrare il caricamento o l'addestramento dei modelli, applicare le trasformazioni necessarie, produrre le previsioni e restituire risultati in un formato coerente per il calcolo del bilancio energetico.

5.5 Calcolo del bilancio energetico

Le previsioni di consumo e produzione non rappresentano ancora, da sole, l'informazione operativa finale. Il loro valore emerge quando vengono combinate per calcolare il bilancio energetico del prosumer. Questa fase trasforma l'output statistico dei modelli in indicatori comprensibili e utilizzabili per prendere decisioni.

Il bilancio energetico netto, denominato canonicamente `net_balance`, è calcolato come produzione prevista meno consumo previsto. La componente positiva costituisce `grid_export`; la componente negativa, cambiata di segno, costituisce `grid_import`. Il valore zero rappresenta equilibrio e non richiede una classificazione obbligatoria.

Gli output formalmente previsti sono `net_balance`, `grid_import`, `grid_export`, i totali dei due flussi e i rispettivi valori cumulativi. Il layer non introduce automaticamente KPI economici, ambientali o di ottimizzazione.

L'Energy Balance Layer ha quindi il compito di separare la previsione dalla sua interpretazione energetica. Questa separazione è importante perché consente di modificare o migliorare i modelli previsionali senza riscrivere le regole di calcolo del bilancio, e viceversa. Inoltre, rende più trasparente il passaggio da dato predetto a informazione decisionale.

5.6 Produzione dei report

L'ultima fase della pipeline riguarda la presentazione dei risultati. Anche una previsione accurata perde valore se non viene comunicata in modo chiaro e azionabile. La piattaforma deve quindi produrre output leggibili per utenti tecnici, stakeholder di progetto e decisori operativi.

I report possono assumere forme differenti. Una dashboard può offrire una vista su consumi previsti, produzione prevista, `grid_import`, `grid_export` e andamento cumulato. I grafici utilizzano risultati già calcolati e non devono ricostruire formule o nomenclature alternative.

Il Visualization Layer ha il compito di trasformare i dati finali della pipeline in rappresentazioni comprensibili. Questo layer non dovrebbe contenere logica di preprocessing, training o calcolo energetico, ma limitarsi a rendere accessibili i risultati. In questo modo, dashboard, grafici e report possono evolvere senza alterare la logica computazionale sottostante.

5.7 Tracciabilità del flusso dati

La tracciabilità è un requisito trasversale dell'intera pipeline. Ogni trasformazione deve essere documentata e verificabile, perché la qualità della previsione dipende non solo dal modello, ma da tutte le scelte applicate ai dati prima e dopo l'inference.

Documentare il flusso dati significa sapere da quale sorgente proviene un dataset, quali controlli sono stati applicati, come sono stati gestiti valori mancanti e anomalie, quali scaler sono stati usati,

quale modello ha prodotto una specifica previsione e quali regole hanno trasformato la previsione in bilancio energetico. Questa capacità è essenziale per il debugging, la validazione, l'audit tecnico e la gestione delle regressioni.

In prospettiva operativa, la tracciabilità abilita anche una migliore collaborazione tra ChatGPT, Codex, Prism e GitHub. ChatGPT può supportare l'analisi concettuale, Codex può intervenire su moduli circoscritti, Prism può mantenere la documentazione allineata e GitHub può tracciare decisioni, issue, versioni e modifiche nel tempo.

5.8 Conclusioni

Il valore della piattaforma di forecasting energetico non deriva solamente dal modello predittivo, ma dall'intera catena di elaborazione che trasforma dati grezzi in informazioni utilizzabili dal cliente. Il modello è una componente importante, ma deve essere inserito in una pipeline più ampia, controllata e tracciabile.

Il flusso consolidato procede da Data e Feature Engineering verso Forecasting. A valle delle previsioni si dirama verso Monitoring, Energy Balance e Visualization; Monitoring ed Energy Balance possono fornire ulteriori risultati al Visualization Layer senza dipendere l'uno dall'altro. La strategia di testing opera trasversalmente senza modificare il flusso applicativo.

Questa impostazione consente alla soluzione di evolvere da prototipo notebook-based a piattaforma modulare, mantenendo coerenza con lo Stack Operativo Ottimizzato e preparando il progetto a successive fasi di implementazione, validazione e operativizzazione.

6 Repository Structure

6.1 Introduzione

Il repository GitHub `AI-IT-stack-lab` non rappresenta semplicemente un archivio di file, ma costituisce l'ambiente operativo dell'intero progetto *Energy Forecasting Platform – AI Operational Stack*. Al suo interno convivono codice, notebook, dati, modelli, esperimenti, documentazione tecnica e conoscenza progettuale. Questa impostazione rende il repository il punto di raccordo tra analisi, progettazione, implementazione, validazione e manutenzione continua.

Nel contesto dello Stack Operativo Ottimizzato, il repository assume quindi un ruolo più ampio rispetto a quello di semplice contenitore di sorgenti. Esso diventa il luogo in cui vengono conservate le decisioni architetturali, le versioni sperimentali, gli artefatti di modello, i dataset, i report e la documentazione prodotta con il supporto di Prism. La struttura del repository deve perciò essere leggibile, tracciabile e coerente con l'evoluzione progressiva della soluzione.

La Figura 1 - Struttura logica del repository AI-IT-stack-lab rappresenta la gerarchia principale del repository e mette in evidenza il ruolo centrale della cartella `07_projects/forecasting_energy`,

che ospita la soluzione applicativa destinata a evolvere dal notebook iniziale verso una piattaforma modulare.

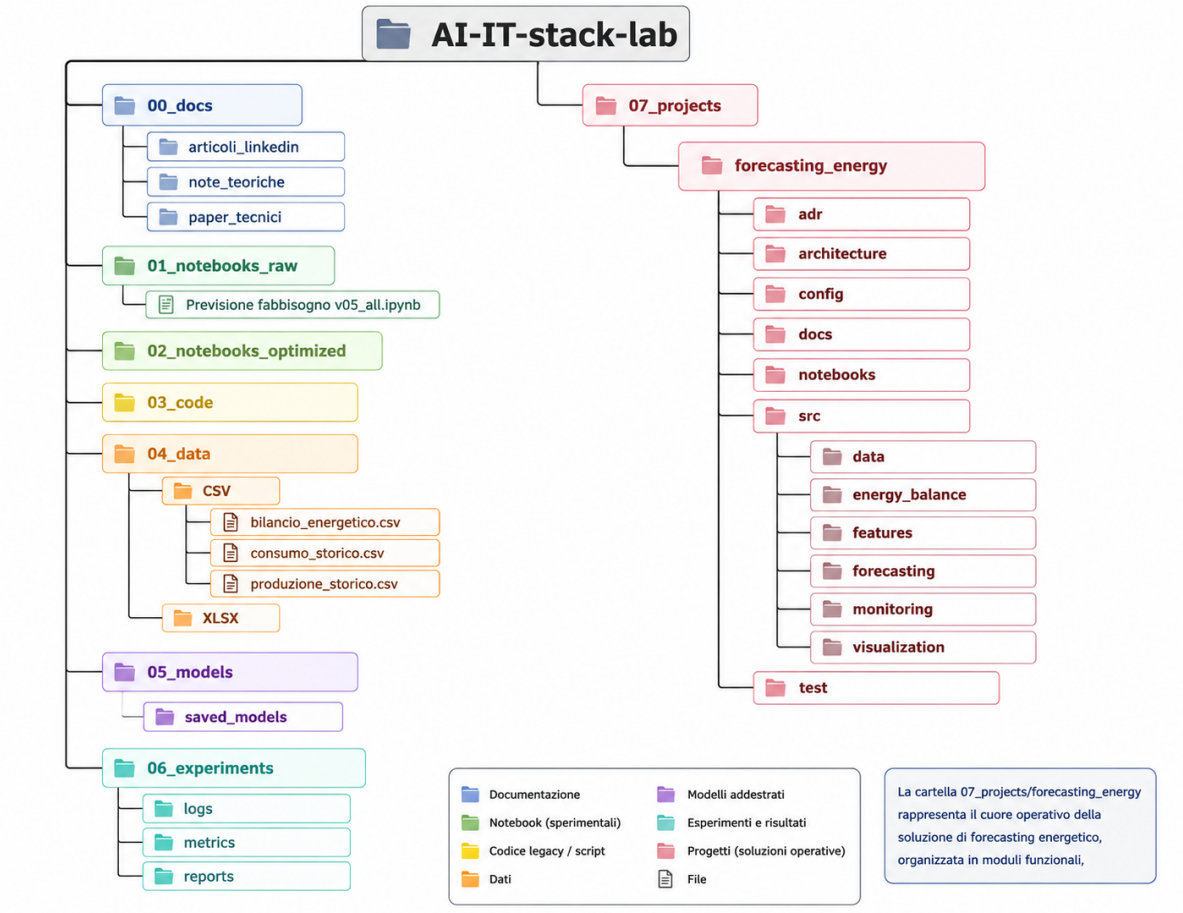


Figura 1 - Struttura logica del repository AI-IT-stack-lab

6.2 Organizzazione generale del repository

L'organizzazione generale del repository è basata su macro-cartelle numerate, ciascuna con una responsabilità prevalente. Questa scelta rende più semplice orientarsi nel progetto e consente di separare gli artefatti in base alla loro natura: documentazione, notebook, codice, dati, modelli, esperimenti e progetti applicativi.

La cartella `00_docs` raccoglie la conoscenza documentale generale. La cartella `01_notebooks_raw` conserva i notebook originali, così come prodotti nella fase iniziale o ricevuti come materiale di partenza. La cartella `02_notebooks_optimized` è destinata alle versioni migliorate dei notebook, progressivamente ripulite, commentate o riorganizzate. La cartella `03_code` rappresenta l'area generale per codice riutilizzabile o componenti condivisi.

La cartella `04_data` contiene i dataset utilizzati o prodotti dalla soluzione, distinguendo i formati e preparando una gestione più ordinata delle sorgenti. La cartella `05_models` conserva gli artefatti dei modelli addestrati. La cartella `06_experiments` contiene log, metriche e report sperimentali. Infine, `07_projects` ospita i progetti applicativi veri e propri, tra cui `forecasting_energy`, che costituisce il contenitore operativo della piattaforma di forecasting energetico.

Questa suddivisione migliora manutenzione, tracciabilità e riutilizzo perché riduce l'ambiguità sul posizionamento degli artefatti. Ogni file ha una collocazione coerente con il proprio ruolo e può essere versionato, discusso e validato in modo più ordinato.

6.3 Area documentale

L'area `00_docs` è dedicata alla documentazione generale e alla produzione di contenuti collegati al progetto. Essa non contiene necessariamente codice eseguibile, ma rappresenta una base di conoscenza utile per contestualizzare le scelte tecniche e comunicare i risultati a pubblici differenti.

La sottocartella `articoli_linkedin` può raccogliere contenuti divulgativi, sintesi pubblicabili e materiali orientati alla comunicazione esterna. La sottocartella `note_teoriche` è pensata per conservare appunti metodologici, spiegazioni concettuali e riflessioni sull'uso combinato di ChatGPT, Codex, Prism e GitHub. La sottocartella `paper_tecnici` può contenere documenti più strutturati, analisi estese e materiali tecnici destinati ad approfondimenti o condivisioni professionali.

Questa area è strettamente collegata a Prism, che svolge il ruolo di ambiente per scrittura, revisione e organizzazione della documentazione. Prism consente di mantenere allineati i documenti tecnici con l'evoluzione della soluzione, trasformando il repository in un luogo in cui non viene conservato solo codice, ma anche la conoscenza progettuale che ne giustifica l'evoluzione.

6.4 Notebook originali e notebook ottimizzati

La cartella `01_notebooks_raw` conserva le versioni originali dei notebook. Nel caso specifico, contiene `Previsione fabbisogno v05_all.ipynb`, che rappresenta l'implementazione iniziale della soluzione di forecasting energetico. Questa cartella ha valore storico e metodologico: permette di risalire allo stato di partenza, confrontare le evoluzioni e comprendere quali parti della soluzione siano state progressivamente estratte o migliorate.

La cartella `02_notebooks_optimized` è invece destinata alle versioni progressivamente migliorate attraverso ChatGPT e Codex. Qui possono essere conservati notebook più leggibili, ripuliti da codice ridondante, arricchiti da commenti, organizzati per sezioni o trasformati in strumenti dimostrativi. Questa distinzione evita di sovrascrivere il materiale originale e consente di mantenere una traccia chiara del percorso evolutivo.

In prospettiva, il notebook ottimizzato non dovrebbe diventare l'unico luogo in cui vive la logica applicativa stabile. Esso può rimanere uno strumento di sperimentazione e comunicazione, mentre il codice maturo viene progressivamente trasferito nei moduli Python del progetto applicativo.

6.5 Gestione dati

La cartella `04_data` è dedicata alla gestione dei dati utilizzati e prodotti dalla piattaforma. Essa contiene due sottocartelle principali: `CSV` e `XLSX`. Questa distinzione consente di gestire formati diversi senza mescolarli con codice, modelli o documentazione.

La sottocartella `CSV` include i dataset storici di consumo e produzione e può ospitare output esportati dall'orchestratore. L'eventuale persistenza del bilancio energetico avviene fuori dai moduli del layer energetico. In particolare, non appartiene a `balance_calculator.py` o `kpi.py`. La sottocartella `XLSX` è destinata a eventuali fogli Excel, spesso utili quando i dati vengono forniti da utenti business, validati manualmente o scambiati in contesti non ancora automatizzati.

Nel repository è importante distinguere tra dati grezzi, dati elaborati e dataset di training. I dati grezzi sono vicini alla sorgente originale e devono essere preservati senza trasformazioni non documentate. I dati elaborati sono il risultato di pulizia, normalizzazione o aggregazione. I dataset di training sono versioni specifiche, coerenti e riproducibili, utilizzate per addestrare o validare i modelli. Questa distinzione favorisce audit, debugging e riproducibilità degli esperimenti.

6.6 Gestione modelli

La cartella `05_models/saved_models` è destinata alla conservazione dei modelli addestrati. Nel contesto della piattaforma, essa può contenere i modelli per il forecasting dei consumi e della produzione fotovoltaica, insieme agli eventuali artefatti necessari per l'inference, come scaler, configurazioni o metadati di training.

La tracciabilità delle versioni dei modelli è un requisito fondamentale. Un modello non dovrebbe essere considerato solo come un file binario, ma come il risultato di una specifica combinazione di dati, feature, codice, iperparametri e metriche. Senza questa tracciabilità, diventa difficile capire quale modello abbia generato una previsione, confrontare esperimenti o riprodurre un risultato.

In una fase più matura, ogni modello salvato dovrebbe essere associato a informazioni minime quali data di training, versione del codice, dataset utilizzato, metriche ottenute e configurazione applicata. Questo approccio rende più affidabile il passaggio da sperimentazione a uso operativo.

6.7 Esperimenti e metriche

La cartella `06_experiments` raccoglie le informazioni prodotte durante l'esecuzione degli esperimenti. Le sottocartelle `logs`, `metrics` e `reports` permettono di separare tre tipologie di output complementari.

I `logs` documentano l'esecuzione delle pipeline, registrando eventi, errori, tempi di esecuzione e informazioni diagnostiche. Le `metrics` raccolgono indicatori quantitativi di qualità, come MAE, RMSE o altre misure di accuratezza applicate ai modelli di consumo e produzione. I `reports` sintetizzano invece i risultati in forma più leggibile, utile per revisione tecnica, confronto tra esperimenti

e comunicazione agli stakeholder.

Questa area supporta il monitoraggio della qualità della soluzione. La piattaforma non deve limitarsi a produrre previsioni, ma deve consentire di capire se tali previsioni migliorano, peggiorano o cambiano nel tempo. Esperimenti e metriche sono quindi essenziali per gestire regressioni, validare refactoring e decidere quando un modello sia sufficientemente stabile per l'uso operativo.

6.8 Il progetto Forecasting Energy

La cartella `07_projects/forecasting_energy` costituisce il contenitore operativo della soluzione di forecasting energetico. A differenza delle aree generali del repository, questa cartella ospita gli artefatti specifici del progetto applicativo: documentazione, configurazioni, notebook, codice sorgente e test.

La sottocartella `adr` raccoglie le Architecture Decision Records, cioè le decisioni architetturali rilevanti e le relative motivazioni. La sottocartella `architecture` contiene la documentazione architetturale, inclusi layer, responsabilità, dipendenze e scelte progettuali. La cartella `config` è destinata a parametri applicativi, configurazioni di training, percorsi e valori controllati. La cartella `docs` conserva documentazione specifica del progetto. La cartella `notebooks` ospita notebook collegati alla soluzione, mantenendoli separati dal codice sorgente stabile.

Il cuore operativo è la cartella `src`, che contiene i moduli Python della piattaforma. Il materiale fornito include il sorgente di una suite automatica specifica per il Visualization Layer, ma non documenta il completamento della suite iniziale trasversale prevista dalla Issue #6. Questa organizzazione rende il progetto compatibile con un'evoluzione progressiva: dal notebook monolitico a una soluzione modulare, testabile e mantenibile.

6.9 Struttura del codice sorgente

La cartella `src` è articolata in moduli coerenti con i layer architetturali già definiti. Ogni modulo ha una responsabilità specifica, input e output attesi, e una relazione chiara con gli altri componenti della pipeline.

`src/data` è responsabile dell'acquisizione, caricamento, generazione sintetica e validazione iniziale dei dati. I suoi input possono essere file CSV, dataset sintetici o future sorgenti esterne. I suoi output sono dataframe puliti, validati e coerenti, pronti per il feature engineering. Questo modulo alimenta direttamente `src/features`.

`src/features` contiene la logica di trasformazione dei dati in input modellabili. Riceve dati validati da `src/data` e produce feature, target scalati, sequenze temporali e strutture encoder-decoder per i modelli LSTM. È il punto di collegamento tra dati grezzi e forecasting.

`src/forecasting` contiene la logica predittiva. Riceve sequenze e feature dal modulo `features`, gestisce costruzione del modello, training e inference dei modelli di consumo e produzione fo-

tovoltaica, e produce previsioni orarie sulle successive 168 ore. I suoi output alimentano sia `src/monitoring` sia `src/energy_balance` e possono essere rappresentati da `src/visualization`.

`src/energy_balance` è destinato a trasformare le previsioni nei campi canonici `net_balance`, `grid_import` e `grid_export`, oltre ai relativi totali e cumulativi. I moduli non gestiscono file I/O e rappresentano il ponte tra previsione statistica e interpretazione energetica.

`src/monitoring` implementa la valutazione quantitativa delle previsioni. Riceve valori osservati e predetti già espressi nella stessa scala fisica; `metrics.py` calcola MAE e RMSE, mentre `evaluation.py` orchestra il confronto e restituisce il risultato strutturato. Logging, drift monitoring, alerting, storicizzazione e retraining automatico non appartengono all'implementazione corrente.

Il materiale fornito documenta il modulo `src/visualization/plotting.py`, con le sette API del Capitolo 22, e contiene la suite specifica `tests/test_visualization_plotting.py`. Queste fonti attestano la disponibilità dei contenuti implementativi esaminati, ma non consentono di verificarne la presenza nel branch ufficiale, né lo stato attuale della Issue #5 o della relativa Pull Request. Dashboard, reporting e salvataggio dei file rimangono esterni al modulo.

Questa suddivisione mantiene il codice coerente con Data, Feature Engineering, Forecasting, Monitoring, Energy Balance e Visualization Layer. La suite specifica di test visuali è disponibile nel materiale fornito; la suite iniziale dell'intera piattaforma rimane un'attività trasversale distinta e non può considerarsi completata sulla sola base di questi test.

6.10 Evoluzione dal notebook monolitico alla struttura modulare

La struttura del repository supporta l'evoluzione progressiva dal notebook monolitico alla soluzione modulare. Il notebook rimane disponibile per sperimentazione, dimostrazione e analisi esplorativa, ma non deve rimanere l'unico luogo in cui vive la logica stabile della piattaforma.

Il codice maturo viene progressivamente trasferito nei moduli Python. Le funzioni di caricamento dati migrano verso `src/data`, le trasformazioni verso `src/features`, la logica predittiva verso `src/forecasting`, il calcolo energetico verso `src/energy_balance` e le funzioni grafiche verso `src/visualization`. La composizione e la persistenza dei report restano responsabilità dell'orchestrazione.

Questa evoluzione non richiede una riscrittura immediata dell'intera soluzione. Al contrario, consente un refactoring incrementale e controllato, coerente con le decisioni architetturali già formalizzate negli ADR. La manutenibilità aumenta nel tempo perché ogni componente diventa più piccolo, più leggibile e più verificabile.

6.11 Benefici architetturali

La struttura proposta produce diversi benefici architetturali. La separazione delle responsabilità riduce l'accoppiamento tra dati, modelli, bilancio energetico e visualizzazione. La riusabilità aumenta perché funzioni e moduli possono essere richiamati da notebook, script, test o future pipeline operative. La scalabilità migliora perché la soluzione può evolvere per parti, senza dover modificare l'intero sistema a ogni cambiamento.

La facilità di testing è uno dei benefici principali. Un modulo isolato può essere validato con test specifici: caricamento dati, costruzione sequenze, calcolo metriche, previsione, bilancio energetico e generazione report. Anche la manutenzione diventa più semplice, perché bug e regressioni possono essere localizzati in aree precise del codice.

La struttura è inoltre coerente con l'uso di Codex e GitHub. Codex può lavorare su issue circoscritte, intervenendo su moduli specifici senza dover manipolare un notebook monolitico. GitHub può tracciare modifiche, pull request, discussioni, versioni e decisioni, rendendo il progetto più governabile e collaborativo.

6.12 Conclusioni

La struttura del repository `AI-IT-stack-lab` costituisce la base organizzativa dell'intero Stack Operativo Ottimizzato. Essa consente di mantenere insieme documentazione, notebook, dati, modelli, esperimenti e codice applicativo, senza confondere le rispettive responsabilità.

Nel progetto *Energy Forecasting Platform – AI Operational Stack*, questa organizzazione prepara il passaggio dalla sperimentazione alla costruzione di una piattaforma modulare. La cartella `07_projects/forecasting_energy` diventa il centro operativo della soluzione, mentre le altre aree del repository supportano conoscenza, dati, modelli ed esperimenti.

Questa impostazione prepara il progetto alle successive fasi di implementazione, validazione e manutenzione continua, mantenendo coerenza con l'architettura logica, gli ADR e il flusso dati già definiti nei documenti precedenti.

7 Development Roadmap

7.1 Introduzione

La roadmap di sviluppo rappresenta il collegamento operativo tra le decisioni architetturali già documentate e le attività concrete di implementazione. Dopo aver definito l'architettura logica, gli ADR, i flussi dati e la struttura del repository, è necessario tradurre tali elementi in un percorso progressivo di sviluppo, validazione e consolidamento della piattaforma.

Nel progetto *Energy Forecasting Platform – AI Operational Stack*, la roadmap ha il compito di

guidare l'evoluzione da notebook monolitico a soluzione modulare. Essa non deve essere interpretata come una sequenza rigida di attività isolate, ma come un piano incrementale che consente di ridurre il rischio tecnico, mantenere la continuità sperimentale e trasformare progressivamente la conoscenza già presente nel notebook in componenti software riutilizzabili.

Il repository GitHub `AI-IT-stack-lab` diventa il luogo in cui questa evoluzione viene tracciata, discussa e versionata. La roadmap consente quindi di passare dalla progettazione alla realizzazione, mantenendo coerenza con lo Stack Operativo Ottimizzato e preparando il progetto allo Strato 3, dedicato alla fase di implementazione.

7.2 Situazione iniziale

La situazione iniziale è rappresentata da un notebook di forecasting energetico multi-serie che contiene l'intero flusso applicativo: generazione o caricamento dei dati, preprocessing, costruzione delle sequenze, addestramento dei modelli, previsione, calcolo del bilancio energetico e visualizzazione dei risultati.

Questa impostazione è utile nella fase esplorativa, perché permette di dimostrare rapidamente il funzionamento end-to-end della soluzione. Tuttavia, con l'aumentare della complessità, il notebook monolitico presenta limiti evidenti: manutenzione più difficile, bassa testabilità, accoppiamento tra responsabilità diverse, difficoltà nel riutilizzo del codice e maggiore rischio di regressioni durante modifiche o refactoring.

La necessità principale è quindi introdurre una modularizzazione progressiva. Lo stato architetturale distingue Data Layer, Feature Engineering Layer, Forecasting Layer, Monitoring Layer, Energy Balance Layer e Visualization Layer. Il materiale disponibile ora documenta codice e test specifici per il Visualization Layer; la suite iniziale di test dell'intera piattaforma rimane pianificata nella Issue #6 e non costituisce un layer applicativo.

7.3 Strategia di evoluzione

La strategia di evoluzione adottata è incrementale. Non si prevede una riscrittura immediata dell'intera soluzione, perché un intervento di questo tipo aumenterebbe il rischio tecnico e renderebbe più difficile verificare la correttezza dei risultati. Al contrario, il notebook originale deve essere conservato come baseline storica, strumento di confronto e ambiente di sperimentazione.

Il codice stabile viene progressivamente migrato dal notebook verso moduli Python dedicati. Ogni migrazione deve riguardare una responsabilità circoscritta, deve essere accompagnata da controlli minimi di correttezza e deve mantenere la possibilità di confrontare i risultati con il comportamento del notebook originale.

Questo approccio riduce il rischio perché consente di validare una parte della pipeline alla volta. Inoltre, rende più semplice assegnare attività a Codex, tracciare modifiche in GitHub e aggiornare la documentazione in Prism. La roadmap diventa quindi uno strumento di governo tecnico: definisce

priorità, deliverable e sequenza logica delle attività.

7.4 Fase 1 – Data Layer

La prima fase riguarda il Data Layer. L'obiettivo è estrarre dal notebook le logiche di caricamento, generazione e validazione iniziale dei dati, separandole dalle fasi successive di preprocessing e modellazione.

Questa fase deve introdurre componenti in grado di leggere misure storiche di consumo, misure storiche di produzione fotovoltaica e dati meteorologici. La piattaforma dovrà supportare inizialmente file locali, come CSV o Excel, ma la struttura dovrà rimanere predisposta per future integrazioni tramite API o sistemi esterni.

Un aspetto centrale è la validazione del dataset. Prima che i dati vengano utilizzati dal Feature Engineering Layer, è necessario controllare la presenza delle colonne attese, la coerenza temporale, la frequenza delle osservazioni e la presenza di valori mancanti o anomalie evidenti. Il Data Layer deve quindi produrre dati uniformi e affidabili, riducendo la dipendenza del resto della pipeline dal formato delle sorgenti originarie.

I deliverable implementati della Fase 1 sono `loader.py`, `validator.py` e `synthetic_data.py`. Il primo gestisce il caricamento dei dataset, il secondo centralizza i controlli di validazione e il terzo preserva la generazione sintetica presente nel notebook. `weather_loader.py` apparteneva alla pianificazione iniziale, ma non è stato implementato e rimane una possibile estensione futura.

7.5 Fase 2 – Feature Engineering Layer

La seconda fase riguarda il Feature Engineering Layer. L'obiettivo è trasferire in moduli dedicati le trasformazioni che rendono i dati utilizzabili dai modelli previsionali.

Questa fase include preprocessing, normalizzazione e costruzione delle sequenze temporali. Il preprocessing deve gestire valori mancanti, anomalie, ordinamento temporale e coerenza delle serie. La normalizzazione deve applicare trasformazioni numeriche coerenti tra training e inference, rendendo esplicito l'uso degli scaler. La costruzione delle sequenze temporali deve trasformare le serie storiche in finestre compatibili con il modello LSTM sequence-to-sequence.

Il Feature Engineering Layer ha una responsabilità delicata: non deve contenere logica di training, ma deve produrre input affidabili per il Forecasting Layer. La correttezza di questa fase è essenziale perché errori nella costruzione delle feature o delle sequenze possono compromettere la qualità delle previsioni anche se il modello è formalmente corretto.

I deliverable principali della Fase 2 sono `preprocessing.py`, `scaling.py` e `sequence_builder.py`. Questi moduli consentiranno di isolare le trasformazioni, renderle testabili e riutilizzarle in modo coerente.

7.6 Fase 3 – Forecasting Layer

La terza fase riguarda il Forecasting Layer. L'obiettivo è estrarre e organizzare la logica predittiva, separando la definizione dei modelli, il training e l'inference.

In coerenza con gli ADR già formalizzati, la soluzione dovrà mantenere due percorsi previsionali distinti: un modello per i consumi energetici e un modello per la produzione fotovoltaica. Questa separazione consente di rispettare la differente natura dei due fenomeni e di ottimizzare i modelli in modo indipendente.

La fase include la definizione dell'architettura LSTM, la gestione del training, l'applicazione delle callback e la generazione delle previsioni sulle successive 168 ore. La valutazione quantitativa è separata nel Monitoring Layer; la logica di inference riceve input preparati dal Feature Engineering Layer e produce output utilizzabili dai layer successivi.

I deliverable principali della Fase 3 sono `lstm_model.py`, `trainer.py` e `predictor.py`. Il primo modulo definisce l'architettura del modello, il secondo governa il processo di addestramento, mentre il terzo gestisce la produzione delle previsioni.

7.7 Fase 4 – Monitoring Layer

La quarta fase architetturale riguarda il Monitoring Layer, implementato nei moduli `metrics.py` ed `evaluation.py`. Il primo calcola MAE e RMSE; il secondo orchestra il confronto tra valori osservati e predetti già riportati alla scala fisica originale.

Il Monitoring Layer è un layer ufficiale e implementato. È stato realizzato mediante un'attività non numerata emersa dopo la definizione del backlog iniziale; questa particolarità riguarda soltanto la tracciabilità storica e non la sua posizione nell'architettura. Non viene associato retroattivamente a una nuova Issue, così da preservare la sequenza delle Issue #1–#6.

7.8 Fase 5 – Energy Balance Layer

La quinta fase riguarda l'Energy Balance Layer. L'obiettivo è trasformare le previsioni di consumo e produzione in informazioni operative per il prosumer.

Questa fase deve calcolare `net_balance` come produzione meno consumo e derivare `grid_export` e `grid_import`. I KPI approvati sono i totali e i cumulativi dei due flussi. Il valore zero non richiede classificazione e gli input non devono essere sottoposti a clipping automatico.

La separazione di questo layer è importante perché evita di mescolare la logica predittiva con la logica energetica. I modelli producono forecast; l'Energy Balance Layer interpreta tali forecast in termini operativi. Questa distinzione migliora leggibilità, testabilità e manutenibilità.

I deliverable definiti per la Fase 5 sono `balance_calculator.py` e `kpi.py`. Il primo applica le

formule canoniche per intervallo; il secondo calcola totali e cumulativi. CSV e file I/O rimangono esterni ai moduli.

7.9 Fase 6 – Visualization Layer

La sesta fase riguarda il Visualization Layer. L'obiettivo è trasformare risultati previsionali, valutazioni quantitative, bilanci e KPI in rappresentazioni comprensibili per utenti tecnici e decisori operativi.

Questa fase centralizzerà le rappresentazioni grafiche di risultati già calcolati. Il layer non produrrà dashboard o logiche operative autonome nell'implementazione iniziale.

Il Visualization Layer deve rimanere separato dalla logica di calcolo. Esso riceve dati già elaborati e li trasforma in output comunicativi, senza introdurre trasformazioni applicative non tracciate.

I sorgenti forniti per la v15 documentano `plotting.py` con le sette API del Capitolo 22 e la suite `tests/test_visualization_plotting.py`. Il risultato di verifica comunicato è di 18 test superati in 5,14 secondi con backend Agg; è stato inoltre comunicato che `git diff --check` non ha rilevato errori, salvo avvisi Git di conversione LF/CRLF. Queste evidenze attestano il perimetro tecnico descritto, non l'integrazione nel repository ufficiale. La v14 riportava la Issue #5 aperta; stato amministrativo attuale, Pull Request, review e merge non sono verificati. Dashboard o integrazioni operative rimangono evoluzioni successive.

7.10 Attività trasversale pianificata – Initial Test Suite

Il materiale fornito include una suite specifica per il Visualization Layer, ma non dimostra il completamento della suite iniziale trasversale prevista dalla Issue #6. Nel backlog documentale la Issue #6 resta quindi aperta/pianificata; il suo stato amministrativo GitHub non è verificato. La suite visuale non costituisce un layer applicativo e non sostituisce unit test, integration test e regressioni per il resto della piattaforma.

Gli unit test e gli integration test dovranno verificare i contratti dei layer senza partecipare al flusso applicativo. La futura validazione della pipeline completa sarà essenziale per garantire che la modularizzazione non alteri il comportamento atteso della soluzione.

7.11 Introduzione CI/CD

La Continuous Integration dovrà essere introdotta progressivamente tramite GitHub Actions. In una prima fase, il workflow potrà limitarsi all'esecuzione automatica dei test. Successivamente, potrà includere controlli di qualità, validazione della struttura del repository e verifica della documentazione.

I quality gate rappresentano condizioni minime che il progetto deve rispettare prima di accettare

modifiche. Essi possono includere test superati, assenza di errori critici, rispetto delle convenzioni di struttura e aggiornamento della documentazione quando necessario.

L'obiettivo non è introdurre complessità DevOps prematura, ma garantire che ogni evoluzione del codice sia verificabile, ripetibile e coerente con le decisioni progettuali.

7.12 Ruolo di Codex

Codex assume un ruolo operativo nella trasformazione della roadmap in attività implementative. Può essere utilizzato per estrarre moduli dal notebook, proporre refactoring, generare test, correggere difetti e preparare modifiche coerenti con le issue di GitHub.

Il valore di Codex aumenta quando il lavoro è scomposto in task piccoli e ben descritti. Una roadmap modulare consente di assegnare attività precise, come implementare `sequence_builder.py`, aggiungere test per `balance_calculator.py` o separare la logica di caricamento dati. In questo modo, Codex può produrre modifiche più controllabili e più facilmente revisionabili.

7.13 Ruolo di ChatGPT

ChatGPT supporta la fase architetturale, la revisione tecnica e la verifica di coerenza progettuale. Può aiutare a formalizzare decisioni, chiarire alternative, individuare rischi e trasformare osservazioni tecniche in documentazione strutturata.

Nel contesto della roadmap, ChatGPT contribuisce a mantenere allineati obiettivi, layer, deliverable e priorità. Può inoltre supportare la preparazione delle issue operative da assegnare a Codex, garantendo che ogni task sia collegato a una motivazione architetturale e a un risultato verificabile.

7.14 Ruolo di Prism

Prism svolge il ruolo di ambiente documentale del progetto. Consente di consolidare la conoscenza, aggiornare la documentazione e mantenere tracciabili le decisioni prese durante l'evoluzione della soluzione.

La roadmap deve rimanere allineata con l'avanzamento del progetto. Quando un modulo viene implementato, una decisione viene modificata o una pipeline viene validata, Prism permette di aggiornare la documentazione in modo coerente. In questo senso, Prism non è soltanto uno strumento di scrittura, ma un componente dello Stack Operativo Ottimizzato dedicato alla memoria progettuale.

7.15 Conclusioni

La roadmap descritta in questo documento trasforma l'architettura in attività implementative. Essa prepara il passaggio verso lo Strato 3 dello Stack Operativo Ottimizzato, in cui le decisioni progettuali vengono convertite in codice, test, pipeline e artefatti operativi.

Il percorso proposto consente di evolvere il notebook monolitico in una piattaforma modulare, riducendo il rischio e preservando la continuità sperimentale. Ogni fase produce deliverable concreti e allineati ai layer architetturali già definiti.

La tabella seguente sintetizza la roadmap di implementazione.

Roadmap di Implementazione

Fase	**Layer**	**Obiettivo principale**	**Deliverable**
1	Data Layer	Caricamento, validazione e generazione sintetica. `weather_loader.py` resta pianificato ma non implementato.	`loader.py`, `validator.py`, `synthetic_data.py`
2	Feature Engineering Layer	Gestire preprocessing, normalizzazione e costruzione delle sequenze temporali.	`preprocessing.py`, `scaling.py`, `sequence_builder.py`
3	Forecasting Layer	Separare modelli, training e inferenza per consumi e produzione fotovoltaica.	`lstm_model.py`, `trainer.py`, `predictor.py`
4	Monitoring Layer	Calcolare MAE e RMSE e orchestrare la valutazione quantitativa.	`metrics.py`, `evaluation.py`
5	Energy Balance Layer	Calcolare `net_balance`, `grid_import`, `grid_export`, totali e cumulativi.	`balance_calculator.py`, `kpi.py`
6	Visualization Layer	Produrre rappresentazioni grafiche di risultati già calcolati.	Codice e test specifici presenti nel materiale fornito; integrazione non verificata
T	Testing trasversale pianificato	Introdurre progressivamente unit test, integration test e controlli di regressione.	Issue #6; suite visuale specifica disponibile, copertura complessiva ancora da realizzare/verificare

8 GitHub Backlog

8.1 Introduzione

Il backlog GitHub rappresenta il collegamento operativo tra la progettazione architetturale e lo sviluppo concreto della piattaforma. Dopo aver definito i requisiti, l'architettura a layer, gli ADR, i flussi dati, la struttura del repository e la roadmap di evoluzione, il passo successivo consiste nel trasformare tali decisioni in unità di lavoro eseguibili, tracciabili e verificabili.

Nel progetto *Energy Forecasting Platform – AI Operational Stack*, il backlog assume quindi il ruolo di ponte tra Strato 2, dedicato alla progettazione e alla pianificazione, e Strato 3, dedicato all'implementazione reale. Le attività non vengono descritte come indicazioni generiche, ma come GitHub Issue collegate a componenti specifici dell'architettura e a deliverable software misurabili.

Le GitHub Issue costituiscono l'unità minima di lavoro tracciabile. Ogni Issue descrive un obiettivo tecnico, chiarisce il perimetro dell'intervento, identifica i file o moduli attesi e consente di seguire lo stato di avanzamento dall'apertura del task fino alla sua chiusura tramite Pull Request. Questo approccio rende il lavoro più controllabile, facilita la collaborazione tra strumenti e persone e prepara il repository `AI-IT-stack-lab` a un processo di sviluppo incrementale supportato da Codex.

8.2 Organizzazione del backlog

Il backlog consolidato mantiene sei Issue numerate: Data Layer, Feature Engineering Layer, Forecasting Layer, Energy Balance Layer, Visualization Layer e suite iniziale di test. Il Monitoring Layer è un layer ufficiale e implementato, ma non dispone di una Issue numerata nel backlog storico: la sua realizzazione viene quindi registrata come attività non numerata, senza creare retroattivamente una nuova Issue.

La scelta di creare una Issue per ciascun macro-componente permette di mantenere chiaro il confine delle responsabilità. Ogni attività deve poter essere compresa, implementata e verificata senza richiedere una riscrittura completa del progetto. Questo riduce il rischio tecnico e consente di procedere per incrementi successivi, mantenendo il notebook `Previsione fabbisogno v05_all.ipynb` come riferimento funzionale durante la fase di refactoring.

Il backlog è inoltre pensato per essere facilmente assegnabile a Codex. Ogni Issue deve contenere un obiettivo operativo, un contesto sufficiente, deliverable espliciti e criteri di completamento verificabili. Codex potrà così leggere il task, modificare il repository, generare codice, aggiornare test e proporre una Pull Request coerente con la roadmap.

8.3 Issue #1 – Estrazione del Data Layer

Titolo: Issue #1 – Estrarre il caricamento dati dal notebook

La prima Issue riguarda l'estrazione del Data Layer dal notebook sorgente. L'obiettivo è identificare tutte le parti del notebook che si occupano di acquisizione, generazione, caricamento e validazione iniziale dei dati, separandole dalle fasi successive di preprocessing, modellazione e visualizzazione.

Questa attività deve produrre una prima struttura modulare in `src/data`. Il modulo dovrà gestire il caricamento delle serie storiche di consumo, produzione fotovoltaica e variabili meteorologiche, mantenendo la possibilità di utilizzare dati sintetici o file locali come base iniziale. La validazione dovrà controllare la presenza delle colonne attese, la coerenza temporale, la frequenza delle osservazioni e la gestione dei valori mancanti.

Gli obiettivi principali della Issue sono:

- identificare nel notebook il codice di acquisizione e caricamento dati;
- creare il modulo `src/data`;
- separare la logica di caricamento dalla logica di validazione;
- predisporre un'interfaccia stabile per i layer successivi.

I deliverable implementati sono `loader.py`, `validator.py` e `synthetic_data.py`. Il modulo `weather_loader.py`, previsto nel backlog iniziale, non è presente nel repository e rimane classificato come estensione pianificata.

8.4 Issue #2 – Creazione Feature Engineering Layer

Titolo: Issue #2 – Implementare il modulo di feature engineering

La seconda Issue riguarda la creazione del Feature Engineering Layer. Questa attività ha l'obiettivo di estrarre dal notebook le trasformazioni necessarie per rendere i dati utilizzabili dai modelli previsionali, mantenendo separata la preparazione degli input dalla logica di training.

Il Feature Engineering Layer dovrà occuparsi di preprocessing, normalizzazione e costruzione delle finestre temporali. Il preprocessing dovrà gestire ordinamento temporale, valori mancanti, coerenza delle serie e trasformazioni preliminari. La normalizzazione dovrà formalizzare l'uso degli scaler e garantire coerenza tra training e inference. La costruzione delle finestre temporali dovrà preparare sequenze compatibili con il modello LSTM sequence-to-sequence adottato nel notebook.

Gli obiettivi principali della Issue sono:

- implementare le funzioni di preprocessing;
- isolare la normalizzazione dei dati;
- costruire finestre temporali coerenti con l'orizzonte previsionale;

- produrre input standardizzati per il Forecasting Layer.

I deliverable previsti sono `preprocessing.py`, `scaling.py` e `sequence_builder.py`. Questi moduli dovranno essere progettati in modo testabile, evitando dipendenze dirette dal notebook e mantenendo stabile il contratto verso il layer di forecasting.

8.5 Issue #3 – Creazione Forecasting Layer

Titolo: Issue #3 – Implementare il motore di forecasting

La terza Issue riguarda la creazione del Forecasting Layer. L'obiettivo è separare la logica predittiva dal resto della pipeline, distinguendo chiaramente definizione del modello, processo di training e fase di inferenza.

Il Forecasting Layer dovrà riflettere l'impostazione già validata nel notebook sorgente, mantenendo una separazione logica tra previsione dei consumi energetici e previsione della produzione fotovoltaica. Questa scelta preserva la differente natura delle due serie e consente di evolvere i modelli in modo indipendente.

Gli obiettivi principali della Issue sono:

- separare la definizione del modello dalla preparazione dati;
- implementare la logica di training;
- implementare la logica di inferenza;
- produrre previsioni utilizzabili dall'Energy Balance Layer.

I deliverable previsti sono `lstm_model.py`, `trainer.py` e `predictor.py`. Il primo modulo dovrà contenere la definizione dell'architettura, il secondo il processo di addestramento e il terzo la generazione delle previsioni operative.

8.6 Issue #4 – Creazione Energy Balance Layer

Titolo: Issue #4 – Implementare il modulo di bilancio energetico

La quarta Issue riguarda la creazione dell'Energy Balance Layer. Questa attività traduce le previsioni di consumo e produzione in informazioni operative per il prosumer, separando la logica energetica dalla logica statistica del modello.

Il layer dovrà calcolare `net_balance`, `grid_export` e `grid_import` a partire dalle previsioni prodotte dal Forecasting Layer. `kpi.py` dovrà produrre esclusivamente totali e cumulativi dei due flussi approvati.

Gli obiettivi principali della Issue sono:

- applicare il naming canonico del bilancio energetico;
- preservare scala fisica e allineamento temporale;
- calcolare totali e cumulativi di importazione ed esportazione;
- fornire output riutilizzabili dal Visualization Layer.

I deliverable previsti sono `balance_calculator.py` e `kpi.py`. I moduli non dovranno applicare clipping automatico agli input, classificare obbligatoriamente il bilancio nullo o gestire CSV e file I/O.

8.7 Issue #5 – Creazione Visualization Layer

Titolo: Issue #5 – Creare il Visualization Layer

La quinta Issue riguarda la separazione delle responsabilità grafiche. Il materiale sorgente fornito contiene l'implementazione di `src/visualization/plotting.py` e la suite specifica `tests/test_visualization_plotting.py`. Sono documentate sette funzioni: `plot_training_history`, `plot_prediction_window`, `plot_error_distribution`, `plot_evaluation_metrics`, `plot_future_forecast`, `plot_grid_exchange` e `plot_cumulative_grid_exchange`. Il modulo rappresenta risultati già calcolati, senza duplicare logiche di forecasting, monitoring o bilancio energetico.

La verifica comunicata riporta 18 test superati in 5,14 secondi con backend Matplotlib Agg e `git diff --check` senza errori, salvo avvisi Git relativi alla conversione LF/CRLF. Non si assume che il test sia stato rieseguito nell'ambito della redazione di questo documento. La v14 riportava la Issue aperta, ma lo stato GitHub attuale, l'esistenza/stato della Pull Request, la review e l'eventuale merge non sono verificabili con i materiali disponibili; l'evidenza del codice fornito non equivale a integrazione nel repository.

8.8 Issue #6 – Introduzione della suite iniziale di test

Titolo: Issue #6 – Creare il framework di testing

La sesta Issue pianifica l'introduzione della suite iniziale di test come capacità trasversale. Il materiale esaminato comprende già la suite visuale `tests/test_visualization_plotting.py`; ciò non completa né sostituisce la suite iniziale dell'intera piattaforma.

I deliverable restano unit test, integration test e controlli di regressione per gli altri layer applicativi. Nel backlog documentale la Issue #6 rimane aperta/pianificata, senza attribuire tale indicazione come stato GitHub verificato.

8.9 Attività non numerata – Monitoring Layer

Il Monitoring Layer è stato implementato nei moduli `metrics.py` ed `evaluation.py`. Poiché questo layer è emerso dopo la definizione del backlog iniziale, la sua implementazione viene registrata come attività non numerata. Questa scelta preserva la sequenza ufficiale delle Issue senza ridurre il Monitoring Layer a componente teorico o secondario.

8.10 Workflow GitHub previsto

Il workflow operativo previsto collega GitHub, Codex e review umana in una sequenza semplice e controllabile:

`Issue` ↓ `Codex` ↓ `Pull Request` ↓ `Review Umana` ↓ `Merge`

La Issue descrive il lavoro da svolgere, il contesto tecnico e i deliverable attesi. Codex utilizza questa descrizione come input operativo, analizza il repository, modifica i file necessari e produce una proposta di implementazione. La Pull Request rende visibili le modifiche, permette di confrontare il codice prodotto con l'obiettivo della Issue e consente di eseguire eventuali controlli automatici.

La review umana mantiene il governo del progetto. In questa fase vengono valutate coerenza architetturale, leggibilità, correttezza funzionale, aderenza agli ADR e impatto sui layer esistenti. Solo dopo la review e gli eventuali aggiustamenti la Pull Request viene integrata tramite merge nel branch principale.

8.11 Ruolo di Codex

Codex avrà il compito di trasformare le GitHub Issue in modifiche concrete al repository. Il suo ruolo non è quello di ridefinire l'architettura, ma di implementare in modo coerente le decisioni già documentate nello Strato 2.

Per ogni Issue, Codex dovrà leggere il task, comprendere il contesto del repository, individuare i file da modificare o creare, generare codice, aggiornare eventuali test e preparare una Pull Request. Il suo contributo è particolarmente utile nelle attività di refactoring dal notebook verso moduli Python, perché può operare in modo incrementale e produrre modifiche tracciabili.

La qualità del lavoro di Codex dipenderà dalla chiarezza delle Issue. Per questo motivo il backlog deve essere esplicito nei deliverable, nei confini del task e nelle dipendenze tra layer. Una Issue ben scritta riduce ambiguità, limita interventi non necessari e facilita la review umana.

8.12 Ruolo di ChatGPT

ChatGPT supporta il progetto nella fase di chiarimento, progettazione e controllo concettuale. Il suo ruolo principale è aiutare a trasformare requisiti, decisioni e obiettivi in istruzioni comprensibili, coerenti e adatte a essere eseguite da Codex.

In particolare, ChatGPT può contribuire al chiarimento dei requisiti, alla review architetturale, alla definizione dei prompt per Codex e alla verifica della coerenza implementativa. Può inoltre aiutare a confrontare le modifiche proposte con la roadmap, evidenziando eventuali disallineamenti tra codice, ADR e struttura prevista del repository.

ChatGPT agisce quindi come supporto di governo e riflessione: non sostituisce la review umana, ma aiuta a rendere più chiari i task, più robusti i criteri di accettazione e più coerente il processo di evoluzione del progetto.

8.13 Ruolo di Prism

Prism rappresenta lo spazio di tracciabilità della conoscenza progettuale. In Prism vengono documentati il backlog, l'avanzamento dei lavori, il collegamento tra ADR e sviluppo, le decisioni prese e le motivazioni che guidano l'evoluzione del repository.

Il valore di Prism è mantenere continuità tra progettazione, implementazione e apprendimento. Ogni Issue non deve essere vista come un'attività isolata, ma come una conseguenza delle decisioni documentate nei capitoli precedenti. Prism consente di preservare questa relazione e di rendere leggibile nel tempo il motivo per cui una certa struttura è stata scelta.

Nel passaggio allo Strato 3, Prism diventa quindi il diario tecnico del progetto. Esso permette di collegare le Pull Request agli ADR, aggiornare lo stato del backlog, registrare eventuali deviazioni motivate e mantenere una documentazione coerente con l'implementazione reale.

8.14 Conclusioni

Il backlog GitHub rappresenta l'ultimo deliverable dello Strato 2. Con esso, l'architettura e la roadmap vengono trasformate in un insieme di attività operative, pronte per essere assegnate, implementate, revisionate e integrate nel repository `AI-IT-stack-lab`.

Questo documento costituisce quindi il punto di ingresso dello Strato 3, dedicato all'implementazione reale. Le sei Issue consolidate e l'attività non numerata del Monitoring Layer descrivono la tracciabilità effettiva del refactoring. La Table 1 conserva il nome storico del backlog iniziale, ma distingue esplicitamente componenti implementati e attività pianificate.

Table 1: Initial GitHub Backlog

Issue	**Layer**	**Obiettivo**	**Deliverable Principali**
Issue #1	Data Layer	Caricamento, validazione e generazione sintetica	`loader.py`, `validator.py`, `synthetic_data.py`
Issue #2	Feature Engineering Layer	Implementare preprocessing, normalizzazione e finestre temporali	`preprocessing.py`, `scaling.py`, `sequence_builder.py`
Issue #3	Forecasting Layer	Separare modello, training e inferenza	`lstm_model.py`, `trainer.py`, `predictor.py`
Issue #4	Energy Balance Layer	Calcolare bilancio canonico, flussi, totali e cumulativi	`balance_calculator.py`, `kpi.py`
Issue #5	Visualization Layer	Centralizzare la rappresentazione grafica	`plotting.py` e test specifici presenti nel materiale; stato GitHub/PR non verificato
Issue #6	Testing trasversale	Introdurre la suite iniziale di test	Aperta/pianificata nel documento; suite visuale specifica disponibile, suite complessiva non dimostrata
Attività non numerata	Monitoring Layer	Calcolare metriche e orchestrare la valutazione	`metrics.py`, `evaluation.py` – implementati

9 Technical Specifications

9.1 Obiettivo delle specifiche tecniche

Questo documento definisce le specifiche tecniche ufficiali del progetto *Energy Forecasting Platform – AI Operational Stack*. Il suo scopo è trasformare l'architettura logica e la roadmap operativa in un riferimento tecnico stabile, utilizzabile durante il refactoring del notebook originale e durante le successive attività di implementazione affidate a Codex.

Le specifiche distinguono tre livelli: architettura concettuale, roadmap pianificata e stato effettivamente implementato. Il repository costituisce il riferimento per identificare i moduli disponibili; gli elementi non presenti vengono indicati esplicitamente come pianificati o futuri.

Queste specifiche costituiscono quindi il riferimento ufficiale per lo sviluppo dello Strato 3. Ogni modulo dovrà essere implementato in modo coerente con i layer descritti, con gli ADR già approvati e con il backlog GitHub definito nel capitolo precedente.

9.2 Data Layer

Il Data Layer è implementato nel modulo `src/data/`. Questo layer ha la responsabilità di acquisire, generare, validare e rendere disponibili i dati necessari alla pipeline di forecasting energetico, rimanendo indipendente dai layer successivi.

Le responsabilità principali del Data Layer sono:

- caricamento dei dati energetici storici, inclusi consumi e produzione fotovoltaica;
- caricamento dei dati meteorologici storici e prospettici;
- validazione dei dataset in ingresso;
- gestione preliminare di anomalie, valori mancanti e incoerenze temporali.

Gli input previsti comprendono file `CSV`, file `XLSX` e future integrazioni tramite API. Nella fase iniziale il layer dovrà supportare sorgenti locali, mantenendo però un disegno compatibile con connettori esterni o servizi remoti.

L'output del Data Layer è costituito da dataframe validati e puliti, con schema coerente, timestamp ordinati e colonne attese disponibili. Questi dataframe rappresentano il contratto di ingresso per il Feature Engineering Layer.

I componenti previsti sono:

- `loader.py`, dedicato al caricamento dei dataset energetici;
- `validator.py`, dedicato ai controlli di qualità, schema e coerenza temporale;
- `synthetic_data.py`, dedicato alla generazione dei dataset sintetici coerenti con il notebook originale.

Il componente `weather_loader.py` appartiene alla pianificazione iniziale, ma non è implementato. Un futuro connettore meteorologico dovrà essere introdotto come estensione distinta.

9.3 Feature Engineering Layer

Il Feature Engineering Layer è implementato nel modulo `src/features/`. Questo layer trasforma i dataset validati in strutture dati utilizzabili dai modelli previsionali, senza contenere logica di modellazione.

Le responsabilità principali sono:

- gestione e normalizzazione dei timestamp;
- costruzione di feature temporali, come ora, giorno, settimana e stagionalità;
- normalizzazione numerica delle variabili;
- costruzione delle sequenze temporali per modelli sequence-to-sequence.

Gli input del layer sono i dataset validati prodotti dal Data Layer. Tali dataset devono già avere uno schema coerente e una qualità minima garantita. Il Feature Engineering Layer non deve duplicare la validazione strutturale dei dati, ma può applicare controlli specifici sulle trasformazioni che esegue.

Gli output sono dataset pronti per il training e per l'inferenza. In particolare, il layer dovrà produrre matrici o strutture sequenziali coerenti con l'orizzonte previsionale previsto e con i requisiti del Forecasting Layer.

I componenti previsti sono:

- `preprocessing.py`, dedicato a ordinamento temporale, pulizia e trasformazioni preliminari;
- `scaling.py`, dedicato alla gestione degli scaler e alla coerenza tra training e inference;
- `sequence_builder.py`, dedicato alla costruzione delle finestre temporali e delle sequenze di input-output.

9.4 Forecasting Layer

Il Forecasting Layer è implementato nel modulo `src/forecasting/`. Questo layer contiene la logica predittiva della piattaforma e rimane separato da preparazione dati, valutazione quantitativa e interpretazione energetica.

Le responsabilità principali sono:

- training dei modelli previsionali;
- inferenza su orizzonti futuri;
- salvataggio dei modelli addestrati;
- gestione delle versioni dei modelli e dei relativi artefatti.

I componenti previsti sono:

- `lstm_model.py`, dedicato alla definizione dell'architettura del modello;

- `trainer.py`, dedicato al processo di training e alla gestione delle callback;
- `predictor.py`, dedicato alla generazione delle previsioni operative.

L'architettura prevede due modelli distinti. Il **Modello A** è dedicato al forecast dei consumi energetici. Il **Modello B** è dedicato al forecast della produzione fotovoltaica. Questa scelta è coerente con ADR-002, che formalizza la separazione tra le due serie previsionali per rispettare la diversa natura fisica e statistica dei fenomeni osservati.

La previsione dei consumi dipende da pattern comportamentali, calendario, stagionalità e dinamiche di utilizzo. La previsione della produzione fotovoltaica dipende invece in modo più diretto da variabili meteorologiche e condizioni ambientali. Separare i modelli consente quindi di ottimizzare architetture, feature e metriche in modo indipendente, riducendo l'accoppiamento tra fenomeni diversi.

9.5 Monitoring Layer

Il Monitoring Layer è implementato nel modulo `src/monitoring/`. Esso riceve valori osservati e predetti già riportati alla scala fisica originale e produce una valutazione quantitativa separata dal training e dalla prediction.

I componenti implementati sono `metrics.py`, che calcola MAE e RMSE, ed `evaluation.py`, che orchestra il confronto e restituisce il risultato strutturato. Il layer non genera grafici, non esegue inverse scaling e non implementa drift monitoring, alerting o retraining automatico.

Il Monitoring Layer è un layer ufficiale e concreto. Non corrisponde a una Issue retroattiva numerata: nella tracciabilità storica del progetto la sua implementazione è registrata come attività non numerata successiva al backlog iniziale.

9.6 Energy Balance Layer

L'Energy Balance Layer è definito nel modulo `src/energy_balance/`. Questo layer interpreta le previsioni prodotte dal Forecasting Layer e costituisce il prossimo contratto implementativo da affidare a Codex.

Le responsabilità principali sono:

- calcolo di `net_balance = production - consumption`;
- calcolo di `grid_export = max(net_balance, 0)`;
- calcolo di `grid_import = max(-net_balance, 0)`;
- calcolo dei totali e dei cumulativi dei due flussi.

Gli input del layer sono il forecast dei consumi e il forecast della produzione fotovoltaica. Il layer non deve conoscere i dettagli interni dei modelli che hanno prodotto tali forecast: deve ricevere serie previsionali coerenti per timestamp e orizzonte temporale.

Gli output canonici sono:

- `net_balance`, `grid_import` e `grid_export`;
- `total_grid_import` e `total_grid_export`;
- `cumulative_grid_import` e `cumulative_grid_export`.

Il layer conserva granularità, indice e unità degli input e opera su valori già nella scala fisica originale.

Il contratto non prevede classificazione obbligatoria del valore zero, clipping automatico di consumo o produzione, né caricamento o salvataggio di file. L'eventuale esportazione CSV appartiene all'orchestrazione o alla persistenza esterna.

I componenti previsti sono:

- `balance_calculator.py`, dedicato al calcolo del bilancio energetico;
- `kpi.py`, dedicato alla definizione e al calcolo degli indicatori energetici.

9.7 Visualization Layer

Il Visualization Layer è definito nel modulo `src/visualization/`. Il materiale sorgente fornito documenta l'implementazione di `plotting.py` e la suite specifica dei test, secondo i contratti qui descritti e nel Capitolo 22. La Issue #5 risultava aperta nella v14; lo stato GitHub corrente e l'eventuale integrazione tramite Pull Request non sono verificabili. Il layer rappresenta previsioni, valori osservati, metriche e risultati energetici già calcolati.

Le responsabilità principali sono:

- curve di training loss e validation loss;
- confronto reale–predetto per una finestra di consumo o produzione;
- distribuzione statistica degli errori mediante istogramma e KDE;
- riepilogo grafico di MAE e RMSE già calcolati;
- forecast futuro combinato di consumo e produzione;
- confronto tra `grid_import` e `grid_export`;

- cumulativi di importazione ed esportazione.

Gli input e le sette API pubbliche sono definiti nel Capitolo 22. Il Visualization Layer non deve modificare la logica di calcolo e non deve introdurre trasformazioni applicative non tracciate. Deve ricevere dati già elaborati e produrre output visuali.

Gli output previsti sono coppie di oggetti Matplotlib `Figure/Axes`, destinate all'analisi delle previsioni, delle metriche e dei risultati energetici. Il salvataggio e la visualizzazione interattiva appartengono all'orchestratore.

I componenti previsti sono:

- `plotting.py`, dedicato alla produzione delle rappresentazioni grafiche previste.

9.8 Contratti tra i moduli

Il flusso tecnico principale della piattaforma segue la direzione già definita nell'architettura:

```
Data Layer
    -> Feature Engineering Layer
    -> Forecasting Layer
    -> Predictions
         |-> Monitoring Layer -> Quantitative Evaluation -> Visualization Layer
         `-> Energy Balance Layer -> Bilanci e KPI -> Visualization Layer
```

Ogni modulo deve essere indipendente e comunicare esclusivamente tramite interfacce documentate. Un layer può conoscere solo gli output contrattuali del layer precedente, non i dettagli interni della sua implementazione. Questo principio riduce l'accoppiamento, semplifica i test e consente di sostituire componenti specifici senza riscrivere l'intera pipeline.

Il Data Layer produce dataset validati. Il Feature Engineering Layer produce dataset trasformati e sequenze. Il Forecasting Layer produce previsioni. Il Monitoring Layer riceve valori osservati e predetti già confrontabili e produce MAE, RMSE e risultati strutturati di valutazione. L'Energy Balance Layer interpreta i forecast attraverso bilanci e KPI. Il Visualization Layer rappresenta graficamente previsioni, valutazioni e risultati energetici già calcolati. La futura suite di test opererà trasversalmente senza diventare un passaggio della pipeline.

La denominazione *Operational Analysis* indica un concetto architetturale più ampio. Comprende l'utilizzo decisionale di forecast, metriche, bilanci e grafici, ma non corrisponde a una cartella Python autonoma.

9.9 Interfacce principali

La seguente interfaccia rappresenta un esempio progettuale del contratto logico end-to-end della piattaforma:

```
def forecast_energy(
    consumption_data,
    production_data,
    weather_data,
    horizon_hours=168
):
    pass
```

Questa funzione non rappresenta l'implementazione definitiva, ma un contratto logico. Essa esplicita che il sistema deve ricevere dati di consumo, dati di produzione, dati meteorologici e un orizzonte previsionale, producendo un risultato coerente con la pipeline di forecasting energetico.

In fase implementativa, Codex potrà trasformare questo contratto in una o più funzioni concrete, classi o servizi applicativi. Tuttavia, il significato dell'interfaccia deve rimanere stabile: l'utente o il layer orchestratore devono poter richiedere una previsione energetica senza conoscere i dettagli interni dei moduli sottostanti.

9.10 Gestione degli artefatti

La gestione degli artefatti deve mantenere separati codice, dati, modelli, esperimenti e documentazione di progetto. Questa separazione è essenziale per garantire tracciabilità, riproducibilità e manutenzione del repository.

Le collocazioni previste sono:

- `04_data/`, per dataset grezzi, dataset elaborati e sorgenti dati locali;
- `05_models/`, per modelli addestrati, scaler e artefatti di inferenza versionati;
- `06_experiments/`, per metriche, report di training, risultati sperimentali e confronti tra run;
- `07_projects/forecasting_energy/`, per il progetto applicativo di forecasting energetico e la sua organizzazione operativa.

I dati devono rimanere separati dal codice sorgente. I modelli devono essere versionati in modo da poter collegare ogni artefatto al dataset, alla configurazione e all'esperimento che lo hanno generato. Metriche e report devono essere archiviati negli experiments, così da rendere confrontabili le evoluzioni del progetto e verificabile l'impatto delle modifiche introdotte.

9.11 Requisiti non funzionali

I requisiti non funzionali definiscono le qualità tecniche che la piattaforma deve preservare durante l'evoluzione dal notebook monolitico a sistema modulare.

- **Manutenibilità**: il codice deve essere organizzato in moduli con responsabilità chiare, così da rendere semplici modifiche, correzioni e refactoring successivi.
- **Modularità**: ogni layer deve essere sviluppato come componente separato, con dipendenze esplicite e limitate.
- **Tracciabilità**: decisioni, artefatti, modelli, metriche e Pull Request devono poter essere collegati a Issue, ADR e documentazione Prism.
- **Testabilità**: le funzioni principali devono poter essere validate tramite unit test e integration test, senza dipendere dall'intero notebook.
- **Scalabilità progressiva**: la piattaforma deve poter evolvere verso nuove sorgenti dati, nuovi modelli e nuove interfacce senza riscritture radicali.
- **Riutilizzabilità dei componenti**: moduli come caricamento dati, sequence building, forecasting e calcolo KPI devono poter essere riutilizzati in contesti futuri o pipeline alternative.

9.12 Conclusioni

Queste specifiche tecniche rappresentano il contratto ufficiale che guiderà il refactoring del notebook `Previsione fabbisogno v05_all.ipynb` e l'implementazione dello Strato 3 mediante Codex. Il documento definisce responsabilità, confini e interfacce dei componenti senza anticipare scelte implementative di dettaglio.

A partire da queste specifiche, le GitHub Issue del backlog potranno essere trasformate in attività di sviluppo concrete. Codex dovrà implementare i moduli rispettando i contratti descritti, mentre la review umana dovrà verificare coerenza architetturale, qualità tecnica e aderenza agli ADR. In questo modo il progetto potrà evolvere da notebook sperimentale a piattaforma modulare, tracciabile e progressivamente estendibile.

10 Implementation Strategy

10.1 Passaggio dalla progettazione all'implementazione

Con questo capitolo inizia lo Strato 3 del progetto *Energy Forecasting Platform – AI Operational Stack*, dedicato all'implementazione. Dopo aver definito il contesto di business, i requisiti, l'architettura, gli ADR, i flussi dati, la roadmap, il backlog e le specifiche tecniche, il progetto entra nella fase in cui le decisioni documentate devono essere trasformate in componenti software reali.

Il passaggio dalla progettazione all'implementazione non deve essere interpretato come una cesura netta, ma come una continuità controllata. Lo Strato 2 ha avuto il compito di chiarire il problema, organizzare la conoscenza e definire il disegno della soluzione. Lo Strato 3 ha il compito di rendere eseguibile quel disegno, mantenendo coerenza con le scelte già formalizzate e riducendo il rischio di dispersione tecnica.

Il repository GitHub `AI-IT-stack-lab` diventa in questa fase il luogo principale in cui la progettazione si traduce in codice, struttura, versionamento e verifica. Prism continua a svolgere il ruolo di repository della conoscenza progettuale, preservando il contesto delle decisioni e consentendo di collegare ogni attività implementativa alla documentazione già prodotta.

10.2 Dal notebook sperimentale alla soluzione strutturata

Il punto di partenza dello Strato 3 è il notebook originale di forecasting energetico. Il notebook rappresenta una risorsa importante perché contiene la prima dimostrazione end-to-end della soluzione: generazione o caricamento dei dati, preparazione delle feature, training dei modelli, produzione delle previsioni, calcolo del bilancio energetico e visualizzazione dei risultati.

Un notebook sperimentale è particolarmente utile nelle fasi iniziali di un progetto data-driven. Permette di esplorare rapidamente ipotesi, visualizzare risultati intermedi, modificare celle e validare intuizioni senza introdurre immediatamente una struttura software complessa. In questa fase, la priorità è dimostrare che il problema può essere affrontato e che il flusso logico produce risultati interpretabili.

Una soluzione software strutturata ha però obiettivi differenti. Deve essere leggibile, manutenibile, testabile e riutilizzabile. Deve consentire a più persone o strumenti di intervenire sul codice senza dipendere dalla sequenza implicita delle celle del notebook. Deve separare responsabilità diverse e rendere espliciti i passaggi tra dati, feature, modelli, bilanci e output decisionali.

La strategia di implementazione consiste quindi nel non scartare il notebook, ma nel trasformarlo progressivamente. Il notebook rimane una baseline funzionale e conoscitiva, mentre il codice stabile viene estratto, riorganizzato e convertito in componenti modulari all'interno del repository.

10.3 Motivazioni del refactoring

Il refactoring del notebook originale è necessario perché la forma sperimentale, pur efficace per validare l'idea, non è sufficiente per sostenere l'evoluzione della piattaforma. Un notebook monolitico tende a mescolare responsabilità diverse: caricamento dati, trasformazioni, modellazione, metriche, grafici e interpretazioni possono convivere nello stesso flusso senza confini chiari.

Questa impostazione rende più difficile capire dove intervenire quando si vuole modificare una parte della soluzione. Un cambiamento nella preparazione dei dati può avere effetti non immediatamente visibili sul training; una modifica al modello può influenzare le metriche; una visualizzazione può dipendere da strutture dati create molte celle prima. La conoscenza rimane spesso implicita nella

sequenza di esecuzione, più che espressa in contratti chiari.

Il refactoring serve a ridurre questa ambiguità. L'obiettivo non è riscrivere tutto da zero, ma rendere esplicite le responsabilità già presenti nel notebook. Le parti che oggi funzionano in modo sequenziale dovranno essere trasformate in componenti riconoscibili, coerenti con l'architettura definita nello Strato 2.

Questa scelta ha anche una motivazione organizzativa. Poiché Codex sarà l'agente principale di sviluppo, il repository deve essere organizzato in modo che le attività possano essere descritte, assegnate, implementate e revisionate per porzioni circoscritte. Un notebook monolitico rende più difficile questo tipo di lavoro; una struttura modulare lo rende naturale.

10.4 Obiettivi della fase di implementazione

La fase di implementazione ha come primo obiettivo la modularizzazione della soluzione. Lo stato consolidato comprende Data, Feature Engineering, Forecasting, Monitoring, Energy Balance e Visualization Layer come parti riconoscibili della piattaforma. Il testing è pianificato come capacità trasversale e non risulta rappresentato da un layer applicativo nella struttura corrente del repository.

Un secondo obiettivo è la separazione delle responsabilità. Ogni componente dovrà avere un ruolo chiaro e limitato. La preparazione dei dati non dovrà essere confusa con il training dei modelli; il calcolo del bilancio energetico non dovrà dipendere dai dettagli interni della rete neurale; la visualizzazione non dovrà modificare la logica applicativa. Questa separazione rende più semplice comprendere, verificare e modificare il sistema.

La riusabilità è un ulteriore obiettivo. Le funzioni e i moduli estratti dal notebook dovranno poter essere utilizzati in contesti diversi: esperimenti, pipeline periodiche, test, report o future interfacce applicative. Il valore della piattaforma aumenta se i suoi componenti non sono legati a una singola esecuzione manuale.

La testabilità rappresenta un criterio fondamentale. Una soluzione che non può essere verificata in modo controllato rimane fragile. Lo Strato 3 dovrà quindi preparare le condizioni per introdurre test progressivi sui dati, sulle trasformazioni, sulle previsioni e sui calcoli energetici. Anche se la validazione completa avverrà in fasi successive, l'implementazione deve nascere con questa possibilità.

Infine, la manutenibilità deve guidare l'intero processo. Il progetto non deve produrre semplicemente un nuovo script funzionante, ma una base software che possa essere letta, migliorata e adattata nel tempo. Questo requisito è essenziale perché il forecasting energetico è un dominio evolutivo: cambiano i dati, cambiano i modelli, cambiano le esigenze decisionali e cambiano le modalità di utilizzo.

10.5 Ruolo di Codex nello sviluppo

Codex avrà un ruolo centrale nello Strato 3. Il suo compito sarà analizzare il codice esistente, identificare le parti del notebook riconducibili ai layer progettati, estrarre componenti coerenti e generare moduli Python compatibili con la struttura del repository.

L'uso di Codex non elimina la necessità di governo umano del progetto. Al contrario, richiede che le attività siano descritte con chiarezza e che ogni intervento sia collegato a una Issue, a una decisione documentata o a una specifica tecnica. Codex deve agire come agente di sviluppo, non come sostituto della progettazione.

Nel processo di refactoring, Codex potrà supportare l'individuazione delle funzioni riutilizzabili, la separazione del codice in moduli, la rimozione di duplicazioni e la creazione di prime strutture testabili. Potrà inoltre proporre modifiche incrementali tramite branch e Pull Request, rendendo visibile il cambiamento prima dell'integrazione nel ramo principale.

La qualità del lavoro di Codex dipenderà dalla qualità del backlog e dalla coerenza della documentazione. Per questo motivo lo Strato 2 non è un esercizio preliminare separato dall'implementazione, ma la base che rende possibile uno sviluppo assistito ordinato, verificabile e allineato agli obiettivi del progetto.

10.6 Ruolo di GitHub nel processo operativo

GitHub sarà l'ambiente centrale di coordinamento dello sviluppo. Le Issue descriveranno le unità di lavoro, i branch isoleranno le modifiche, le Pull Request renderanno visibili le proposte di integrazione e la review permetterà di controllare qualità, coerenza e impatto delle modifiche.

Questo flusso è importante perché consente di mantenere tracciabilità. Ogni cambiamento rilevante potrà essere collegato a un obiettivo, a una parte dell'architettura o a una decisione progettuale. Il versionamento non riguarda soltanto il codice, ma l'intero percorso di evoluzione della piattaforma.

Il repository `AI-IT-stack-lab` dovrà quindi diventare il luogo in cui convivono sperimentazione controllata e costruzione progressiva. Il notebook originale potrà rimanere come riferimento, ma il baricentro del progetto dovrà spostarsi gradualmente verso una struttura più organizzata e adatta allo sviluppo collaborativo.

10.7 Evoluzione prevista del repository

L'evoluzione prevista del repository segue una direzione chiara: dai notebook verso una struttura modulare basata su codice sorgente organizzato. La cartella `src` diventerà progressivamente il luogo in cui collocare la logica stabile della piattaforma, mentre i notebook manterranno un ruolo di analisi, confronto, dimostrazione o sperimentazione.

Parallelamente, dati, modelli, esperimenti e codice dovranno rimanere separati. I dati non devono

essere confusi con la logica applicativa; i modelli addestrati devono poter essere conservati e versionati; gli esperimenti devono mantenere memoria delle metriche e dei risultati; il codice deve restare leggibile e governabile.

Questa separazione non è solo una scelta organizzativa. È una condizione per rendere il progetto evolvibile. Quando i confini sono chiari, diventa più semplice sostituire una sorgente dati, aggiornare un modello, confrontare due esperimenti o modificare un report senza destabilizzare l'intera soluzione.

10.8 Implementazione incrementale e riduzione del rischio

La strategia dello Strato 3 sarà incrementale. Il notebook non verrà trasformato in piattaforma con un unico intervento massivo, perché una riscrittura completa aumenterebbe il rischio di introdurre errori difficili da individuare. L'approccio corretto consiste nel procedere per piccoli passi, ognuno dei quali deve produrre un miglioramento chiaro e verificabile.

Ogni incremento dovrà avere un perimetro limitato. Una fase potrà riguardare l'estrazione del caricamento dati, un'altra la normalizzazione, un'altra ancora la costruzione delle sequenze o il calcolo del bilancio energetico. In questo modo sarà possibile confrontare progressivamente il comportamento dei nuovi moduli con quello del notebook originale.

Le verifiche continue sono parte integrante della strategia. Non devono essere considerate un'attività finale, ma un meccanismo di controllo durante l'evoluzione. Ogni passo deve ridurre l'incertezza, non aumentarla. La piattaforma crescerà quindi attraverso una sequenza di trasformazioni controllate, documentate e revisionabili.

10.9 Relazione tra Strato 2 e Strato 3

Lo Strato 2 ha definito il disegno della soluzione. Ha chiarito il problema di business, formalizzato i requisiti, descritto l'architettura, documentato le decisioni principali e trasformato la roadmap in un backlog operativo. Lo Strato 3 deve ora trasformare quel disegno in implementazione reale.

La relazione tra i due strati è diretta. Ogni modulo implementato dovrà essere riconducibile a una responsabilità architetturale; ogni attività di sviluppo dovrà essere collegata a una Issue; ogni deviazione significativa dovrà essere discussa, documentata e, se necessario, ricondotta a una decisione progettuale aggiornata.

In questo senso, l'implementazione non è una fase puramente esecutiva. È il momento in cui la progettazione viene messa alla prova. Se un requisito risulta ambiguo, se un confine tra layer non è chiaro o se una scelta architetturale produce complessità inattesa, Prism dovrà preservare l'evidenza di tale apprendimento e consentire l'aggiornamento consapevole della conoscenza progettuale.

10.10 Conclusioni

Il risultato atteso dello Strato 3 non è semplicemente un nuovo algoritmo di forecasting. Il notebook originale contiene già una dimostrazione del flusso predittivo; il valore della fase di implementazione consiste nel trasformare quella dimostrazione in una piattaforma software organizzata, manutenibile ed evolvibile.

La strategia adottata mette al centro modularizzazione, separazione delle responsabilità, riusabilità, testabilità e manutenibilità. Codex agirà come agente principale di sviluppo, GitHub governerà il flusso operativo e Prism conserverà la conoscenza progettuale necessaria a mantenere coerenza nel tempo.

Lo Strato 3 rappresenta quindi il passaggio dalla possibilità tecnica alla capacità operativa. A partire dal notebook esistente e dall'architettura definita nello Strato 2, il progetto potrà costruire una base software pronta per le successive fasi di validazione, test e progressiva adozione operativa.

11 Notebook Analysis

11.1 Ruolo del notebook nel progetto

Il notebook `Previsione Fabbisogno v05_all.ipynb` rappresenta il prototipo iniziale della piattaforma di forecasting energetico. Esso costituisce il punto di partenza dell'intera evoluzione architetturale descritta nei capitoli precedenti, perché contiene una prima realizzazione end-to-end del processo: dai dati di input fino alla generazione delle previsioni, dal calcolo del bilancio energetico fino alla visualizzazione dei risultati.

Nel contesto dello Stack Operativo Ottimizzato, il notebook ha avuto un ruolo esplorativo e dimostrativo. Ha permesso di verificare che il problema potesse essere affrontato con un flusso dati coerente, con modelli previsionali separati per consumo e produzione, e con una trasformazione finale dei forecast in informazioni energetiche operative. In questa fase, la priorità non era ancora costruire una piattaforma software modulare, ma dimostrare la fattibilità concettuale del processo.

Il valore del notebook non risiede quindi soltanto nel codice che contiene, ma nella conoscenza operativa che ha consolidato. Esso mostra quali dati sono necessari, quali trasformazioni risultano utili, quale tipo di modello è stato sperimentato, quali output sono attesi e quali informazioni sono rilevanti per il prosumer. Per questo motivo il refactoring non deve partire da una pagina bianca: deve partire dall'analisi del notebook, preservandone la logica funzionale e rendendo espliciti i confini tra responsabilità diverse.

11.2 Struttura generale del notebook

La struttura del notebook segue una pipeline progressiva. Le prime sezioni impostano l'ambiente di lavoro, importano le librerie, definiscono i parametri generali e preparano le funzioni di supporto.

Successivamente il notebook genera o acquisisce i dati, costruisce il dataset storico, prepara le sequenze temporali, addestra i modelli, valuta le previsioni e produce un bilancio energetico atteso.

La prima area funzionale riguarda l'acquisizione e la generazione dei dati. Nel prototipo, il notebook utilizza prevalentemente dati sintetici per simulare temperatura, irraggiamento, vento, consumo energetico e produzione fotovoltaica. Questa scelta consente di validare il comportamento della pipeline anche in assenza di dati reali consolidati, mantenendo un controllo completo sulle variabili generate.

La seconda area riguarda la preparazione del dataset. Le serie storiche vengono combinate in una struttura tabellare che include variabili meteorologiche, consumo e produzione. Il dataset viene poi pulito, imputato e suddiviso in training, validation e test secondo una logica temporale, evitando di trattare le osservazioni come campioni indipendenti non ordinati.

La terza area riguarda il feature engineering e la costruzione delle sequenze temporali. Il notebook prepara input compatibili con un modello sequence-to-sequence, distinguendo tra una finestra storica utilizzata come contesto e un orizzonte futuro da prevedere. Questa fase rappresenta uno dei passaggi più importanti dell'intero flusso, perché traduce le serie temporali in strutture utilizzabili dal modello predittivo.

La quarta area riguarda l'addestramento dei modelli. Il notebook costruisce e addestra due modelli distinti: uno per la previsione dei consumi e uno per la previsione della produzione fotovoltaica. Questa separazione è coerente con l'architettura progettuale definita nello Strato 2, perché riconosce la diversa natura dei due fenomeni.

Le sezioni finali producono forecast sui sette giorni successivi, calcolano il bilancio energetico, stimano energia acquistata ed energia immessa in rete, e generano grafici riepilogativi. In questo modo il notebook non si limita a prevedere valori numerici, ma chiude il ciclo trasformando le previsioni in informazioni operative.

11.3 Analisi dei dati utilizzati

Il notebook utilizza quattro categorie principali di dati: consumo energetico, produzione fotovoltaica, variabili meteorologiche storiche e variabili meteorologiche future. Nella versione analizzata, tali dati sono generati sinteticamente, ma sono costruiti in modo da simulare pattern coerenti con il dominio energetico.

I dati di consumo rappresentano il fabbisogno orario del prosumer. La generazione sintetica include un comportamento giornaliero, con variazioni legate alle fasce orarie, e un effetto della temperatura sul consumo. Questo consente al prototipo di rappresentare, almeno in forma semplificata, la relazione tra condizioni ambientali e domanda energetica.

I dati di produzione rappresentano la generazione fotovoltaica oraria. Nel notebook, la produzione viene costruita a partire dall'irraggiamento sintetico, dalla capacità dell'impianto e da un coefficiente di efficienza. La produzione è quindi collegata direttamente alla disponibilità solare simulata, con valori limitati a grandezze non negative.

I dati meteorologici includono temperatura, irraggiamento e velocità del vento. Essi alimentano sia la generazione dei dati sintetici sia la costruzione delle feature per i modelli. Il notebook distingue inoltre tra meteo storico, utilizzato per costruire dataset e training, e meteo futuro, utilizzato come input per generare previsioni sui sette giorni successivi.

L'uso di dati sintetici è un punto di forza nella fase prototipale, perché consente di testare l'intera pipeline senza dipendere da disponibilità, qualità o formati di sorgenti reali. Allo stesso tempo, rappresenta un limite per l'adozione operativa: la piattaforma dovrà successivamente integrare dati reali, tracciabili e validati per poter produrre forecast utilizzabili nel contesto del cliente.

11.4 Analisi delle trasformazioni effettuate

Le trasformazioni presenti nel notebook rispondono a un'esigenza precisa: rendere i dati compatibili con un processo di forecasting sequence-to-sequence. Il flusso non si limita a passare i dati grezzi al modello, ma li prepara attraverso pulizia, imputazione, scaling, costruzione di feature e creazione di finestre temporali.

La pulizia dati è gestita attraverso una funzione che lavora su dataframe indicizzati temporalmente. Il notebook introduce anche valori mancanti casuali e li tratta tramite interpolazione temporale, seguita da riempimento in avanti e indietro. Questa scelta consente di simulare una condizione realistica, in cui le serie energetiche o meteo possono presentare buchi informativi.

La gestione del timestamp è centrale. Le serie sono organizzate con frequenza oraria, coerente con l'obiettivo di produrre forecast orari. La suddivisione in training, validation e test rispetta l'ordine temporale e include un buffer legato alla finestra storica, così da mantenere coerenza nella costruzione delle sequenze.

La normalizzazione viene applicata mediante scaler, con attenzione al principio di evitare data leakage. Il notebook contiene una revisione esplicita in cui gli scaler vengono addestrati sui dati di training e poi applicati agli altri insiemi. Questo è un punto metodologico importante, perché impedisce che informazioni provenienti da validation o test influenzino indirettamente la fase di apprendimento.

La costruzione delle finestre temporali trasforma le serie in input per il modello. Una finestra passata di 168 ore fornisce il contesto storico, mentre una finestra futura di 168 ore rappresenta l'orizzonte previsionale. Il modello riceve quindi una sequenza di encoder basata sul passato e una sequenza di decoder basata sulle variabili meteorologiche future.

Queste trasformazioni mostrano che il notebook contiene già una logica avanzata e coerente con l'architettura futura. Tuttavia, molte di esse sono concentrate nello stesso ambiente esecutivo e dipendono da variabili globali definite in celle precedenti. Questo rende necessario estrarle in componenti più chiari e riutilizzabili.

11.5 Analisi del modello predittivo

Il modello predittivo utilizzato nel notebook è una rete neurale LSTM Sequence-to-Sequence implementata con Keras. La scelta è coerente con la natura del problema, perché il forecasting energetico richiede di apprendere pattern temporali e produrre una sequenza futura, non soltanto un singolo valore puntuale.

La struttura del modello prevede un encoder che riceve la finestra storica e un decoder che produce l'orizzonte previsionale. Sono presenti layer LSTM, meccanismi di dropout e un output temporale che consente di generare una previsione per ogni passo futuro. Questo assetto permette di trattare la previsione dei sette giorni successivi come una sequenza coerente.

Il notebook addestra due modelli: uno per i consumi e uno per la produzione. Il training utilizza validation set, early stopping e callback di salvataggio del modello migliore. Questa impostazione mostra una buona consapevolezza del rischio di overfitting e della necessità di controllare l'apprendimento durante l'addestramento.

Le metriche vengono calcolate su training, validation e test. Un aspetto positivo è l'attenzione alla scala originale: il notebook include funzioni per invertire la trasformazione degli scaler e calcolare le metriche in unità fisicamente interpretabili. Questo è importante perché MAE e RMSE calcolati su scala normalizzata sarebbero meno utili per interpretazioni energetiche.

L'orizzonte temporale utilizzato è di 168 ore, pari a sette giorni. La scelta è coerente con gli obiettivi funzionali del progetto, perché fornisce una previsione abbastanza ampia da supportare pianificazione operativa e analisi settimanale, mantenendo una granularità oraria.

11.6 Analisi del bilancio energetico

Una delle parti più rilevanti del notebook è la trasformazione delle previsioni in bilancio energetico. Dopo aver generato forecast di consumo e produzione, il notebook calcola la differenza tra produzione prevista e consumo previsto. Questa differenza diventa la base informativa per stimare energia acquistata ed energia immessa in rete.

Quando la produzione prevista supera il consumo previsto, il bilancio è positivo e il notebook interpreta la differenza come energia potenzialmente cedibile o immessa in rete. Quando il consumo previsto supera la produzione prevista, il bilancio è negativo e la differenza viene trasformata in energia da acquistare dalla rete.

Questa logica è essenziale perché collega il modello predittivo al problema di business. Il prosumer non è interessato soltanto a una curva di consumo o produzione; ha bisogno di capire se in un certo momento dovrà acquistare energia o se avrà un surplus da gestire. Il notebook mostra quindi il passaggio dalla previsione statistica all'informazione decisionale.

Sono presenti anche riepiloghi aggregati sui sette giorni successivi, come energia totale da acquistare ed energia totale da immettere. Questi valori rappresentano primi KPI operativi, utili per comu-

nicare il risultato della previsione a stakeholder non necessariamente coinvolti nella modellazione.

11.7 Analisi degli output

Gli output generati dal notebook sono molteplici. Il primo output è costituito dai forecast di consumo e produzione, raccolti in un dataframe indicizzato temporalmente. Questo dataframe contiene le previsioni orarie per il periodo futuro e rappresenta la base per tutte le elaborazioni successive.

Un secondo output è il bilancio energetico. Il notebook usa prima `bilancio` con una classificazione testuale, quindi `net_balance`, `to_grid` e `to_buy`; calcola inoltre somme e, nella sezione grafica, valori cumulativi. Salva anche un file CSV. La nuova architettura conserva le formule, rinomina i flussi in `grid_export` e `grid_import` e mantiene classificazione, visualizzazione e persistenza fuori dai moduli di calcolo.

Il notebook produce inoltre metriche di valutazione, in particolare MAE e RMSE, calcolate per consumo e produzione su training, validation e test. Queste metriche consentono di verificare la qualità dei modelli e di confrontare il comportamento predittivo sulle diverse porzioni temporali del dataset.

La parte visuale include grafici delle loss function, confronti tra valori reali e predetti, distribuzione degli errori, curve di consumo e produzione previste, energia da acquistare o cedere e andamento cumulativo di acquisto e immissione. Questi grafici hanno un ruolo importante perché rendono interpretabile il comportamento del modello e trasformano risultati numerici in evidenze leggibili.

Nel complesso, gli output coprono l'intero percorso: metriche per valutare il modello, forecast per rappresentare il futuro energetico, KPI per sintetizzare il bilancio e grafici per comunicare i risultati. Questa ricchezza conferma il valore del notebook come prototipo completo, ma evidenzia anche la necessità di separare le responsabilità in componenti dedicati.

11.8 Limiti dell'attuale implementazione

Il limite principale dell'attuale implementazione è la concentrazione della logica in un unico notebook. Anche se il flusso è coerente e funziona come dimostrazione end-to-end, la presenza di molte responsabilità nello stesso ambiente rende più difficile manutenzione, estensione e verifica.

La manutenzione è complessa perché le celle dipendono dall'ordine di esecuzione e da variabili condivise. Una modifica in una fase iniziale può influenzare celle successive senza che il collegamento sia immediatamente evidente. Questo aumenta il rischio di errori silenziosi, soprattutto durante attività di refactoring o sperimentazione.

La riusabilità è limitata. Funzioni e logiche sono presenti nel notebook, ma non sono ancora organizzate come moduli importabili e utilizzabili da pipeline, test o applicazioni esterne. Questo impedisce di valorizzare pienamente il lavoro già svolto e rende più difficile costruire una piattaforma

stabile.

La testabilità è ridotta. Sebbene alcune funzioni siano già isolate, il comportamento complessivo dipende da celle, stati intermedi e oggetti globali. Non esiste ancora una suite strutturata di test che consenta di verificare automaticamente caricamento dati, costruzione sequenze, training, forecast e bilancio energetico.

Un ulteriore limite è la dipendenza dall'esecuzione manuale. Il notebook richiede l'interazione dell'utente e non rappresenta ancora una pipeline operativa schedulabile, versionabile e riproducibile in modo pienamente controllato. Per passare a una piattaforma software, sarà necessario ridurre questa dipendenza e spostare la logica stabile in codice organizzato.

11.9 Opportunità di miglioramento

L'analisi del notebook evidenzia numerose opportunità di miglioramento. La prima riguarda la modularizzazione. Le funzioni di generazione o caricamento dati, pulizia, scaling, costruzione sequenze, modellazione, forecast, calcolo del bilancio e visualizzazione possono essere separate in componenti coerenti con l'architettura definita nello Strato 2.

La seconda opportunità riguarda la separazione delle responsabilità. Il notebook contiene già i confini logici della futura piattaforma, ma tali confini devono essere resi espliciti nel repository. Il Data Layer dovrà occuparsi dei dati, il Feature Engineering Layer delle trasformazioni, il Forecasting Layer dei modelli, l'Energy Balance Layer dell'interpretazione energetica e il Visualization Layer della comunicazione dei risultati.

L'introduzione di test rappresenta un'altra opportunità fondamentale. Una volta estratti i componenti, sarà possibile verificare in modo automatico che le trasformazioni mantengano coerenza, che le sequenze abbiano forma corretta, che il bilancio energetico rispetti le regole attese e che le metriche siano calcolate nella scala corretta.

La gestione strutturata dei modelli è un ulteriore ambito di evoluzione. Il notebook utilizza callback e salvataggio del modello migliore, ma la piattaforma dovrà rendere più chiaro il versionamento dei modelli, la relazione con i dataset e la conservazione delle metriche sperimentali.

Infine, l'integrazione con GitHub consentirà di trasformare il lavoro di refactoring in attività tracciabili. Le Issue potranno descrivere interventi circoscritti, i branch isoleranno le modifiche, le Pull Request renderanno revisionabile il lavoro di Codex e la documentazione Prism manterrà il collegamento tra decisioni e implementazione.

11.10 Conclusioni

Il notebook `Previsione Fabbisogno v05_all.ipynb` ha assolto correttamente il proprio ruolo di prototipo e validazione concettuale. Ha dimostrato che è possibile costruire una pipeline completa di forecasting energetico, capace di generare dati, preparare sequenze, addestrare modelli, produrre

previsioni e trasformarle in bilancio energetico operativo.

La crescita del progetto richiede ora una trasformazione. La logica presente nel notebook deve essere preservata, ma riorganizzata in una struttura software più chiara, modulare, testabile e manutenibile. Il valore dello Strato 3 consisterà proprio in questo passaggio: non creare un prototipo alternativo, ma convertire il prototipo esistente in una piattaforma evolvibile.

Il documento successivo, *12 - Refactoring Plan*, descriverà il piano operativo per realizzare questa trasformazione. L'analisi svolta in questo capitolo costituisce quindi la base funzionale per decidere cosa estrarre, in quale ordine procedere e quali responsabilità assegnare ai futuri moduli della piattaforma.

12 Refactoring Plan

12.1 Finalità del piano di refactoring

Il presente capitolo descrive il piano di refactoring del notebook di forecasting energetico verso la struttura modulare definita nello Strato 2. Il riferimento tecnico è `Previsione Fabbisogno v05_all.ipynb`. Dopo l'analisi funzionale dello stato attuale, l'obiettivo è stabilire una traiettoria ordinata di trasformazione: dal prototipo eseguibile in notebook a una piattaforma software organizzata, leggibile, testabile e predisposta all'evoluzione.

Il refactoring non viene inteso come una semplice attività di pulizia del codice. La conoscenza del notebook viene ricondotta a Data, Feature Engineering, Forecasting, Monitoring, Energy Balance e Visualization Layer, affiancati da una strategia trasversale di testing ancora pianificata.

Il repository `AI-IT-stack-lab` diventa il luogo operativo di questa trasformazione. Il notebook rimane il riferimento tecnico reale, mentre la documentazione consolidata nel progetto e in Prism fornisce il contesto metodologico e architetturale. Il piano qui descritto collega quindi tre dimensioni: ciò che il notebook già fa, ciò che l'architettura richiede e ciò che dovrà essere progressivamente reso manutenibile nel repository.

12.2 Motivazioni del refactoring

Il notebook ha assolto correttamente il proprio ruolo di prototipo. Ha permesso di costruire una pipeline completa di forecasting energetico, includendo generazione dei dati sintetici, preparazione delle serie temporali, addestramento di modelli LSTM sequence-to-sequence, previsione sui sette giorni successivi, calcolo del bilancio energetico e visualizzazione dei risultati.

Questa completezza è un punto di forza, ma rende anche evidente la necessità del refactoring. Nel notebook convivono responsabilità diverse: simulazione dei dati, pulizia, normalizzazione, costruzione delle sequenze, training, valutazione, forecast, calcolo degli indicatori energetici e grafici. Tale concentrazione è accettabile in fase esplorativa, ma diventa problematica quando il progetto

deve evolvere in una piattaforma.

Il refactoring è quindi motivato dalla necessità di rendere il sistema governabile. Una soluzione destinata a crescere non può dipendere dalla memoria dell'autore del notebook, dall'ordine manuale di esecuzione delle celle o da variabili globali distribuite lungo il flusso. Deve invece esporre confini chiari, responsabilità distinte e punti di verifica espliciti.

La trasformazione è inoltre necessaria per consentire a Codex di operare in modo efficace. Un agente di sviluppo può intervenire con maggiore sicurezza quando il lavoro è scomposto in componenti circoscritti, collegati a obiettivi chiari e riconducibili a una struttura architetturale documentata. Il refactoring rende quindi possibile una collaborazione più ordinata tra progettazione umana, assistenza architetturale di ChatGPT e implementazione assistita da Codex.

12.3 Limiti dell'attuale notebook monolitico

Il limite principale dell'implementazione attuale è la natura monolitica del notebook. Anche se il flusso è diviso in sezioni, l'esecuzione complessiva dipende dallo stato condiviso delle celle. Questo significa che molte variabili vengono create in un punto e riutilizzate più avanti, senza che il contratto tra le sezioni sia formalizzato come interfaccia stabile.

Questa impostazione riduce la manutenibilità. Una modifica al modo in cui vengono scalati i dati, ad esempio, può influenzare la costruzione delle sequenze, il calcolo delle metriche e la generazione dei grafici. Nel notebook tale dipendenza è leggibile solo seguendo il flusso completo; in una piattaforma modulare dovrà invece essere resa esplicita.

La riusabilità è un altro limite. Il notebook contiene funzioni utili, come la generazione del meteo sintetico, la pulizia dei dati, la creazione delle sequenze, la costruzione del modello e la visualizzazione degli output. Tuttavia, queste funzioni non sono ancora organizzate come componenti importabili e riutilizzabili in pipeline, test o applicazioni future.

La testabilità è limitata dalla compresenza di codice operativo, analisi, visualizzazioni e output intermedi. Alcune funzioni possono essere testate singolarmente, ma il comportamento complessivo non è ancora progettato per verifiche automatiche. Questo rende più difficile accertare che una modifica non produca regressioni.

Infine, il notebook dipende da un'esecuzione manuale. Per un prototipo è normale; per una piattaforma operativa è un vincolo. La soluzione futura dovrà poter essere eseguita in modo controllato, ripetibile e integrabile con processi di versionamento, review e validazione.

12.4 Obiettivi della trasformazione architetturale

L'obiettivo principale della trasformazione è allineare l'implementazione reale al disegno architetturale dello Strato 2. I layer già documentati non devono rimanere categorie concettuali, ma diventare aree effettive del codice e del repository.

La modularizzazione consente di separare parti della soluzione che oggi sono vicine nel notebook ma concettualmente diverse. Il caricamento o la generazione dei dati non deve essere confuso con il feature engineering; il training del modello non deve contenere la logica del bilancio energetico; la visualizzazione non deve essere necessaria per eseguire il calcolo.

La separazione delle responsabilità permette di ridurre l'accoppiamento. Ogni componente dovrà produrre output comprensibili e consumabili dal componente successivo. Questo non implica ancora descrivere l'implementazione dei moduli, ma definisce la direzione del refactoring: ogni sezione del notebook dovrà trovare una collocazione coerente nella nuova architettura.

La trasformazione mira anche a migliorare riuso, testabilità e manutenzione. Una volta estratte dal notebook, le funzioni potranno essere riutilizzate in contesti diversi, sottoposte a test e modificate senza dover rieseguire manualmente l'intero flusso. La piattaforma potrà così evolvere verso scenari più maturi, inclusa l'integrazione con dati reali, la gestione versionata dei modelli e l'automazione delle pipeline.

12.5 Strategia di migrazione progressiva

La migrazione dovrà avvenire in modo progressivo. Non è opportuno trasformare l'intero notebook in un unico intervento, perché ciò aumenterebbe il rischio di perdere coerenza con il comportamento attuale. Il notebook deve restare la baseline di confronto durante tutto il processo.

Il principio guida è estrarre una responsabilità alla volta. Ogni passaggio dovrà identificare una sezione del notebook, comprenderne il ruolo operativo, trasferirne la logica in una collocazione coerente e verificare che il comportamento rimanga equivalente. In questo modo il refactoring diventa un processo controllato, non una riscrittura.

La prima fase dovrebbe concentrarsi sulle parti a monte della pipeline, perché i layer successivi dipendono dalla qualità degli input. Dati, pulizia e preparazione delle serie temporali costituiscono la base del processo. Solo dopo aver stabilizzato queste parti sarà opportuno procedere verso modelli, bilancio energetico e visualizzazione.

La migrazione dovrà inoltre distinguere tra logica stabile e codice esplorativo. Alcune celle del notebook hanno valore operativo e dovranno essere estratte; altre hanno valore diagnostico, esplicativo o visuale e potranno essere mantenute come riferimento, trasformate in report o lasciate nei notebook di analisi.

12.6 Sezioni del notebook da estrarre

Il notebook contiene una sequenza di sezioni chiaramente riconoscibili. Le prime celle definiscono parametri generali, tra cui orizzonte storico, frequenza oraria, finestre temporali, rapporti di train-validation-test e parametri fisici per produzione e consumo. Queste informazioni dovranno essere separate dalla logica operativa, perché costituiscono configurazione di progetto.

Le funzioni di generazione dati sintetici rappresentano un primo gruppo estraibile. Esse includono la generazione del range temporale, del meteo sintetico, del consumo sintetico e della produzione fotovoltaica sintetica. Nel prototipo servono a simulare il caso d'uso; nella piattaforma potranno rimanere utili per test, demo e validazione iniziale.

Le sezioni di pulizia, imputazione e split temporale costituiscono un secondo gruppo. Qui il notebook gestisce valori mancanti, interpolazione, ordinamento temporale e suddivisione coerente dei dati. Questa parte è essenziale perché prepara il dataset per le fasi successive e riduce il rischio di errori nel training.

Il blocco di preparazione dati per i modelli è uno dei più rilevanti. Include normalizzazione, attenzione al data leakage, costruzione delle feature per consumo e produzione, gestione degli scaler e costruzione delle sequenze. Questa sezione dovrà essere trattata con particolare cautela, perché collega direttamente dati storici, meteo e target previsionali.

Le sezioni di costruzione, addestramento e valutazione dei modelli rappresentano un altro gruppo da estrarre. Il notebook costruisce due modelli distinti, addestra consumo e produzione separatamente, utilizza validation set, early stopping e metriche in scala originale. Questa logica dovrà essere separata dal resto della pipeline per rendere più controllabile l'evoluzione dei modelli.

Infine, le sezioni di forecast futuro, bilancio energetico e grafici finali costituiscono la parte orientata all'output decisionale. Esse traducono le previsioni in energia da acquistare, energia da immettere in rete, riepiloghi e visualizzazioni. Questa parte è cruciale perché collega il modello al valore operativo per gli stakeholder.

12.7 Mapping verso la nuova architettura

Il mapping tra notebook e architettura modulare consente di stabilire dove dovrà confluire ciascuna responsabilità. Questo mapping non definisce ancora l'implementazione dei moduli, ma chiarisce la destinazione logica delle sezioni esistenti.

Le funzioni di acquisizione, generazione e validazione dei dati confluiranno nel perimetro `src/data/`. In questa area rientrano la generazione del calendario storico, la simulazione dei dati meteo, consumo e produzione, e in prospettiva la sostituzione o integrazione con sorgenti reali. Il ruolo di questo layer sarà fornire dataset coerenti e affidabili ai passaggi successivi.

Le trasformazioni sui dati confluiranno in `src/features/`. Qui si collocano pulizia, imputazione, gestione temporale, normalizzazione, scaling e costruzione delle sequenze. Questa è probabilmente la parte più delicata del refactoring, perché contiene molte assunzioni operative sul modo in cui le serie temporali vengono trasformate in input per i modelli.

La costruzione e l'addestramento dei modelli confluiranno nel perimetro `src/forecasting/`. Qui rientrano la logica LSTM sequence-to-sequence, il training separato per consumo e produzione, la generazione delle previsioni e la gestione delle metriche previsionali. Il mapping dovrà rispettare la scelta già documentata di mantenere due modelli distinti.

Il calcolo del bilancio energetico confluirà in `src/energy_balance/`. In questa area dovranno essere ricondotte le regole che trasformano forecast di consumo e produzione in `net_balance`, `grid_import`, `grid_export`, totali e cumulativi. Il layer resterà indipendente dai dettagli del modello predittivo e dal file I/O.

Le funzioni grafiche confluiranno in `src/visualization/`. Qui rientrano loss, confronti reale–predetto, distribuzione degli errori, forecast combinato di consumo e produzione, `grid_import`, `grid_export` e cumulativi. La composizione dei report e il salvataggio dei file resteranno esterni al layer.

12.8 Ordine consigliato delle attività

L'ordine consigliato delle attività segue la direzione naturale del flusso dati. Prima occorre stabilizzare ciò che alimenta la pipeline, poi ciò che trasforma i dati, poi ciò che produce previsioni, infine ciò che interpreta e comunica i risultati.

La prima area da affrontare è quella dei dati. Estrarre generazione, caricamento e validazione permette di creare una base controllata e riutilizzabile. In questa fase il notebook resta utile per confrontare dataset prodotti e verificare che il comportamento rimanga coerente.

La seconda area è il feature engineering. Qui il rischio è maggiore, perché piccoli cambiamenti nello scaling, nelle finestre o nella gestione dei target possono modificare significativamente il comportamento del modello. Per questo motivo la migrazione dovrà essere graduale e accompagnata da verifiche continue sulle forme degli array, sulla coerenza temporale e sull'assenza di data leakage.

La terza area è il forecasting. Solo dopo aver stabilizzato dati e feature sarà opportuno estrarre la logica dei modelli. La separazione tra modello consumo e modello produzione dovrà essere mantenuta, così come la distinzione tra training, valutazione e previsione futura.

La quarta area è l'Energy Balance Layer. Questa parte può essere isolata con relativa chiarezza, perché riceve forecast già prodotti e applica formule deterministiche. La sua estrazione renderà testabili naming canonico, segno del bilancio, flussi, totali e cumulativi.

La quinta area è la visualizzazione. Essa dovrà essere estratta dopo aver stabilizzato gli output intermedi, perché i grafici dipendono da forecast, metriche e bilanci. Separarla alla fine riduce il rischio di confondere problemi di calcolo con problemi di rappresentazione.

12.9 Riduzione del rischio

La riduzione del rischio dipende dalla capacità di mantenere un confronto continuo con il notebook originale. Ogni componente estratto dovrà essere verificato rispetto al comportamento precedente, almeno in termini di struttura degli input, forma degli output e coerenza dei risultati principali.

Un rischio tipico del refactoring è modificare involontariamente la logica mentre si cambia la strut-

tura. Per evitarlo, il piano deve privilegiare passaggi piccoli e reversibili. La migrazione non deve introdurre contemporaneamente nuove funzionalità, nuovi modelli e nuove sorgenti dati. Prima occorre preservare il comportamento; solo dopo sarà opportuno migliorarlo.

Un secondo rischio riguarda la perdita di contesto. Il notebook contiene commenti, esperimenti, correzioni e scelte maturate durante lo sviluppo. Prism dovrà preservare questo contesto, mentre GitHub dovrà tracciare l'evoluzione dei file. In questo modo il refactoring non diventa una riscrittura opaca, ma una trasformazione documentata.

Un terzo rischio riguarda l'eccessiva automazione non governata. Codex potrà accelerare il lavoro, ma dovrà essere guidato da prompt, documentazione e review. Il valore dello Stack Operativo Ottimizzato consiste proprio nel coordinare strumenti diversi, evitando che l'automazione sostituisca la responsabilità progettuale.

12.10 Ruolo di ChatGPT e Codex

ChatGPT avrà il ruolo di supporto architetturale e metodologico. Potrà aiutare a interpretare il notebook, chiarire i confini tra layer, trasformare osservazioni tecniche in decisioni documentate e preparare istruzioni operative comprensibili per Codex. Il suo contributo principale sarà mantenere coerenza tra Business Context, Requirements, Architecture, Technical Specifications e piano di refactoring.

Codex avrà invece il ruolo di agente operativo di refactoring. A partire dal piano, dal backlog e dalla struttura del repository, potrà analizzare il codice, proporre estrazioni progressive, riorganizzare funzioni e preparare modifiche revisionabili. In questa fase, Codex non dovrà inventare una nuova architettura, ma tradurre in repository la struttura già progettata.

La collaborazione tra ChatGPT e Codex deve rimanere controllata. ChatGPT aiuta a formulare il perché e il cosa; Codex interviene sul come operativo, sempre entro i limiti definiti dalla documentazione. Prism conserva il sapere progettuale, mentre GitHub rende tracciabile l'evoluzione del codice.

12.11 Benefici attesi

Il primo beneficio atteso è la manutenibilità. Una soluzione modulare rende più semplice individuare dove intervenire, riduce la dipendenza dalla sequenza del notebook e consente di modificare una parte del sistema senza compromettere l'intero flusso.

Il secondo beneficio è la testabilità. Una volta separate le responsabilità, sarà possibile verificare singole funzioni e singoli layer: qualità dei dati, trasformazioni, sequenze, previsioni, bilancio energetico e visualizzazioni. Questo renderà più sicura ogni evoluzione successiva.

Il terzo beneficio è il riuso. Componenti estratti dal notebook potranno essere utilizzati in pipeline automatiche, esperimenti, demo, report o future applicazioni. La logica non resterà confinata in un

file interattivo, ma diventerà patrimonio software del progetto.

Il quarto beneficio è l'evoluzione futura. Una piattaforma modulare potrà integrare dati reali, sostituire modelli, aggiungere metriche, introdurre nuove visualizzazioni e supportare processi di validazione più maturi. Il refactoring è quindi una condizione abilitante, non un'attività accessoria.

12.12 Conclusioni

Il notebook `Previsione Fabbisogno v05_all.ipynb` rappresenta la sorgente tecnica reale del progetto. Il piano di refactoring qui descritto non intende sostituirlo con una soluzione astratta, ma trasformarne progressivamente la logica in una struttura coerente con l'architettura dello Strato 2.

La trasformazione dovrà procedere per passi controllati: prima i dati, poi le feature, poi il forecasting, quindi il bilancio energetico e infine la visualizzazione. Ogni passaggio dovrà preservare la conoscenza già validata, ridurre il rischio e avvicinare il repository `AI-IT-stack-lab` a una piattaforma software realmente manutenibile.

Il risultato atteso non è ancora l'implementazione dei singoli moduli, ma un piano chiaro per realizzarla. Questo capitolo collega l'analisi del notebook alla futura struttura modulare e prepara il terreno per i documenti successivi dello Strato 3, nei quali verranno approfonditi organizzazione dei moduli, workflow di sviluppo e strategia di test.

13 Module Structure

13.1 Finalità della struttura modulare

La struttura modulare descritta in questo capitolo rappresenta il risultato atteso del refactoring del notebook `Previsione Fabbisogno v05_all.ipynb`. Dopo aver analizzato il prototipo e definito il piano di refactoring, è necessario chiarire come la logica oggi concentrata nel notebook verrà distribuita nella futura piattaforma software.

La struttura proposta è coerente con l'architettura dello Strato 2 e con il piano di refactoring dello Strato 3. Il notebook rimane la sorgente tecnica principale: le funzioni di generazione dati, pulizia, normalizzazione, costruzione sequenze, training, forecasting, bilancio energetico e visualizzazione costituiscono il materiale da cui verranno estratti i componenti. La nuova organizzazione non introduce una soluzione concettualmente diversa, ma rende esplicite responsabilità che nel notebook sono già presenti in forma sequenziale.

La directory `src/` rappresenta il punto di raccolta della logica applicativa stabile. La struttura implementata è la seguente:

```
src/
```

```
|-- data/
|-- features/
|-- forecasting/
|-- monitoring/
|-- energy_balance/
‘-- visualization/
```

Questa struttura costituisce il ponte tra progettazione e implementazione. Essa consente a Codex di operare su parti circoscritte del repository, a GitHub di tracciare modifiche più leggibili e a Prism di mantenere il collegamento tra decisioni architetturali, refactoring e documentazione di progetto.

13.2 Visione d’insieme del flusso dati

Il flusso logico della piattaforma segue una direzione controllata e si articola in due rami a valle delle previsioni:

```
Data -> Features -> Forecasting -> Predictions
                                   |-> Monitoring
                                   |      ‘-> Evaluation -> Visualization
                                   ‘-> Energy Balance
                                          ‘-> KPI -> Visualization
```

Nel notebook questo flusso è già presente, ma è distribuito tra celle successive. La generazione o acquisizione dei dati alimenta il dataset storico; il preprocessing e lo scaling preparano gli input; la costruzione delle sequenze rende i dati compatibili con il modello LSTM sequence-to-sequence; il modello produce forecast di consumo e produzione; il bilancio energetico trasforma i forecast in energia da acquistare o immettere; i grafici rendono leggibile il risultato.

La modularizzazione rende questo percorso esplicito. Ogni modulo riceve input definiti, produce output comprensibili e comunica con il modulo successivo senza dipendere dai dettagli interni della sua implementazione. Questo principio è essenziale per manutenzione, testabilità e futura evoluzione della piattaforma.

13.3 Modulo `src/data/`

Il modulo `src/data/` rappresenta il punto di ingresso informativo della piattaforma. La sua responsabilità è acquisire, generare o validare i dati necessari al processo di forecasting. Nel notebook questa responsabilità è distribuita tra le funzioni `genera_date_range_storico`, `genera_meteo_sintetico`, `genera_consumo_sintetico` e `genera_produzione_sintetica`, oltre alle celle che costruiscono il dataset storico e il meteo futuro.

Gli input del modulo potranno essere dati reali di consumo, dati reali di produzione fotovoltaica, dati meteorologici storici e dati meteorologici previsionali. Nella fase iniziale, il modulo dovrà anche

preservare la possibilità di generare dati sintetici, perché questa funzionalità è utile per test, demo e confronto con il comportamento del notebook originale.

L'output atteso è costituito da dataset coerenti, indicizzati temporalmente e pronti per essere consegnati al modulo di feature engineering. Il Data Layer non deve conoscere la logica del modello predittivo, né deve calcolare bilanci energetici. Il suo compito è fornire dati affidabili e tracciabili.

Il file `loader.py` raccoglierà le responsabilità di caricamento dei dati energetici. In prospettiva, esso dovrà gestire sorgenti reali come file locali o future integrazioni esterne. Nel passaggio dal notebook alla piattaforma, il suo ruolo sarà sostituire le celle di costruzione manuale dei dataset con una modalità più ordinata e riutilizzabile.

Il file `validator.py` avrà il compito di verificare la qualità minima dei dataset. Il notebook include controlli impliciti, ad esempio sull'uso di indici temporali e sulla coerenza delle forme degli array. In una piattaforma modulare, tali verifiche dovranno essere rese più esplicite, così da intercettare dati mancanti, colonne assenti, timestamp non coerenti o frequenze inattese.

Il file `synthetic_data.py` centralizza la generazione sintetica di meteo, consumo e produzione presente nel notebook. `weather_loader.py` non è implementato e rimane una possibile estensione per future sorgenti meteorologiche reali.

La separazione del modulo data migliora manutenzione e testabilità perché consente di verificare il comportamento delle sorgenti informative senza coinvolgere modelli, bilanci o grafici. Inoltre rende più semplice sostituire i dati sintetici con dati reali quando il progetto passerà a una fase più operativa.

13.4 Modulo `src/features/`

Il modulo `src/features/` contiene la logica che trasforma i dataset validati in input utilizzabili dai modelli previsionali. Nel notebook questa parte è centrale e comprende pulizia, imputazione, split temporale, normalizzazione, gestione degli scaler e costruzione delle finestre temporali.

Gli input del modulo sono dataset storici già validati, contenenti variabili energetiche e meteorologiche. Gli output sono strutture pronte per il training, la validation, il test e l'inference. In particolare, il modulo dovrà produrre sequenze coerenti con la finestra storica di 168 ore e con l'orizzonte previsionale di 168 ore utilizzati nel notebook.

Il file `preprocessing.py` raccoglierà le trasformazioni preliminari. La funzione `pulisci_e_imputa` del notebook rappresenta un riferimento diretto: essa lavora su dataframe temporali, gestisce valori mancanti tramite interpolazione e completa eventuali lacune residue. In piattaforma, questa logica dovrà essere resa riutilizzabile e controllabile.

Il file `scaling.py` sarà dedicato alla normalizzazione. Nel notebook questa parte è stata già oggetto di attenzione metodologica, in particolare per evitare data leakage: gli scaler vengono addestrati sui dati di training e applicati poi agli altri insiemi. Isolare questa responsabilità è fondamentale, perché errori nello scaling possono alterare metriche, grafici e qualità delle previsioni.

Il file `sequence_builder.py` conterrà la logica di costruzione delle sequenze temporali. La funzione `crea_sequence_dataset` del notebook è il punto di partenza naturale. Essa traduce serie storiche e variabili meteorologiche future in strutture compatibili con il modello sequence-to-sequence. La sua estrazione renderà possibile testare forme, finestre e coerenza temporale senza dover addestrare l'intero modello.

Il modulo features è uno dei più importanti per la qualità complessiva della piattaforma. Separarlo dal Data Layer e dal Forecasting Layer consente di controllare con precisione il passaggio dai dati grezzi agli input del modello. Questa separazione riduce ambiguità, favorisce il riuso e rende più semplice introdurre nuove feature in futuro.

13.5 Modulo `src/forecasting/`

Il modulo `src/forecasting/` è responsabile della logica predittiva. Comprende definizione del modello LSTM sequence-to-sequence, addestramento separato e generazione delle previsioni. Metriche e valutazione quantitativa appartengono al Monitoring Layer.

Gli input del modulo saranno le sequenze prodotte dal Feature Engineering Layer e, in fase di inference, le variabili necessarie per costruire il decoder futuro. Gli output saranno forecast orari di consumo e produzione, espressi in scala originale e quindi interpretabili dal punto di vista energetico.

Il file `lstm_model.py` raccoglierà la definizione del modello. Nel notebook, il riferimento principale è la funzione dedicata alla costruzione del modello sequence-to-sequence. Essa costruisce una rete basata su LSTM, dropout e output sequenziale. La separazione della definizione del modello rende più semplice confrontare architetture future senza modificare preprocessing o bilancio energetico.

Il file `trainer.py` avrà il compito di governare il training. Nel notebook questa responsabilità è distribuita nelle sezioni di addestramento del modello consumi e del modello produzioni, con uso di validation set, early stopping e salvataggio del modello migliore. In una struttura modulare, il training dovrà rimanere separato dall'inference e dalla valutazione operativa del bilancio.

Il file `predictor.py` sarà dedicato alla generazione delle previsioni. Nel notebook questa logica è presente nella sezione di previsione sui prossimi sette giorni, dove vengono costruiti encoder e decoder futuri, eseguite le predizioni e invertita la normalizzazione. Isolare tale responsabilità consente di produrre forecast senza rieseguire l'intero notebook.

La separazione del modulo forecasting è motivata dalla necessità di mantenere indipendente la logica statistica dalla logica energetica. Il modello deve produrre previsioni; non deve decidere come interpretarle in termini di acquisto o immissione. Questa distinzione migliora manutenibilità, consente test più mirati e rende possibile sostituire il modello in futuro senza riscrivere tutta la pipeline.

13.6 Modulo src/monitoring/

Il modulo src/monitoring/ è implementato attraverso metrics.py ed evaluation.py. Il primo calcola MAE e RMSE; il secondo orchestra la valutazione su valori reali e predetti già espressi nella scala originale.

Il layer non esegue training, prediction, scaling, visualizzazione o drift monitoring. È un componente ufficiale e implementato dell'architettura; la sua realizzazione è registrata come attività non numerata soltanto per preservare la sequenza storica del backlog.

13.7 Modulo src/energy_balance/

Il modulo src/energy_balance/ è definito architetturalmente per trasformare i forecast in informazioni operative. Nel notebook questa responsabilità appare dopo la previsione futura attraverso bilancio, net_balance, to_grid, to_buy, somme e cumulativi.

Gli input del modulo sono due serie previsionali allineate e nella scala fisica originale: consumption e production. Gli output canonici sono net_balance, grid_import, grid_export, totali e cumulativi.

Il file balance_calculator.py conterrà le formule canoniche per intervallo. Non applicherà clipping automatico agli input, non imporrà una classificazione del caso zero e non conterrà file I/O.

Il file kpi.py calcolerà i totali total_grid_import e total_grid_export. Calcolerà inoltre le serie cumulative_grid_import e cumulative_grid_export. Non introdurrà KPI economici, ambientali o di ottimizzazione non formalizzati.

Separare l'Energy Balance Layer dal Forecasting Layer ha un valore architetturale importante. Le previsioni sono output statistici; il bilancio energetico è interpretazione di dominio. Tenere distinti questi due livelli rende il sistema più leggibile e consente di modificare regole o KPI senza toccare i modelli.

13.8 Modulo src/visualization/

Il modulo src/visualization/ è dedicato alla comunicazione dei risultati. Il materiale sorgente fornito documenta plotting.py con le sette API pubbliche e la suite tests/test_visualization_plotting.py. La v14 riportava la Issue #5 aperta; stato GitHub attuale e integrazione dei file nel branch ufficiale non sono verificati. Nel notebook questa responsabilità è presente nei grafici delle loss function, nei confronti tra valori reali e predetti, nella distribuzione degli errori e nei grafici finali su consumo, produzione, importazione/esportazione e cumulativi.

Gli input del modulo saranno forecast, valori osservati, metriche e altri elementi visuali già preparati dai layer precedenti. Gli output saranno grafici e rappresentazioni destinate all'analisi. La visualizzazione non dovrà modificare il dato, ma renderlo comprensibile.

Il file `plotting.py` nel materiale esaminato raccoglie le sette API grafiche definite nel Capitolo 22. Il modulo mantiene esclusivamente la responsabilità visuale, ricevendo valori, metriche e risultati energetici già calcolati. Non si assume che dashboard, report automatici o integrazione nel notebook siano implementati.

La separazione del Visualization Layer evita che la pipeline dipenda dalla produzione grafica. Una previsione o un bilancio devono poter essere calcolati anche senza generare grafici. Il materiale comprende una suite di test specifica per le API; questo non dimostra l'integrazione in report, dashboard, servizi o notebook ottimizzato, che restano esterni o non verificati.

13.9 Relazioni tra i moduli

La relazione tra i moduli si dirama a valle del Forecasting Layer. Le previsioni alimentano Monitoring, Energy Balance e Visualization; i risultati di Monitoring ed Energy Balance possono alimentare a loro volta Visualization. Monitoring ed Energy Balance non dipendono l'uno dall'altro. La verifica automatica rimane una capacità trasversale pianificata nella Issue #6.

Questa sequenza riduce l'accoppiamento. Ogni modulo deve poter essere compreso a partire dai suoi input e dai suoi output. Il Data Layer non deve conoscere il modello; il Forecasting Layer non deve conoscere la logica grafica; il Visualization Layer non deve modificare il bilancio energetico.

In termini architetturali, la struttura modulare rende esplicita la pipeline già presente nel notebook. Il vantaggio non è soltanto ordinare i file, ma rendere controllabile l'evoluzione. Se in futuro cambierà la sorgente dei dati, l'impatto sarà concentrato nel modulo data. Se cambierà il modello, il cambiamento riguarderà il forecasting. Se cambieranno gli indicatori richiesti dagli stakeholder, l'intervento sarà localizzato nel bilancio energetico o nella visualizzazione.

13.10 Benefici della struttura modulare

Il primo beneficio è la manutenzione. Una struttura modulare consente di individuare rapidamente dove si trova una responsabilità e dove deve essere applicata una modifica. Questo riduce il costo cognitivo del progetto e rende il repository più leggibile per persone e strumenti AI.

Il secondo beneficio è il riuso. Le funzioni estratte dal notebook potranno essere utilizzate in pipeline automatiche, test, notebook di analisi, report o futuri servizi applicativi. La conoscenza non rimarrà confinata in una sequenza di celle, ma diventerà patrimonio software riutilizzabile.

Il terzo beneficio è la testabilità. Ogni modulo potrà essere verificato con controlli mirati: dati validi, scaling coerente, sequenze con forma attesa, forecast generabili, bilanci corretti, grafici prodotti a partire da input noti. Questo rende più sicuro il passaggio verso fasi successive di validazione.

Il quarto beneficio è l'evoluzione futura. Una piattaforma modulare può integrare dati reali, nuovi modelli, nuove metriche o nuove interfacce senza dover riscrivere l'intera soluzione. Questa capacità è essenziale per un progetto di forecasting energetico, che per natura dovrà adattarsi a dati, impianti,

comportamenti e requisiti in evoluzione.

13.11 Conclusioni

La struttura descritta in questo capitolo rappresenta la forma modulare definitiva verso cui dovrà convergere il refactoring del notebook. Essa traduce in organizzazione software i layer progettati nello Strato 2 e rende operativo il piano di refactoring definito nello Strato 3.

Il notebook `Previsione Fabbisogno v05_all.ipynb` rimane il riferimento tecnico principale: le sue funzioni e sezioni indicano cosa deve essere preservato, mentre la nuova struttura stabilisce dove ciascuna responsabilità dovrà essere collocata. Il risultato atteso non è una semplice riorganizzazione dei file, ma una piattaforma più chiara, manutenibile, testabile e pronta per evoluzioni successive.

Questa struttura costituisce il ponte tra progettazione e implementazione operativa. Da un lato conserva la coerenza con Business Context, Requirements, Architecture, Refactoring Plan e Technical Specifications; dall'altro prepara il terreno per il lavoro concreto di Codex, che potrà trasformare progressivamente il notebook in moduli Python revisionabili, versionati e integrabili nel repository `AI-IT-stack-lab`.

14 Development Workflow

14.1 Dal disegno architetturale all'implementazione controllata

Il notebook `Previsione Fabbisogno v05_all.ipynb` rappresenta la sorgente tecnica originaria del progetto. Al suo interno sono già presenti gli elementi fondamentali della soluzione: generazione dei dati sintetici, costruzione delle serie temporali, pulizia e imputazione, suddivisione train/validation/test, normalizzazione, creazione delle finestre temporali, addestramento dei modelli LSTM sequence-to-sequence, produzione dei forecast e calcolo del bilancio energetico tra consumo e produzione fotovoltaica.

Questa forma è adatta alla sperimentazione, perché consente di osservare l'intero flusso in modo sequenziale e immediato. Tuttavia, non è sufficiente per sostenere una piattaforma software destinata a evolvere nel tempo. Il passaggio dall'analisi alla realizzazione richiede quindi un cambio di disciplina: il codice non viene più trattato come una sequenza di celle eseguibili, ma come un insieme di componenti con responsabilità definite, versionati, revisionati e verificabili.

Il Development Workflow descritto in questo capitolo costituisce il processo operativo attraverso cui tale trasformazione viene governata. Esso applica al progetto il principio dello Stack Operativo Ottimizzato, integrando ChatGPT, Codex, GitHub e Prism in un flusso unico. ChatGPT supporta la chiarificazione architetturale e tecnica; GitHub organizza il lavoro in Issue, branch e Pull Request; Codex agisce come agente di sviluppo incaricato di produrre modifiche concrete al repository; Prism conserva la memoria documentale del progetto e mantiene il collegamento tra decisioni, attività e

risultati.

L'obiettivo non è automatizzare indiscriminatamente lo sviluppo, ma costruire una collaborazione uomo-AI controllata. L'intelligenza artificiale accelera analisi, refactoring e produzione di codice, mentre la responsabilità finale delle scelte, delle revisioni e dell'integrazione resta umana. In questo modo il progetto può beneficiare della velocità operativa degli strumenti AI senza perdere governo, tracciabilità e qualità.

14.2 Il ruolo delle Issue GitHub come unità di lavoro

Nello Strato 2 il disegno architetturale è stato trasformato in un backlog GitHub iniziale. Questo passaggio è essenziale perché rende eseguibile la progettazione. Un'architettura descritta solo in termini generali non è ancora sufficiente per guidare un refactoring; deve essere convertita in attività circoscritte, ordinate e verificabili.

Le Issue GitHub svolgono questa funzione. Ogni Issue rappresenta una unità di lavoro autonoma, dotata di un obiettivo tecnico, di un perimetro, di deliverable attesi e di criteri di coerenza rispetto all'architettura. La Issue non è quindi un semplice promemoria, ma il contratto operativo che collega la decisione progettuale all'implementazione.

Nel caso del refactoring del notebook, le Issue permettono di evitare una riscrittura indistinta dell'intero prototipo. Il lavoro viene suddiviso per layer: dati, trasformazioni, forecasting, Energy Balance, Visualization e suite iniziale trasversale di test. Il materiale fornito documenta già codice e test specifici per la visualizzazione, ma non dimostra la revisione o integrazione nel branch ufficiale. Questa sequenza riduce il rischio perché ogni modifica può essere compresa, revisionata e integrata prima di procedere alla successiva.

La trasformazione delle Issue in attività operative avviene attraverso una descrizione chiara del contesto. Una buona Issue deve indicare quale parte del notebook viene coinvolta, quale responsabilità deve essere estratta, quali file o moduli devono essere prodotti, quali dipendenze devono essere rispettate e quali comportamenti del notebook devono rimanere invariati. In questo modo Codex riceve un input sufficientemente strutturato per lavorare in modo mirato, mentre il reviewer umano dispone di una base oggettiva per valutare il risultato.

14.3 Workflow operativo: Issue, Codex, Pull Request, Review, Merge, Repository

Il flusso operativo adottato dal progetto segue una sequenza lineare e controllabile:

`Issue` → `Codex` → `Pull Request` → `Review Umana` → `Merge` → `Repository`

La Issue è il punto di partenza. Essa descrive il problema da risolvere e il risultato atteso. Nel contesto del progetto, una Issue può chiedere l'estrazione del Data Layer, la separazione della logica

di feature engineering o la creazione di un modulo dedicato al bilancio energetico. La Issue deve essere sufficientemente precisa da impedire interventi troppo ampi, ma non così rigida da impedire a Codex di proporre una soluzione coerente con il repository.

Codex interviene come agente di sviluppo. A partire dalla Issue, analizza il repository, individua i file coinvolti, confronta il codice esistente con la struttura modulare prevista e genera le modifiche necessarie. Il suo lavoro deve rimanere circoscritto al task assegnato. Se l'obiettivo è estrarre la generazione dei dati, Codex non deve modificare la strategia di forecasting; se l'obiettivo è separare il bilancio energetico, non deve ridefinire la struttura dei dati. Questo vincolo è fondamentale per mantenere il refactoring incrementale e revisionabile.

Le modifiche prodotte da Codex vengono raccolte in un branch di sviluppo. Il branch isola il lavoro rispetto al ramo principale del repository e consente di sperimentare senza compromettere la versione stabile del codice. Ogni branch dovrebbe corrispondere a una Issue o a una parte ben delimitata di essa, così da mantenere una relazione leggibile tra richiesta, implementazione e revisione.

Quando il lavoro è pronto, Codex prepara una Pull Request. La Pull Request rende visibile la proposta di modifica: mostra i file aggiunti o modificati, consente di leggere il diff, collega il lavoro alla Issue di origine e permette di discutere eventuali correzioni. Essa rappresenta il punto in cui il contributo dell'agente AI diventa oggetto di controllo umano.

La review umana verifica che il risultato sia coerente con l'architettura, con il piano di refactoring e con il comportamento del notebook originario. Il reviewer controlla che i confini tra moduli siano rispettati, che non siano state introdotte dipendenze improprie, che i nomi siano comprensibili, che il codice sia manutenibile e che la modifica non allarghi indebitamente il perimetro della Issue. ChatGPT può supportare questa fase come strumento di analisi, spiegazione e confronto, ma non sostituisce la decisione finale del reviewer.

Dopo la review, la Pull Request può essere corretta, approvata e infine integrata tramite merge. Il merge trasferisce il contenuto del branch nel ramo principale, aggiornando la sorgente ufficiale del codice. Da quel momento il repository GitHub diventa il riferimento consolidato per quella parte di implementazione. Il notebook rimane memoria tecnica del prototipo, ma la piattaforma evolve nel repository attraverso moduli progressivamente più stabili.

14.4 Applicazione del workflow alle Issue dello Strato 2

Le sei Issue consolidate costituiscono la sequenza operativa ufficiale del refactoring. Il Monitoring Layer è un layer ufficiale già implementato; la sua realizzazione è documentata separatamente come attività non numerata, poiché è emersa dopo la definizione del backlog storico.

Issue #1 – Estrazione del Data Layer. La prima attività riguarda il punto di ingresso dei dati. Codex utilizza le parti del notebook dedicate alla generazione dell'indice temporale, al meteo sintetico, al consumo sintetico e alla produzione fotovoltaica sintetica come materiale di partenza per costruire un Data Layer separato. L'obiettivo è spostare queste responsabilità in moduli dedicati, preservando la logica originaria ma rendendola riutilizzabile. La Pull Request associata deve

mostrare chiaramente quali funzioni sono state estratte, quali interfacce sono state introdotte e come il resto del flusso potrà consumare dataset coerenti.

Issue #2 – Creazione del Feature Engineering Layer. La seconda attività interviene sulle trasformazioni applicate ai dati. Nel notebook questa area comprende pulizia, imputazione, split temporale, scaling e costruzione delle sequenze per il modello LSTM. Codex deve separare tali responsabilità in componenti leggibili, evitando che la preparazione degli input resti intrecciata con l'addestramento del modello. Il risultato atteso è un layer capace di ricevere dati validati e produrre strutture pronte per il forecasting.

Issue #3 – Creazione del Forecasting Layer. La terza attività riguarda il cuore predittivo della soluzione. Il notebook costruisce modelli LSTM sequence-to-sequence per stimare consumo e produzione sui sette giorni successivi. Nel refactoring, Codex deve isolare la definizione del modello, la logica di training e la logica di inferenza. Questo consente di distinguere ciò che appartiene all'architettura del modello da ciò che riguarda l'esecuzione dell'addestramento o la produzione di previsioni. La Pull Request deve rendere più chiara questa separazione senza modificare arbitrariamente l'impostazione tecnica del prototipo.

Issue #4 – Separazione della logica Energy Balance. La quarta attività applica ai forecast il contratto canonico `net_balance`, `grid_import` e `grid_export`, separando calcoli per intervallo e KPI totali o cumulativi. Codex dovrà preservare scala e allineamento, senza clipping automatico, classificazione obbligatoria del valore zero o file I/O nei moduli.

Issue #5 – Creazione del Visualization Layer. Il materiale sorgente fornito contiene `plotting.py` e la suite dedicata di test, con sette API che rappresentano risultati già elaborati senza ricalcolare previsioni, metriche, bilanci o KPI. La v14 riportava la Issue aperta; stato GitHub attuale, Pull Request, review e merge non sono verificati. Il risultato di test comunicato è di 18 test superati in 5,14 secondi con backend Agg.

Issue #6 – Introduzione della suite di test. La sesta attività resta aperta/pianificata nel backlog documentale e riguarda la suite trasversale di unit test, integration test e controlli di regressione dell'intera piattaforma. La suite visuale fornita è una verifica specifica di un singolo layer e non prova il completamento di questa attività complessiva; lo stato amministrativo GitHub non è verificato.

Attività Monitoring Layer. I moduli `metrics.py` ed `evaluation.py` sono stati implementati dopo la definizione del backlog iniziale. L'attività non riceve un numero retroattivo, ma il layer rimane pienamente ufficiale, tracciato nella documentazione e presente nel repository.

Questa sequenza permette di trasformare il notebook senza perdere continuità. Ogni Issue prende una parte del prototipo, la rende esplicita in un modulo e la sottopone a un ciclo di sviluppo controllato. Il risultato non è una riscrittura improvvisa, ma una migrazione progressiva verso una piattaforma software strutturata.

14.5 Branch di sviluppo e Pull Request come strumenti di qualità

Il branch di sviluppo è lo spazio in cui una modifica può essere costruita senza alterare immediatamente il ramo principale. Questa separazione è particolarmente importante in un progetto assistito da AI, perché permette di valutare il contributo di Codex prima che diventi parte della base ufficiale del codice.

Ogni branch deve avere un obiettivo riconoscibile. Una convenzione semplice può collegare il nome del branch alla Issue di riferimento, rendendo immediata la tracciabilità. Il branch contiene i commit prodotti durante il lavoro: ciascun commit rappresenta un passo della trasformazione e consente di ricostruire l'evoluzione della modifica.

La Pull Request è il meccanismo attraverso cui il branch viene proposto per l'integrazione. Essa non serve solo a "chiedere il merge", ma a rendere il cambiamento leggibile. Il reviewer può osservare quali file sono stati creati, quali funzioni sono state spostate, quali interfacce sono state introdotte e quali parti del notebook sono state trasformate in moduli applicativi.

Nel workflow adottato, Codex può generare modifiche al codice, organizzare commit coerenti e preparare la Pull Request. Tuttavia, l'approvazione resta un atto umano. Questa distinzione preserva il controllo del progetto: l'agente AI accelera l'esecuzione, mentre la review umana garantisce coerenza, qualità e responsabilità.

14.6 Continuous Integration e controlli prima del merge

La Continuous Integration introduce un ulteriore livello di controllo nel workflow. Prima che una Pull Request venga integrata, il repository può eseguire automaticamente una serie di verifiche. In questa fase il concetto rilevante non è ancora il dettaglio dei singoli test, ma il principio operativo: nessuna modifica dovrebbe entrare nel ramo principale senza essere stata sottoposta a controlli ripetibili.

Nel progetto, la Continuous Integration avrà il compito di eseguire i controlli automatici associati alla suite di test e di segnalare eventuali regressioni prima del merge. Questo meccanismo è particolarmente utile durante il refactoring del notebook, perché consente di verificare che l'estrazione di funzioni e moduli non rompa il comportamento atteso.

La CI non sostituisce la review umana. Essa controlla aspetti eseguibili e ripetibili, mentre il reviewer valuta aspetti architetturali, leggibilità, coerenza con la roadmap e adeguatezza della soluzione. Insieme, review e CI compongono un doppio presidio: automatico per la stabilità tecnica, umano per la qualità progettuale.

14.7 GitHub come sorgente ufficiale e piattaforma di tracciabilità

GitHub svolge il ruolo di sorgente ufficiale del codice. Il repository non è soltanto un contenitore di file, ma il luogo in cui il progetto rende verificabile la propria evoluzione. Ogni modifica può

essere collegata a una Issue, a un branch, a una Pull Request, a una review e a un merge. Questo collegamento produce tracciabilità completa.

Nel passaggio dal notebook alla piattaforma, tale tracciabilità è decisiva. Senza un repository governato, il refactoring rischierebbe di diventare una sequenza di modifiche difficili da ricostruire. Con GitHub, invece, ogni trasformazione può essere associata a un motivo: una funzione viene spostata perché appartiene al Data Layer; una trasformazione viene isolata perché appartiene al Feature Engineering Layer; il calcolo del fabbisogno netto viene separato perché appartiene all'Energy Balance Layer.

GitHub supporta inoltre la collaborazione tra persone e strumenti AI. Le Issue rendono esplicito il lavoro, Codex produce una proposta, la Pull Request la espone, la review la valuta e il merge la consolida. Questo ciclo crea un ambiente in cui la collaborazione non dipende da scambi informali, ma da artefatti osservabili e versionati.

14.8 ChatGPT come supporto architetturale e tecnico

ChatGPT interviene nel workflow con un ruolo diverso da Codex. Non è l'agente incaricato di modificare direttamente il repository, ma il supporto di ragionamento che aiuta a mantenere coerenza tra requisiti, architettura, backlog e implementazione.

Durante la preparazione delle Issue, ChatGPT può aiutare a formulare task più chiari, a individuare dipendenze tra attività, a definire criteri di accettazione e a trasformare decisioni progettuali in istruzioni operative. Durante la review, può essere utilizzato per analizzare una modifica, spiegare un diff, confrontare una soluzione con l'architettura prevista o evidenziare potenziali rischi.

Il valore di ChatGPT risiede quindi nella capacità di ampliare la comprensione del progetto. Esso aiuta a evitare che il lavoro di sviluppo diventi una sequenza di interventi locali scollegati dalla visione complessiva. In un progetto in cui il notebook originale viene progressivamente scomposto in layer, questa funzione di raccordo è particolarmente importante.

14.9 Prism come memoria documentale del progetto

Prism rappresenta la memoria documentale dello Stack Operativo Ottimizzato. Se GitHub conserva la storia del codice, Prism conserva la storia del significato progettuale. In Prism vengono raccolte le decisioni architetturali, l'avanzamento delle attività, le modifiche significative, i risultati delle review e lo stato della roadmap.

Questo ruolo è complementare a quello del repository. Una Pull Request può mostrare che una funzione è stata spostata, ma Prism deve spiegare perché quel cambiamento è coerente con l'evoluzione del progetto. Una Issue può indicare cosa deve essere fatto, ma Prism deve mantenere il collegamento con i capitoli di architettura, refactoring e struttura modulare. Una review può approvare una modifica, ma Prism deve preservare eventuali osservazioni utili per decisioni future.

Nel workflow operativo, Prism viene quindi aggiornato lungo tutto il ciclo di sviluppo. Quando una Issue viene avviata, Prism può registrare il contesto e l'obiettivo; quando Codex produce una Pull Request, Prism può documentare le modifiche principali; quando la review individua correzioni o approva il lavoro, Prism può conservare l'esito; quando il merge viene completato, Prism può aggiornare lo stato della roadmap.

In questo modo la documentazione non rimane separata dall'implementazione. Essa cresce insieme al codice e consente al progetto di mantenere memoria delle scelte, anche quando le attività diventano numerose o distribuite nel tempo.

14.10 Benefici del workflow integrato

Il workflow descritto produce benefici concreti per il progetto. Il primo è un maggiore controllo. Ogni attività viene avviata da una Issue, sviluppata in un branch, discussa in una Pull Request e integrata solo dopo review. Questo riduce la probabilità che modifiche estese o non coerenti entrino nel repository principale.

Il secondo beneficio è la riduzione del rischio. Il refactoring del notebook viene affrontato per passi incrementali, ciascuno associato a una responsabilità architetturale. Questo rende più semplice individuare eventuali problemi e correggerli prima che si propaghino ad altri layer.

Il terzo beneficio è la tracciabilità completa. Ogni modifica può essere ricondotta a una Issue, a una decisione progettuale, a una Pull Request e a uno stato della roadmap. Questa tracciabilità è particolarmente importante in una piattaforma di forecasting energetico, dove dati, modelli, metriche e logiche operative devono poter essere spiegati e governati.

Il quarto beneficio è il miglioramento della qualità del codice. La separazione in layer, la review umana, l'uso di branch dedicati e l'introduzione progressiva della Continuous Integration favoriscono codice più leggibile, modulare e manutenibile.

Il quinto beneficio è una collaborazione uomo-AI strutturata. ChatGPT, Codex, GitHub e Prism non operano come strumenti isolati, ma come componenti di un processo coordinato. L'AI contribuisce alla velocità e alla capacità di analisi; l'essere umano mantiene responsabilità, giudizio e direzione; GitHub e Prism rendono osservabile l'intero percorso.

14.11 Conclusioni

Il Development Workflow costituisce la base operativa dello Strato 3. Esso permette di trasformare il notebook `Previsione Fabbisogno v05_all.ipynb` in una piattaforma software strutturata senza perdere il controllo sul processo. Le Issue definiscono il lavoro, Codex produce modifiche implementative, le Pull Request rendono visibile il cambiamento, la review umana garantisce qualità, la Continuous Integration prepara controlli ripetibili, GitHub consolida il codice e Prism conserva la memoria progettuale.

Questo workflow rende il refactoring governabile. Ogni passo è piccolo, tracciato e collegato alla struttura architetturale definita nei capitoli precedenti. La piattaforma può quindi evolvere dal prototipo notebook verso un repository modulare, mantenendo coerenza tra progettazione, implementazione e documentazione.

Il capitolo successivo, `15 - Testing Strategy`, costruisce su questa base distinguendo la suite visuale specifica documentata nel materiale fornito dalla suite iniziale trasversale ancora pianificata. La validazione complessiva e il controllo delle regressioni restano da affrontare per l'evoluzione del progetto.

15 Testing Strategy

15.1 Il testing come componente dello Stack Operativo Ottimizzato

La strategia di testing rappresenta il passaggio naturale dopo la definizione del Development Workflow. Il capitolo precedente ha descritto come il notebook `Previsione Fabbisogno v05_all.ipynb` venga trasformato in una piattaforma software attraverso Issue, branch, Pull Request, review umana e integrazione nel repository GitHub. Questo capitolo chiarisce come tale trasformazione venga resa affidabile attraverso una disciplina di verifica progressiva.

Nel notebook originario il controllo della correttezza avviene soprattutto attraverso l'esecuzione sequenziale delle celle, l'osservazione dei grafici, il confronto delle metriche e la verifica manuale degli output. Questo approccio è adeguato nella fase esplorativa, ma diventa fragile quando la logica viene distribuita in moduli separati. Nel momento in cui funzioni come `genera_meteo_sintetico`, `pulisci_e_imputa`, `crea_sequence_dataset`, `build_seq2seq_model` o i calcoli di bilancio energetico vengono estratti dal notebook, occorre garantire che continuino a comportarsi in modo coerente.

Il testing diventa quindi una componente essenziale dello Stack Operativo Ottimizzato. Esso non è un'attività finale, da eseguire soltanto quando l'implementazione è conclusa, ma un meccanismo di controllo che accompagna il refactoring. Ogni modulo estratto deve essere verificabile; ogni Pull Request deve poter dimostrare di non compromettere funzionalità già consolidate; ogni modifica deve lasciare il repository in uno stato più ordinato, non più incerto.

La qualità della futura piattaforma di forecasting energetico dipenderà dalla capacità di mantenere insieme tre dimensioni: affidabilità dei risultati, manutenibilità del codice e tracciabilità delle modifiche. I test contribuiscono a tutte e tre. Rendono più sicuro il comportamento tecnico, consentono di modificare il codice con minore rischio e producono evidenza oggettiva a supporto della review umana.

15.2 Introduzione progressiva dei test durante il refactoring

La suite di test verrà introdotta progressivamente, seguendo la stessa logica incrementale adottata per il refactoring. Non è necessario costruire immediatamente una copertura completa dell'intera piattaforma; è invece necessario associare a ogni estrazione modulare un primo insieme di verifiche mirate.

Quando viene creato il Data Layer, devono nascere i primi test sui dataset, sugli indici temporali, sulle colonne richieste e sui valori mancanti. Quando viene separato il Feature Engineering Layer, devono essere aggiunti controlli sulle trasformazioni, sulla normalizzazione e sulla costruzione delle sequenze temporali. Quando viene isolato il Forecasting Layer, devono essere verificati la costruzione del modello, il caricamento degli artefatti e il comportamento dell'inferenza su input controllati. Quando viene separato l'Energy Balance Layer, devono essere controllati i calcoli energetici e gli indicatori di business. Quando viene organizzato il Visualization Layer, devono essere verificati la produzione dei grafici e la coerenza minima dei report.

Questo approccio consente di evitare due rischi opposti. Il primo rischio è rinviare i test a una fase finale, quando il codice è già cambiato molto e diventa difficile capire l'origine degli errori. Il secondo rischio è bloccare il refactoring in attesa di una suite perfetta. La strategia corretta è intermedia: test piccoli, mirati, introdotti insieme ai moduli e rafforzati man mano che la piattaforma cresce.

15.3 Livelli di testing previsti

La strategia prevede più livelli di verifica, ciascuno con una funzione specifica.

Gli Unit Test controllano singole funzioni o componenti isolati. Servono a verificare che una responsabilità elementare produca il risultato atteso in condizioni note. Nel progetto, esempi di unit test riguardano la validazione di un dataset, l'imputazione dei valori mancanti, la costruzione di una finestra temporale, il calcolo del fabbisogno netto o la generazione di un grafico senza errori.

Gli Integration Test verificano che più moduli collaborino correttamente. Sono necessari perché il valore della piattaforma non deriva da funzioni isolate, ma dal flusso complessivo: dati validati, trasformazioni, forecasting, bilancio energetico e output finale. Un integration test può controllare che un dataset prodotto dal Data Layer sia accettato dal Feature Engineering Layer, o che un output del Forecasting Layer possa essere utilizzato dall'Energy Balance Layer senza adattamenti manuali.

I Validation Test verificano la coerenza della soluzione rispetto alle regole attese del dominio e al comportamento del prototipo. Nel contesto energetico, non basta che una funzione venga eseguita senza errori: occorre anche che i risultati siano plausibili. Eventuali valori negativi di produzione devono essere intercettati o governati a monte mediante una policy esplicita; l'Energy Balance Layer non deve correggerli silenziosamente. `net_balance` deve essere coerente con produzione meno consumo e le sequenze devono rispettare la granularità prevista.

I Regression Test proteggono funzionalità già esistenti da modifiche successive. Sono fondamentali

durante il refactoring, perché una modifica apparentemente innocua può alterare comportamenti consolidati. Cambiare il formato di una colonna, modificare l'ordine delle feature, aggiornare una funzione di scaling o ritoccare la costruzione delle finestre temporali può produrre effetti a valle su training, inferenza, bilancio energetico e report. I regression test servono a intercettare questi effetti prima del merge.

15.4 Strategia di verifica per il modulo `src/data/`

Il modulo `src/data/` costituisce il punto di ingresso informativo della piattaforma. Nel notebook questa responsabilità è rappresentata dalle funzioni che generano l'indice temporale storico, simulano il meteo, costruiscono consumo e produzione sintetici e preparano i dati per le fasi successive. Nel repository modulare, questo layer dovrà gestire dati sintetici nella fase iniziale e, in prospettiva, dati reali provenienti da sorgenti esterne o file locali.

I test del Data Layer dovranno verificare innanzitutto la validità strutturale dei dataset. Sarà necessario controllare che l'indice temporale sia effettivamente un `DatetimeIndex`, che la frequenza sia coerente con l'impostazione oraria del progetto, che le colonne attese siano presenti e che i tipi dei dati siano compatibili con le trasformazioni successive.

Un secondo insieme di controlli riguarderà i dati mancanti. Il notebook prevede una fase di pulizia e imputazione; nella piattaforma questa responsabilità deve essere resa esplicita e verificabile. I test dovranno accertare che valori assenti vengano individuati, trattati secondo la logica prevista e non trasferiti in modo incontrollato ai layer successivi.

Un terzo ambito riguarda formati e anomalie. Timestamp duplicati, dati fuori ordine, valori negativi dove non ammessi, produzione fotovoltaica anomala o colonne meteorologiche incomplete possono compromettere l'intero flusso. I test devono quindi mitigare il rischio che input non validi producano errori tardivi, difficili da interpretare, durante il training o il calcolo del bilancio energetico.

15.5 Strategia di verifica per il modulo `src/features/`

Il modulo `src/features/` trasforma i dataset validati in input utilizzabili dai modelli previsionali. Nel notebook questa area comprende pulizia, imputazione, split temporale, normalizzazione, gestione degli scaler e costruzione delle sequenze per il modello LSTM sequence-to-sequence.

I test dovranno verificare la correttezza delle trasformazioni. Una trasformazione errata può non produrre un errore immediato, ma alterare silenziosamente il significato delle feature. Per questo motivo occorre controllare che le colonne vengano preservate o trasformate secondo le regole previste, che lo split temporale mantenga l'ordine cronologico e che non si introducano contaminazioni tra training, validation e test.

La normalizzazione richiede particolare attenzione. Gli scaler devono essere addestrati sui dati corretti, applicati in modo coerente e, quando necessario, utilizzati per riportare i risultati alla scala originale. Un uso improprio dello scaler può generare valutazioni fuorvianti o rendere non

confrontabili output diversi. I test dovranno quindi controllare forme, range attesi e coerenza delle trasformazioni inverse.

La costruzione delle sequenze temporali è uno dei punti più delicati del notebook. La funzione che prepara finestre passate e future deve rispettare `PAST_WINDOW` e `FUTURE_WINDOW`, preservare l'allineamento tra feature e target e produrre array con dimensioni coerenti con il modello. I test devono mitigare il rischio di errori di indicizzazione, slittamenti temporali o mismatch di forma che potrebbero compromettere il forecasting senza essere immediatamente visibili.

15.6 Strategia di verifica per il modulo `src/forecasting/`

Il modulo `src/forecasting/` contiene la logica predittiva della piattaforma. Nel notebook questa responsabilità include la costruzione del modello LSTM sequence-to-sequence, l'addestramento dei modelli di consumo e produzione, la generazione delle previsioni e la gestione degli artefatti collegati al modello.

I test su questo layer devono verificare il corretto funzionamento del training in forma controllata. Non è necessario che ogni test esegua addestramenti lunghi o valuti prestazioni predittive complesse; è invece importante verificare che il processo di training accetti input validi, produca un modello utilizzabile e gestisca correttamente configurazioni minime pensate per l'ambiente di test.

Un secondo ambito riguarda il caricamento dei modelli. La futura piattaforma dovrà poter distinguere tra definizione del modello, addestramento, salvataggio e riutilizzo. I test dovranno quindi verificare che un modello salvato possa essere ricaricato e usato in modo coerente, riducendo il rischio di incompatibilità tra training e inferenza.

L'inferenza dovrà essere controllata su input noti e dimensioni attese. Il risultato deve avere una forma compatibile con l'orizzonte previsionale, non deve contenere valori non numerici e deve poter essere consegnato all'Energy Balance Layer senza trasformazioni manuali. In questa fase non si trattano ancora metriche avanzate di forecasting, monitoraggio del modello o retraining: l'obiettivo del testing è garantire che il componente funzioni correttamente nel flusso software.

15.7 Strategia di verifica per il modulo `src/monitoring/`

Il Monitoring Layer implementa MAE, RMSE e l'orchestrazione della valutazione. La futura suite dovrà verificare formule metriche, validazione minima degli input numerici, coerenza delle forme e uso esclusivo di valori già riportati alla scala originale.

La verifica non deve coinvolgere training, prediction, grafici, drift monitoring o alerting, perché tali responsabilità non appartengono ai moduli `metrics.py` ed `evaluation.py`.

15.8 Strategia di verifica per il modulo `src/energy_balance/`

Il modulo `src/energy_balance/` traduce i forecast in informazione operativa mediante `net_balance`, `grid_import` e `grid_export`. I test dovranno verificare anche totali e cumulativi prodotti da `kpi.py`.

I test dovranno verificare il segno canonico: `net_balance` è positivo quando la produzione supera il consumo e negativo nel caso opposto. Per il valore zero entrambi i flussi devono essere zero, senza richiedere etichette. Casi controllati dovranno inoltre confermare che valori negativi in ingresso non vengano corretti silenziosamente mediante clipping.

I KPI dovranno essere verificati rispetto a casi controllati, confrontando somme e somme cumulative con risultati calcolabili manualmente. Dovranno inoltre essere verificati allineamento temporale, scala fisica e assenza di file I/O nei moduli.

I test di business hanno un ruolo specifico: verificano che il significato operativo del risultato resti corretto. La piattaforma non deve limitarsi a produrre numeri, ma deve produrre numeri interpretabili. Il layer di bilancio energetico è quindi uno dei punti in cui il testing protegge direttamente il valore decisionale della soluzione.

15.9 Strategia di verifica per il modulo `src/visualization/`

Il modulo `src/visualization/plotting.py` rende leggibili risultati già calcolati. Il materiale fornito include `tests/test_visualization_plotting.py`, una suite specifica per le sette API; non rappresenta la suite trasversale completa della Issue #6.

La suite fornita copre il contratto dei nomi, parametri e default delle API; il tipo e gli elementi essenziali degli output; input validi e varie categorie di input invalidi; finestre, timestamp e allineamento temporale; configurazione di titolo, dimensione, griglia e colori; mancata modifica degli input; controlli statici su dipendenze, file I/O e ricalcoli. È stato comunicato che 18 test sono passati in 5,14 secondi con backend Agg. La suite non valuta la qualità estetica, non esaurisce tutte le combinazioni possibili e non dimostra l'integrazione end-to-end con gli altri layer o con il notebook.

Un ulteriore aspetto riguarda la coerenza delle rappresentazioni. Se un grafico confronta consumo reale e previsto, le serie devono rispettare il contratto della finestra; se rappresenta risultati energetici, deve utilizzare `grid_import`, `grid_export` e cumulativi già prodotti dal layer responsabile.

15.10 Regressioni software e rischio delle modifiche apparentemente innocue

Una regressione software si verifica quando una modifica introduce un errore in una funzionalità che in precedenza funzionava correttamente. Durante un refactoring questo rischio è particolarmente elevato, perché il codice viene spostato, rinominato, separato e reso più modulare pur dovendo conservare lo stesso comportamento logico.

Nel progetto, una regressione può nascere da interventi molto piccoli. Una modifica alla funzione di pulizia dei dati può cambiare il numero di righe disponibili per la costruzione delle sequenze. Una variazione nello split temporale può introdurre leakage tra training e validation. Un aggiornamento alla normalizzazione può rendere incoerente l'inverse transform. Una modifica al formato dell'output del forecasting può rompere il calcolo del bilancio energetico. Un cambiamento nei nomi delle colonne può produrre report errati pur senza generare errori immediati.

I regression test servono a rendere visibili questi problemi. Essi costruiscono una memoria automatica del comportamento atteso e la rieseguono a ogni modifica rilevante. In questo senso, i test svolgono una funzione analoga a Prism sul piano documentale: preservano continuità. Prism conserva il perché delle decisioni; i test conservano il comportamento tecnico che non deve essere compromesso.

15.11 GitHub Actions e Continuous Integration

GitHub Actions e la Continuous Integration permettono di eseguire automaticamente la suite di test durante il ciclo di sviluppo. Ogni Pull Request dovrà essere sottoposta ai controlli previsti prima del merge nel ramo principale. Questo principio rafforza il workflow definito nel capitolo precedente: la Pull Request non è soltanto un luogo di discussione, ma anche un punto di verifica tecnica.

Nel processo operativo, Codex potrà proporre modifiche e aggiornare i test; GitHub Actions eseguirà i controlli automatici; il reviewer umano valuterà sia il contenuto del codice sia l'esito della suite. Una Pull Request che non supera i test non dovrebbe essere integrata, salvo decisioni esplicite e motivate. Questo evita che il ramo principale diventi instabile e riduce il costo di correzione degli errori.

La Continuous Integration non deve essere interpretata come un sostituto della qualità progettuale. Essa non decide se una scelta architetturale è opportuna, non valuta la chiarezza del codice e non sostituisce la comprensione del dominio. Tuttavia, fornisce un presidio ripetibile e oggettivo: ogni modifica viene eseguita contro gli stessi controlli, riducendo la dipendenza da verifiche manuali occasionali.

15.12 Ruolo di ChatGPT nella definizione dei casi di test

ChatGPT supporta la strategia di testing nella fase di ragionamento. Può aiutare a individuare edge case, scenari di errore, vincoli di dominio e condizioni limite che potrebbero non emergere immediatamente durante lo sviluppo. Nel Data Layer, ad esempio, può suggerire controlli su timestamp duplicati, valori mancanti o frequenze temporali non coerenti. Nel Feature Engineering Layer può aiutare a ragionare su leakage, allineamento delle sequenze e uso corretto degli scaler.

ChatGPT può inoltre contribuire alla revisione della logica di business. Nel bilancio energetico, può aiutare a esplicitare casi semplici ma critici: consumo maggiore della produzione, produzione maggiore del consumo, valori nulli, surplus, fabbisogno positivo o assenza di domanda residua.

Questi casi diventano scenari utili per verificare che il codice rifletta correttamente il significato operativo del progetto.

Nel supporto alla QA, ChatGPT può essere usato per leggere una Pull Request, confrontare i test proposti con la Issue di riferimento e suggerire controlli mancanti. Il suo ruolo rimane consultivo: aiuta a migliorare la qualità delle domande e delle verifiche, ma non sostituisce l'esecuzione automatica dei test né la responsabilità della review umana.

15.13 Ruolo di Codex nella produzione e manutenzione dei test

Codex ha un ruolo operativo nella produzione e manutenzione dei test automatici. A partire dalle Issue e dalle indicazioni progettuali, può creare test coerenti con i moduli estratti, aggiornare test esistenti quando le interfacce cambiano e proporre casi aggiuntivi quando individua comportamenti non coperti.

Durante il refactoring, Codex dovrà lavorare in modo incrementale. Quando estrae una funzione dal notebook, deve anche predisporre controlli minimi che ne dimostrino il comportamento. Quando modifica un'interfaccia, deve aggiornare i test collegati. Quando introduce un nuovo modulo, deve rendere esplicito come quel modulo verrà verificato nella suite.

Questo non significa delegare a Codex la responsabilità finale della qualità. I test generati devono essere revisionati come qualsiasi altra modifica. Una suite formalmente esistente ma logicamente debole non garantisce affidabilità. Per questo motivo il contributo di Codex deve essere inserito nello stesso ciclo operativo del codice applicativo: Issue, branch, Pull Request, CI, review umana e merge.

15.14 Qualità continua e fondamento dello Strato 4

La strategia descritta introduce il concetto di qualità continua. La qualità non viene controllata una sola volta alla fine del progetto, ma viene costruita e verificata lungo tutto il ciclo di sviluppo. Ogni modulo estratto dal notebook diventa più affidabile perché accompagnato da test; ogni Pull Request diventa più sicura perché controllata automaticamente; ogni merge diventa più governabile perché sostenuto da evidenze tecniche.

Questa impostazione è coerente con lo Stack Operativo Ottimizzato. GitHub fornisce il luogo in cui eseguire e tracciare i controlli; Codex contribuisce alla produzione dei test; ChatGPT supporta la definizione degli scenari e la QA; Prism conserva decisioni, avanzamento e risultati significativi delle review. La qualità non è quindi affidata a un singolo strumento, ma a un processo integrato.

Il testing costituisce il fondamento dello Strato 4 – Validazione e Test. Dopo aver trasformato la progettazione in workflow operativo e dopo aver avviato il refactoring modulare, il progetto deve ora garantire che la piattaforma resti affidabile mentre evolve. I test automatici, la Continuous Integration e la review umana formano la base su cui potranno essere costruite le successive attività di validazione, senza entrare ancora nei temi di monitoraggio del modello, retraining o metriche

avanzate di forecasting che appartengono agli strati successivi.

16 Codex Execution Plan

16.1 Introduzione

Con la definizione della strategia di testing si conclude la fase di progettazione documentale della soluzione. I capitoli precedenti hanno chiarito il contesto di business, i requisiti, l'architettura logica, le decisioni architetturali, il piano di refactoring, la struttura modulare, il workflow di sviluppo e la strategia di verifica. Il progetto entra quindi nella fase di implementazione reale, nella quale il notebook `Previsione Fabbisogno v05_all.ipynb` dovrà essere trasformato progressivamente in una piattaforma software modulare, testabile e coerente con l'architettura progettata.

Questo documento definisce il piano operativo che guiderà Codex durante tale trasformazione. Codex viene utilizzato come agente di sviluppo incaricato di eseguire attività concrete sul repository GitHub: analizzare il codice esistente, creare moduli Python, spostare logica stabile dal notebook alla cartella `src`, aggiornare il notebook ottimizzato, predisporre test automatici e preparare Pull Request sottoposte a review umana.

È importante distinguere tra GitHub Issues e Codex Tasks. Le GitHub Issues descrivono il lavoro da svolgere a livello di backlog: indicano obiettivo, perimetro, layer coinvolto e deliverable attesi. I Codex Tasks traducono invece la Issue in istruzioni operative più dettagliate: quali file analizzare, quali moduli creare, quali funzioni estrarre, quali vincoli rispettare e quali criteri utilizzare per considerare completata l'attività. La Issue definisce il "cosa"; il Codex Task chiarisce il "come" operativo.

Il piano qui descritto costituisce quindi il riferimento ufficiale per le attività successive affidate a Codex. Ogni intervento dovrà essere coerente con questo documento, con la documentazione consolidata fino al capitolo `15 - Testing Strategy` e con il principio generale dello Stack Operativo Ottimizzato: accelerare lo sviluppo tramite AI senza perdere controllo umano, tracciabilità e qualità.

16.2 Materiale fornito a Codex

Codex dovrà operare a partire da un insieme di artefatti già prodotti e documentati. Il primo riferimento è il notebook originale `Previsione Fabbisogno v05_all.ipynb`, che rappresenta la sorgente tecnica reale del progetto. Al suo interno sono presenti la generazione dei dati sintetici, la pulizia e imputazione, lo split temporale, la normalizzazione, la costruzione delle sequenze, la definizione del modello LSTM sequence-to-sequence, l'addestramento, la predizione, la valutazione, la visualizzazione e il calcolo del fabbisogno energetico.

Il secondo riferimento è l'architettura logica consolidata: Data Layer, Feature Engineering Layer, Forecasting Layer, Monitoring Layer, Energy Balance Layer e Visualization Layer. Il repository

determina quali componenti risultano effettivamente implementati; il testing rimane una capacità trasversale pianificata nella Issue #6.

Le ADR costituiscono il terzo riferimento. Le decisioni architetturali documentate indicano le scelte da rispettare durante il refactoring: separazione delle responsabilità, modularizzazione progressiva, mantenimento del notebook come riferimento iniziale, uso del repository GitHub come sorgente ufficiale e introduzione graduale dei test.

La struttura del repository fornisce il quarto riferimento operativo. Essa chiarisce dove collocare notebook grezzi, notebook ottimizzati, codice sorgente, dati, esperimenti, documentazione e test. Codex dovrà rispettare questa organizzazione, evitando di collocare logica stabile in posizioni non previste o di confondere codice applicativo, esperimenti e documentazione.

La sequenza ufficiale mantiene Issue #1 Data, #2 Feature Engineering, #3 Forecasting, #4 Energy Balance, #5 Visualization e #6 suite iniziale di test. Il Monitoring Layer è un layer ufficiale implementato mediante un'attività non numerata, senza alterare retroattivamente il backlog.

Le specifiche tecniche completano il materiale fornito a Codex. Esse descrivono responsabilità, input, output, dipendenze e criteri di qualità dei moduli previsti. Codex dovrà mantenere piena coerenza con tali specifiche e, in caso di ambiguità, dovrà limitare l'intervento al perimetro della Issue, lasciando alla review umana eventuali decisioni di ampliamento.

16.3 Strategia generale di refactoring

La strategia generale di refactoring si basa su un principio conservativo e incrementale: il notebook originale deve rimanere invariato come memoria tecnica del prototipo. Esso conserva il flusso completo della soluzione e permette di confrontare il comportamento della piattaforma modulare con il comportamento iniziale. La trasformazione non deve quindi cancellare il notebook originario, ma affiancargli progressivamente una versione ottimizzata e una struttura Python riutilizzabile.

Il codice stabile verrà spostato nella cartella `src`. Questo spostamento non deve essere una semplice copia meccanica delle celle, ma una riorganizzazione in moduli coerenti con l'architettura. Le funzioni dovranno essere separate per responsabilità, dotate di interfacce comprensibili e rese utilizzabili sia dal notebook ottimizzato sia da eventuali componenti applicativi futuri.

Il notebook ottimizzato assumerà il ruolo di orchestratore. Invece di contenere tutta la logica applicativa, dovrà importare i moduli Python e coordinare il flusso: caricamento dati, preprocessing, generazione sequenze, training o inferenza, bilancio energetico e visualizzazione. In questo modo il notebook rimane utile per analisi, dimostrazione e controllo, ma non è più il luogo principale in cui vive il codice stabile.

La distinzione tra `01_notebooks_raw` e `02_notebooks_optimized` è parte essenziale della strategia. La cartella `01_notebooks_raw` conserva il notebook originale, non modificato, come baseline tecnica. La cartella `02_notebooks_optimized` conterrà invece la versione riorganizzata, capace di richiamare i moduli estratti e di mostrare il flusso aggiornato senza duplicare tutta la logica.

La cartella `07_projects/forecasting_energy/src` rappresenta il nucleo della piattaforma. Data, Feature Engineering, Forecasting e Monitoring risultano implementati. `energy_balance` dispone del contratto definitivo per la successiva implementazione. Per il Visualization Layer, i sorgenti e la suite specifica sono presenti nel materiale fornito; non è verificata l'integrazione dei file nel repository ufficiale né lo stato amministrativo della Issue #5. La Issue #6 resta aperta/pianificata nel documento: i test visuali non provano il completamento della suite iniziale complessiva.

16.4 Piano operativo per Issue #1 – Refactor Data Layer from Notebook

La prima Issue ha l'obiettivo di estrarre dal notebook la logica relativa alla gestione dei dati. Nel prototipo questa responsabilità è distribuita tra le celle che generano l'indice temporale, costruiscono dati meteorologici sintetici, generano consumo e produzione fotovoltaica e preparano i dataset iniziali. Codex dovrà analizzare queste celle e trasformare la logica stabile in un Data Layer riutilizzabile.

Le attività previste comprendono l'identificazione delle funzioni del notebook legate ai dati, la separazione tra caricamento, generazione e validazione, la definizione di interfacce semplici e la predisposizione dei moduli iniziali. Il layer dovrà supportare la fase attuale basata su dati sintetici, ma dovrà essere organizzato in modo da poter accogliere in futuro dati reali senza riscrivere l'intera pipeline.

Codex dovrà creare la seguente struttura:

```
src/data/
|-- loader.py
|-- validator.py
‘-- synthetic_data.py
```

Il modulo `loader.py` contiene la logica di ingresso dei dati, `validator.py` centralizza i controlli e `synthetic_data.py` preserva la generazione sintetica. `weather_loader.py` non è implementato.

Gli output attesi sono moduli importabili, funzioni riutilizzabili, responsabilità chiaramente separate e una prima riduzione della dipendenza dal notebook monolitico. Il criterio di completamento è soddisfatto quando la logica dati può essere richiamata da codice Python esterno al notebook e quando il comportamento rimane coerente con il prototipo originario.

16.5 Piano operativo per Issue #2 – Create Feature Engineering Layer

La seconda Issue riguarda la creazione del Feature Engineering Layer. Nel notebook questa area comprende pulizia, imputazione, gestione dei timestamp, split temporale, normalizzazione, scaling e costruzione delle sequenze necessarie al modello LSTM sequence-to-sequence. Si tratta di una parte critica perché collega i dati grezzi al formato richiesto dal Forecasting Layer.

Codex dovrà separare la logica di preprocessing dalla logica predittiva. Le trasformazioni sui dati non devono rimanere intrecciate con il training del modello, perché questa sovrapposizione rende più difficile testare, sostituire o riutilizzare i componenti. La normalizzazione dovrà essere gestita in modo esplicito, preservando la distinzione tra fit degli scaler, trasformazione dei dataset e trasformazione inversa degli output quando necessaria.

La gestione dei timestamp dovrà mantenere l'ordine cronologico e impedire contaminazioni tra training, validation e test. La generazione delle sequenze dovrà rispettare le finestre temporali previste dal notebook, in particolare la logica basata su `PAST_WINDOW` e `FUTURE_WINDOW`. Codex dovrà prestare particolare attenzione all'allineamento tra feature passate, feature future e target futuri.

La struttura da creare è la seguente:

```
src/features/
|-- preprocessing.py
|-- scaling.py
‘-- sequence_builder.py
```

Il modulo `preprocessing.py` conterrà pulizia, imputazione e preparazione iniziale. Il modulo `scaling.py` gestirà normalizzazione, scaler e trasformazioni inverse. Il modulo `sequence_builder.py` conterrà la costruzione delle finestre temporali. Il criterio di completamento è raggiunto quando i dataset validati dal Data Layer possono essere trasformati in input coerenti per il Forecasting Layer senza logica residua duplicata nel notebook.

16.6 Piano operativo per Issue #3 – Create Forecasting Layer

La terza Issue ha l'obiettivo di creare il Forecasting Layer. Nel notebook questa responsabilità include la definizione del modello LSTM sequence-to-sequence, l'addestramento dei modelli per consumo e produzione, la generazione delle predizioni e la gestione degli artefatti collegati al modello.

Codex dovrà separare la logica LSTM in componenti distinti. La definizione dell'architettura del modello non deve essere confusa con la procedura di training; la predizione non deve dipendere da celle notebook eseguite in precedenza; la gestione dei modelli deve essere organizzata in modo da consentire salvataggio, caricamento e riutilizzo controllato.

Il training dovrà essere incapsulato in funzioni o classi capaci di ricevere input già preparati dal Feature Engineering Layer. La prediction dovrà esporre un'interfaccia chiara, capace di produrre output utilizzabili dal layer di bilancio energetico. La gestione dei modelli dovrà evitare percorsi rigidi o dipendenze implicite dal contesto locale del notebook.

La struttura da creare è la seguente:

```
src/forecasting/
```

```
|-- lstm_model.py
|-- trainer.py
`-- predictor.py
```

Il modulo `lstm_model.py` conterrà la definizione del modello. Il modulo `trainer.py` conterrà la logica di addestramento. Il modulo `predictor.py` conterrà inferenza e utilizzo del modello addestrato. Il criterio di completamento è soddisfatto quando il notebook ottimizzato può richiamare il Forecasting Layer senza mantenere al proprio interno la logica principale di costruzione, training e predizione.

16.7 Attività non numerata – Create Monitoring Layer

Il Monitoring Layer è stato implementato successivamente alla definizione del backlog iniziale attraverso `src/monitoring/metrics.py` e `src/monitoring/evaluation.py`. L'attività non introduce una nuova numerazione retroattiva: documenta l'implementazione conclusa di un layer ufficiale, distinta dalle Issue #1–#6.

16.8 Piano operativo per Issue #4 – Create Energy Balance Layer

La quarta Issue riguarda la creazione dell'Energy Balance Layer. Il layer dovrà tradurre i forecast nei campi canonici `net_balance`, `grid_export` e `grid_import`, mantenendo separati i calcoli per intervallo dai KPI totali e cumulativi.

Codex dovrà individuare nel notebook le parti che confrontano consumo e produzione e trasformano tali valori in risultati energetici. La logica dovrà essere resa esplicita, leggibile e verificabile. Un errore di segno, un'inversione tra consumo e produzione o un disallineamento temporale potrebbe produrre risultati di business errati anche in presenza di un modello tecnicamente funzionante.

La struttura da creare è la seguente:

```
src/energy_balance/
|-- balance_calculator.py
`-- kpi.py
```

Il modulo `balance_calculator.py` dovrà applicare esclusivamente le formule canoniche, senza clipping degli input, classificazione obbligatoria del caso zero o file I/O. `kpi.py` dovrà produrre esclusivamente totali e cumulativi di `grid_import` e `grid_export`. Il criterio di completamento richiede inoltre scala fisica e indice temporale preservati.

16.9 Piano operativo per Issue #5 – Create Visualization Layer

Il materiale sorgente fornito contiene `plotting.py` e la suite specifica dei test per il Visualization Layer. Nel notebook originale la visualizzazione è utilizzata per confrontare valori reali e predetti, rappresentare loss function, distribuzioni degli errori, forecast futuri e risultati energetici. La v14 riportava la Issue #5 aperta, ma stato GitHub attuale, Pull Request, review e merge non sono verificati. Il modulo mantiene la rappresentazione separata dalla logica di calcolo.

L'implementazione delle funzioni è documentata come presente nel materiale sorgente fornito; non è documentato che sia stata revisionata o integrata nel repository. Le verifiche successive devono confermare la corrispondenza tra sorgente ufficiale, specifiche del Capitolo 22 e notebook ottimizzato, senza modificare la responsabilità del layer.

La struttura da creare è la seguente:

```
src/visualization/
‘-- plotting.py
```

Il materiale fornito contiene le sette API pubbliche definite nel Capitolo 22 e la suite corrispondente. Il risultato di 18 test superati in 5,14 secondi con backend Agg e l'esito positivo di `git diff --check` sono informazioni comunicate. Esse costituiscono evidenza tecnica sul materiale, ma il criterio end-to-end resta da verificare rispetto al repository, al notebook ottimizzato e alla review umana; nessuna chiusura di Issue o merge viene dedotta.

16.10 Piano operativo per Issue #6 – Create Initial Test Suite

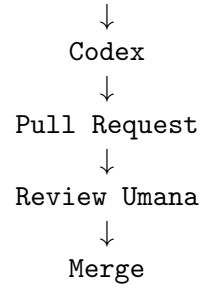
La sesta Issue ha l'obiettivo di creare la suite iniziale di test. Essa deriva direttamente dalla strategia descritta nel capitolo `15 - Testing Strategy` e ha la funzione di accompagnare il refactoring con controlli automatici progressivi. La suite iniziale non deve essere esaustiva, ma deve introdurre una base concreta per unit test, integration test e verifiche di regressione.

Nel documento la Issue #6 rimane aperta/pianificata per la suite iniziale trasversale. La presenza del test specifico del Visualization Layer nel materiale fornito non equivale al completamento degli unit test, integration test e controlli di regressione per tutti i layer. Stato GitHub attuale e integrazione dei test nel repository non sono verificati.

16.11 Workflow Codex

Il workflow operativo previsto per Codex segue la sequenza già definita nei capitoli precedenti:

`Issue`



La Issue descrive il lavoro da svolgere e collega l'attività al backlog GitHub. Codex riceve la Issue e il relativo task operativo, analizza il repository, individua i file coinvolti, crea o modifica i moduli previsti e prepara una proposta di implementazione. La Pull Request rende visibili le modifiche e permette di verificare se il lavoro rispetta obiettivi, perimetro e criteri di completamento.

La review umana rimane obbligatoria. Ogni modifica dovrà essere controllata prima dell'integrazione nel repository principale. Il reviewer dovrà valutare coerenza architetturale, leggibilità, rispetto della Issue, assenza di interventi fuori perimetro e adeguatezza dei test. Quando la Continuous Integration sarà attiva, la Pull Request dovrà superare la suite automatica prima del merge.

Il merge consolida il lavoro nel ramo principale e aggiorna la sorgente ufficiale del codice. Dopo il merge, Prism dovrà essere utilizzato per registrare l'avanzamento, le decisioni rilevanti, eventuali deviazioni motivate e lo stato della roadmap. In questo modo il lavoro di Codex rimane inserito in un processo tracciato, revisionato e documentato.

16.12 Deliverable del piano di esecuzione

La fase di implementazione dovrà produrre un insieme di artefatti coerenti e verificabili. Il primo deliverable è costituito dai moduli Python organizzati nella cartella `07_projects/forecasting_energy/src`. Tali moduli rappresenteranno la base applicativa stabile della futura piattaforma.

Il secondo deliverable è il notebook ottimizzato, collocato nella cartella `02_notebooks_optimized`. Esso dovrà orchestrare i moduli estratti senza duplicare la logica principale, mantenendo leggibile il flusso end-to-end della soluzione.

Il terzo deliverable pianificato è la suite iniziale di test automatici trasversale prevista dalla Issue #6. Il materiale fornito comprende già la suite specifica del Visualization Layer; non vengono attribuiti al repository ufficiale file o coperture complessive ulteriori non verificati.

Il quarto deliverable è costituito dalle Pull Request prodotte da Codex. Ogni Pull Request dovrà essere collegata a una Issue, contenere modifiche circoscritte, includere eventuali test pertinenti e rendere comprensibile il cambiamento proposto.

Il quinto deliverable è la documentazione aggiornata. Prism dovrà conservare lo stato delle attività, le decisioni emerse durante la review, le modifiche significative e l'avanzamento della roadmap. La documentazione dovrà rimanere allineata al repository, evitando divergenze tra progetto descritto

e progetto implementato.

Il sesto deliverable è il repository strutturato. Il progetto dispone di una base organizzata in notebook raw, notebook ottimizzati, sorgenti Python e documentazione; il materiale fornito documenta test automatici specifici per il Visualization Layer, mentre la suite iniziale trasversale resta affidata alla Issue #6. Questo asset costituisce il punto di partenza per le successive attività di validazione, consolidamento e ulteriore evoluzione della piattaforma.

Questo Codex Execution Plan diventa quindi il riferimento ufficiale per tutte le attività operative successive. Ogni task assegnato a Codex dovrà essere formulato, eseguito e revisionato alla luce di questo piano, mantenendo continuità tra notebook originario, architettura progettata, workflow GitHub, strategia di testing e memoria documentale del progetto.

17 Data Layer Progress

17.1 Obiettivo del Data Layer

Il Data Layer rappresenta il primo layer architetturale completato nella fase di implementazione reale dello Strato 3 – Build Reale. La sua realizzazione conclude la Issue #1 – Refactor Data Layer from notebook e segna il passaggio concreto dal notebook monolitico `Previsione Fabbisogno v05_all.ipynb` a una struttura software modulare, coerente con lo Stack Operativo Ottimizzato.

L'obiettivo della rifattorizzazione è stato estrarre dal notebook tutta la logica relativa alla gestione dei dati, separandola dalle fasi successive di feature engineering, forecasting, calcolo del bilancio energetico e visualizzazione. Nel notebook originario, generazione dei dati sintetici, validazione, pulizia, caricamento e preparazione preliminare erano parte di un unico flusso sequenziale. Questa impostazione era adatta alla sperimentazione, ma non sufficiente per una piattaforma software manutenibile.

Il refactoring ha quindi perseguito quattro obiettivi principali. Il primo è eliminare la dipendenza da codice monolitico, rendendo esplicite le responsabilità che nel notebook erano distribuite tra celle successive. Il secondo è realizzare componenti Python riutilizzabili, testabili e facilmente manutenibili. Il terzo è creare un Data Layer indipendente dai layer superiori, in modo che la gestione dei dati non dipenda da scelte di modello, training o visualizzazione. Il quarto è mantenere invariato il comportamento funzionale del notebook originale, preservando formule, logiche di simulazione e assunzioni tecniche già validate nella fase prototipale.

In questa prospettiva, il Data Layer non introduce una nuova logica di dominio, ma riorganizza quella esistente in componenti più chiari. Il notebook continua a rappresentare il riferimento funzionale del progetto, mentre il codice stabile viene progressivamente trasferito nella struttura modulare del repository.

17.2 Architettura del Data Layer

All'interno dello Stack Operativo Ottimizzato, il Data Layer ha il compito di fornire dati affidabili, coerenti e pronti per essere utilizzati dai layer successivi. Esso costituisce il punto di ingresso della piattaforma e separa le responsabilità di acquisizione, generazione, validazione e pulizia dalle attività di trasformazione delle feature e di modellazione predittiva.

La struttura definitiva realizzata è la seguente:

```
src/
‘-- data/
    |-- validator.py
    |-- loader.py
    ‘-- synthetic_data.py
```

Il modulo `validator.py` centralizza le verifiche di qualità sui dataset temporali. Il suo ruolo è assicurare che i dati rispettino le condizioni minime richieste dal sistema: indice temporale valido, colonne obbligatorie presenti, tipi numerici corretti, assenza di dataframe vuoti, allineamento temporale e gestione dei valori mancanti.

Il modulo `loader.py` gestisce il caricamento dei dataset storici da file CSV e l'accesso ordinato alla cartella dati. Esso rappresenta il punto di ingresso per dati persistiti, come storico dei consumi e storico della produzione fotovoltaica. Il modulo non contiene logica di business né logica di forecasting: il suo compito è caricare dati e sottoporli a una validazione preliminare.

Il modulo `synthetic_data.py` conserva e riorganizza la logica di generazione sintetica presente nel notebook. Esso permette di produrre date range, dati meteorologici, consumi e produzione fotovoltaica simulati, mantenendo integralmente le formule matematiche e le assunzioni del prototipo.

Questa separazione consente al Data Layer di operare come componente autonomo. I layer successivi ricevono dati già caricati, generati o validati, senza dover conoscere i dettagli delle sorgenti informative o delle funzioni di simulazione.

17.3 Modulo `validator.py`

Il modulo `validator.py` rappresenta il presidio centrale di qualità del Data Layer. Esso raccoglie le funzioni di validazione e pulizia dei dataframe temporali utilizzati dal sistema di forecasting energetico. La sua introduzione consente di evitare controlli duplicati o dispersi nel notebook, rendendo il comportamento più leggibile, verificabile e riutilizzabile.

Le funzionalità implementate coprono le principali condizioni che un dataset deve rispettare prima di essere utilizzato nel flusso applicativo. La validazione dei DataFrame permette di intercettare strutture non idonee o dataset privi di contenuto. Il controllo del `DatetimeIndex` garantisce che

le serie siano effettivamente indicizzate temporalmente, condizione essenziale per split cronologici, allineamento, costruzione delle sequenze e forecasting.

La verifica delle colonne obbligatorie assicura che i dataset contengano le variabili richieste dal processo. Questo controllo è importante perché un dataset incompleto potrebbe generare errori tardivi nelle fasi di feature engineering o training. La validazione dei tipi numerici verifica che le colonne destinate a calcoli, scaling o modelli siano effettivamente utilizzabili come grandezze numeriche.

Il controllo dell'allineamento temporale verifica la compatibilità tra dataset diversi. Nel progetto, consumo, produzione e variabili meteorologiche devono poter essere confrontati lungo lo stesso asse temporale. Un disallineamento degli indici potrebbe produrre feature non coerenti o bilanci energetici errati.

Il modulo include inoltre la gestione dell'imputazione dei valori mancanti. Questa responsabilità era già presente nel notebook, ma viene ora resa centralizzata e richiamabile da più componenti. La pipeline completa di pulizia dei dati combina le verifiche principali e le operazioni di preparazione preliminare, offrendo un punto di ingresso ordinato per trattare dataframe temporali prima del passaggio ai layer successivi.

Le funzioni principali realizzate includono:

- `validate_datetime_index()`;
- `validate_required_columns()`;
- `validate_no_empty_dataframe()`;
- `validate_aligned_index()`;
- `validate_numeric_columns()`;
- `impute_missing_values()`;
- `clean_time_series_dataframe()`.

Il modulo dipende esclusivamente da `Pandas`, mantenendo una superficie tecnica ridotta e coerente con il suo ruolo architetturale.

17.4 Modulo `loader.py`

Il modulo `loader.py` introduce la responsabilità di caricamento dei dati storici. Nel notebook originario, la costruzione dei dataset avveniva direttamente nel flusso esecutivo; nella piattaforma modulare, questa responsabilità viene isolata in un componente dedicato, così da rendere più chiaro il confine tra acquisizione dei dati e trasformazioni successive.

Il modulo gestisce la cartella dati e fornisce funzioni per il caricamento di file CSV storici. Questa scelta consente al progetto di distinguere tra dati generati sinteticamente e dati persistiti su file, preparando l'evoluzione futura verso sorgenti informative più realistiche o operative.

Tra le responsabilità principali rientrano il caricamento dello storico dei consumi e il caricamento dello storico della produzione fotovoltaica. Questi dataset rappresentano due grandezze fondamentali per il progetto, perché il valore operativo della piattaforma nasce dal confronto tra fabbisogno previsto e produzione locale prevista.

Il caricamento non si limita alla lettura fisica del file. Il modulo esegue anche una validazione preliminare dei dataset, richiamando le responsabilità centralizzate nel Data Layer. In questo modo eventuali problemi di struttura, formato o contenuto possono essere intercettati subito, prima che i dati vengano consegnati al Feature Engineering Layer.

È importante sottolineare che `loader.py` non contiene alcuna logica di business né alcuna logica di forecasting. Non calcola fabbisogni, non costruisce sequenze, non applica modelli e non produce visualizzazioni. Il suo compito è esclusivamente fornire dati storici caricati e preliminarmente validati, mantenendo il Data Layer indipendente dai livelli superiori dell'architettura.

17.5 Modulo `synthetic_data.py`

Il modulo `synthetic_data.py` raccoglie la logica di generazione sintetica dei dati originariamente presente nel notebook. Questa componente è rilevante perché il prototipo del progetto si basa su un flusso controllato di simulazione, utile per sviluppare, verificare e dimostrare il comportamento della piattaforma anche in assenza di dataset reali completi.

Le funzioni implementate sono:

- `generate_date_range()`;
- `generate_weather_data()`;
- `generate_consumption_data()`;
- `generate_production_data()`.

Esse corrispondono alle responsabilità già presenti nel notebook: costruzione dell'indice temporale, generazione delle variabili meteorologiche, simulazione dei consumi e simulazione della produzione fotovoltaica.

La funzione `generate_date_range()` produce l'asse temporale della simulazione, coerente con la granularità oraria utilizzata dal progetto. La funzione `generate_weather_data()` genera variabili meteorologiche sintetiche, mantenendo il modello utilizzato nel notebook per rappresentare temperatura, irraggiamento e variabilità climatica. La funzione `generate_consumption_data()` produce consumi coerenti con pattern giornalieri e con l'effetto della temperatura. La funzione

`generate_production_data()` simula la produzione fotovoltaica sulla base dell'irraggiamento e dei parametri fisici del sistema.

Durante il refactoring sono state mantenute integralmente le formule matematiche, il modello meteorologico, il modello dei consumi, il modello della produzione fotovoltaica e il rumore gaussiano utilizzato nella simulazione. Questa scelta è essenziale per preservare il comportamento del notebook originale e garantire che la modularizzazione non modifichi il significato tecnico del prototipo.

Il modulo dipende esclusivamente da `NumPy` e `Pandas`. Questa scelta mantiene la generazione sintetica leggera, autonoma e coerente con il ruolo del Data Layer. Non sono state introdotte dipendenze verso modelli predittivi, librerie di deep learning, visualizzazione o logiche di business.

17.6 Benefici architetturali

Il completamento del Data Layer produce benefici architetturali rilevanti. Il primo beneficio è la separazione delle responsabilità. Validazione, caricamento e generazione sintetica non sono più distribuiti nel notebook, ma organizzati in moduli dedicati, ciascuno con un perimetro chiaro.

Il secondo beneficio è la maggiore modularità. I tre moduli possono evolvere in modo indipendente: `validator.py` può essere esteso con nuovi controlli, `loader.py` può supportare nuove sorgenti dati e `synthetic_data.py` può essere utilizzato per scenari di simulazione, demo o test senza interferire con gli altri layer.

Il terzo beneficio è la riutilizzabilità del codice. Le funzioni del Data Layer possono essere richiamate dal notebook ottimizzato, dai test automatici, da script di sviluppo o da futuri componenti applicativi. Questo riduce duplicazioni e rende più semplice mantenere un comportamento coerente nel tempo.

Il quarto beneficio è la migliore testabilità. Ogni funzione può essere verificata in modo isolato, senza dover eseguire l'intero notebook. Questo aspetto è particolarmente importante in vista della Continuous Integration e della progressiva estensione della suite di test.

Il quinto beneficio è la maggiore manutenibilità. La logica sui dati è ora localizzata in una porzione chiara del repository, rendendo più semplice individuare errori, aggiornare comportamenti e discutere modifiche tramite Pull Request circoscritte.

Infine, il Data Layer è indipendente da Feature Engineering, Forecasting, Monitoring, Energy Balance e Visualization Layer. Questa indipendenza riduce il rischio che cambiamenti nei componenti superiori impattino la gestione dei dati.

17.7 Stato della Issue #1

La Issue #1 – Refactor Data Layer from notebook è stata completata con successo. Il lavoro ha portato alla realizzazione dei tre moduli previsti per il Data Layer: `validator.py`, `loader.py` e

`synthetic_data.py`. Insieme, questi componenti coprono validazione, pulizia, caricamento dei dati storici e generazione sintetica dei dataset utilizzati dal progetto.

Il notebook originale continua a rappresentare il riferimento funzionale del progetto. Il refactoring non ne altera il comportamento, ma ne estrae la logica relativa alla gestione dei dati in componenti Python riutilizzabili. In questo modo il prototipo rimane la baseline tecnica, mentre il repository acquisisce una struttura software più robusta.

Tutta la logica relativa alla gestione dei dati è stata rifattorizzata nei tre moduli del Data Layer. La piattaforma dispone ora del primo layer architetturale completamente implementato, coerente con il piano operativo definito per Codex e con la struttura progettata nello Stack Operativo Ottimizzato.

Il completamento del Data Layer abilita la fase successiva del refactoring. Il progetto è ora pronto per iniziare la realizzazione del Feature Layer, che avrà il compito di trasformare i dataset validati e caricati in input adatti alla normalizzazione, alla costruzione delle sequenze temporali e al successivo utilizzo da parte del Forecasting Layer.

18 Feature Engineering Layer

18.1 Obiettivo del Feature Engineering Layer

Il Feature Engineering Layer rappresenta il collegamento architetturale tra il Data Layer e il Forecasting Layer. Dopo il completamento del Data Layer, il progetto dispone di dataset validati, coerenti e puliti, generati o caricati attraverso componenti dedicati e controllati da una logica centralizzata di validazione. Il passo successivo consiste nel trasformare tali dataset in una rappresentazione numerica adatta all'addestramento dei modelli di Machine Learning.

Il ruolo del Feature Engineering Layer non è modificare il significato dei dati, né introdurre logiche di business o di forecasting. La sua responsabilità è prepararli per il training, preservando la coerenza temporale e rendendo esplicite le trasformazioni necessarie al modello. In questo senso, il layer agisce come uno strato di traduzione: riceve dati puliti e strutturati dal Data Layer e produce input numerici pronti per il Forecasting Layer.

Nel notebook `Previsione Fabbisogno v05_all.ipynb`, questa responsabilità è distribuita tra celle dedicate alla preparazione dei dataframe, alla selezione delle feature, alla costruzione dei target, alla normalizzazione e alla creazione delle sequenze temporali per il modello LSTM sequence-to-sequence. La progettazione del Feature Engineering Layer rende tali responsabilità esplicite e le organizza in moduli separati.

L'obiettivo architetturale è mantenere il comportamento funzionale del notebook originale, ma ridurre la dipendenza da un flusso monolitico. Le trasformazioni devono diventare componenti riutilizzabili, testabili e indipendenti dal modello specifico che verrà addestrato. In questo modo, la piattaforma potrà evolvere verso modelli diversi o strategie previsionali alternative senza dover riscrivere l'intera pipeline di preparazione dei dati.

18.2 Pipeline del Feature Engineering

La pipeline logica del Feature Engineering Layer è organizzata secondo una sequenza lineare e controllata:

```
Data Layer
        |
        v
preprocessing.py
        |
        v
scaling.py
        |
        v
sequence_builder.py
        |
        v
Forecasting Layer
```

Il Data Layer fornisce dataset già validati, puliti e coerenti dal punto di vista temporale. Questi dataset rappresentano l'input della pipeline di feature engineering e costituiscono la base informativa su cui vengono costruiti feature set e target previsionali.

Il modulo `preprocessing.py` prepara i dataframe per il training. In questa fase vengono organizzate le variabili, costruiti i dataset necessari per consumo e produzione, definiti i target e applicata la suddivisione temporale in training, validation e test. Il modulo non esegue normalizzazione, perché tale responsabilità appartiene esclusivamente al modulo successivo.

Il modulo `scaling.py` gestisce la normalizzazione dei dati. La separazione di questa responsabilità è importante perché lo scaling deve rispettare regole precise per evitare data leakage. Gli scaler vengono adattati sul training set e poi applicati in modo coerente a validation e test.

Il modulo `sequence_builder.py` costruisce le sequenze temporali richieste dal modello LSTM. Esso trasforma dataset tabellari già preparati e normalizzati in tensori organizzati secondo finestre passate e future, producendo le variabili di input e target che verranno consegnate al Forecasting Layer.

Questa pipeline rende esplicito il principio di separazione delle responsabilità. Ogni modulo ha un compito definito, riceve input coerenti dal passaggio precedente e produce output destinati al passaggio successivo. La linearità del flusso riduce ambiguità, facilita la review e rende più semplice introdurre test automatici mirati.

18.3 Modulo `preprocessing.py`

Il modulo `preprocessing.py` ha la responsabilità di preparare i dataset per il training. Esso opera dopo il Data Layer e prima della normalizzazione, ricevendo dataframe già validati e puliti. Il suo compito è trasformare tali dataframe in strutture coerenti con le esigenze del modello, senza alterare il significato originario dei dati.

La prima responsabilità riguarda la preparazione dei DataFrame. I dati devono essere organizzati in modo da distinguere variabili energetiche, variabili meteorologiche, indici temporali e colonne destinate alla previsione. Questa fase rende esplicita la struttura informativa che nel notebook era gestita in modo sequenziale.

La seconda responsabilità riguarda la costruzione delle feature. Il modulo deve definire quali variabili entrano nel processo predittivo, mantenendo coerenza con il flusso tecnico del notebook. Le feature possono includere grandezze meteorologiche, informazioni temporali e variabili energetiche necessarie alla previsione.

La terza responsabilità riguarda la costruzione dei dataset di consumo e produzione. Il progetto prevede due fenomeni previsionali distinti: il consumo energetico e la produzione fotovoltaica. Il preprocessing deve quindi predisporre strutture separate o logicamente distinguibili per supportare entrambi i percorsi previsionali, evitando sovrapposizioni improprie.

La quarta responsabilità riguarda la preparazione dei target. I target rappresentano le grandezze che il modello dovrà prevedere. Essi devono essere costruiti preservando l'allineamento temporale tra dati storici, variabili future disponibili e orizzonte di previsione.

La quinta responsabilità riguarda la suddivisione temporale in training, validation e test. Poiché il progetto lavora su serie temporali, questa suddivisione non può essere casuale. Deve preservare l'ordine cronologico e impedire che informazioni future contaminino le fasi di addestramento.

È importante sottolineare che `preprocessing.py` non esegue alcuna normalizzazione dei dati. Questa scelta mantiene il confine architetturale chiaro: il preprocessing organizza e prepara i dataset; lo scaling applica esclusivamente le trasformazioni numeriche di normalizzazione.

18.4 Modulo `scaling.py`

Il modulo `scaling.py` gestisce esclusivamente la normalizzazione dei dati. La sua separazione in un componente dedicato risponde a una necessità tecnica e metodologica: controllare in modo rigoroso come vengono stimati e applicati i parametri di scaling.

Il fit degli scaler deve essere eseguito esclusivamente sul training set. Questa regola è fondamentale perché validation e test rappresentano dati non ancora osservati dal modello durante l'addestramento. Utilizzare informazioni provenienti da validation o test per calcolare i parametri di normalizzazione introdurrebbe una contaminazione del processo valutativo.

I dataset di validation e test devono quindi utilizzare gli stessi parametri stimati sul training set. In questo modo, il comportamento della pipeline rimane coerente con uno scenario reale, nel quale il modello dispone soltanto delle informazioni apprese dai dati di addestramento.

Il modulo deve gestire separatamente feature scaler e target scaler. Le feature e i target hanno ruoli diversi nel processo predittivo e possono richiedere trasformazioni distinte. Separare questi scaler rende più chiaro il flusso, facilita la trasformazione inversa dei risultati e riduce il rischio di errori durante forecasting e valutazione.

Il modulo deve inoltre prevedere operazioni di inverse transform. Durante forecasting e valutazione, le previsioni generate su scala normalizzata devono poter essere riportate alla scala originale, così da risultare interpretabili in termini energetici. Questa responsabilità è particolarmente importante perché il valore operativo del progetto dipende dalla leggibilità di consumi, produzione e fabbisogno.

La progettazione di `scaling.py` evita esplicitamente fenomeni di data leakage. Centralizzare la normalizzazione e vincolare il fit al solo training set consente di mantenere separati addestramento, validazione e test, rafforzando la qualità metodologica della piattaforma.

18.5 Modulo `sequence_builder.py`

Il modulo `sequence_builder.py` costruisce le sequenze temporali necessarie al modello LSTM. Esso rappresenta l'ultimo passaggio del Feature Engineering Layer prima del Forecasting Layer e trasforma i dataset preparati e normalizzati in tensori utilizzabili dal modello.

La responsabilità principale riguarda la costruzione delle finestre temporali. Nel notebook originale, il modello utilizza finestre passate per fornire contesto storico e finestre future per produrre previsioni sull'orizzonte operativo definito. Il modulo deve rendere questa logica esplicita e riutilizzabile.

La gestione delle finestre passate e future è un elemento centrale. Le finestre passate descrivono il contesto storico disponibile al modello; le finestre future rappresentano l'orizzonte di previsione. La corretta relazione tra queste due dimensioni è essenziale per mantenere l'allineamento tra input e target.

Il modulo prepara i tensori di input necessari al modello LSTM sequence-to-sequence. Questa trasformazione converte dataframe tabellari in strutture multidimensionali coerenti con le aspettative del Forecasting Layer. La preparazione dei tensori deve preservare ordine temporale, numero di feature e dimensione delle finestre.

Il modulo costruisce inoltre le variabili `X` e `y`. Le variabili `X` rappresentano gli input del modello, mentre `y` rappresenta il target da prevedere. Questa distinzione deve rimanere chiara sia per il percorso del consumo sia per quello della produzione fotovoltaica.

Essendo l'ultimo passaggio del Feature Engineering Layer, `sequence_builder.py` definisce il contratto di ingresso del Forecasting Layer. Una volta costruite le sequenze, il layer predittivo non deve più occuparsi di preparazione dei dati, scaling o gestione dei dataframe, ma soltanto di modellazione, training e inferenza.

18.6 Dipendenze architetturali

Il Feature Engineering Layer utilizza esclusivamente i servizi messi a disposizione dal Data Layer. In particolare, si appoggia ai componenti `validator.py`, `loader.py` e `synthetic_data.py` per ricevere dati validati, caricati o generati secondo le regole definite nel layer precedente.

Nessuna funzionalità del Data Layer viene duplicata. Il Feature Engineering Layer non deve ripetere la logica di caricamento dei file, non deve rigenerare dati sintetici e non deve ridefinire le regole generali di validazione dei dataframe temporali. Queste responsabilità restano nel Data Layer.

La separazione delle responsabilità permette di mantenere un confine architetturale netto. Il Data Layer risponde alla domanda: i dati sono disponibili, coerenti e puliti? Il Feature Engineering Layer risponde alla domanda: i dati sono rappresentati nella forma numerica corretta per il training? Il Forecasting Layer risponderà alla domanda successiva: il modello può apprendere e produrre previsioni da tali rappresentazioni?

Questa indipendenza riduce accoppiamento e complessità. Se in futuro cambierà la sorgente dati, sarà possibile intervenire sul Data Layer senza riscrivere la logica di feature engineering. Se cambierà il modello predittivo, sarà possibile modificare il Forecasting Layer senza duplicare la preparazione dei dati.

18.7 Miglioramenti architetturali

La progettazione del Feature Engineering Layer introduce benefici rilevanti per l'evoluzione della piattaforma. Il primo beneficio è la separazione delle responsabilità. Preparazione dei dataset, normalizzazione e costruzione delle sequenze diventano fasi distinte, ciascuna associata a un modulo dedicato.

Il secondo beneficio è la maggiore modularità. Ogni modulo può essere progettato, revisionato e testato separatamente. Questo rende più semplice individuare errori, discutere modifiche tramite Pull Request e mantenere coerenza con il piano operativo definito per Codex.

Il terzo beneficio è la migliore riusabilità del codice. La pipeline potrà essere richiamata dal notebook ottimizzato, da test automatici, da script sperimentali o da futuri componenti applicativi senza duplicare logiche già definite.

Il quarto beneficio è la possibilità di testare ogni componente separatamente. `preprocessing.py` potrà essere verificato su dataset controllati, `scaling.py` potrà essere testato rispetto al rischio di data leakage e `sequence_builder.py` potrà essere validato sulle forme attese dei tensori.

Il quinto beneficio è l'indipendenza tra Data Layer, Feature Layer e Forecasting Layer. Ogni layer mantiene un ruolo chiaro e comunica con gli altri attraverso output definiti. Questa indipendenza rende più robusta l'architettura e riduce il rischio che modifiche locali generino effetti collaterali sull'intero sistema.

Infine, la nuova architettura facilita l'estensione del progetto con nuovi modelli di Machine Learning. Se in futuro si vorranno sperimentare modelli diversi dall'LSTM, la pipeline di preparazione dei dati potrà rimanere stabile o essere adattata in modo circoscritto, senza ripartire dal notebook monolitico. Questo rende il Feature Engineering Layer un elemento fondamentale per la scalabilità tecnica della piattaforma.

Il Feature Engineering Layer è stato completamente implementato attraverso i tre moduli `preprocessing.py`, `scaling.py` e `sequence_builder.py`, sviluppati secondo il workflow operativo definito nel progetto. Tutti i componenti sono stati sottoposti a revisione tecnica, integrati nel repository GitHub mediante Pull Request e successivo merge sul branch `main`, completando così la Issue #2 del piano di sviluppo. Il layer costituisce ora il collegamento stabile tra il Data Layer e il futuro Forecasting Layer, che rappresenterà la successiva fase evolutiva dell'architettura. La struttura modulare adottata rende il processo di preparazione dei dati indipendente dal modello predittivo, favorendo riusabilità, testabilità ed evoluzione futura della piattaforma.

19 Forecasting Layer

19.1 Obiettivo e perimetro del Forecasting Layer

Il Forecasting Layer costituisce il nucleo predittivo della piattaforma di forecasting energetico. Esso riceve dal Feature Engineering Layer sequenze temporali già preparate, normalizzate e organizzate nella forma richiesta dai modelli di Machine Learning, e restituisce previsioni quantitative relative al consumo energetico e alla produzione fotovoltaica. Il suo compito è quindi trasformare rappresentazioni numeriche coerenti in stime future, mantenendo separate la logica di modellazione dalle attività di caricamento dei dati, preprocessing, scaling, valutazione delle prestazioni, calcolo del bilancio energetico e visualizzazione.

L'analisi del notebook `Previsione Fabbisogno v05_all.ipynb` evidenzia che il processo predittivo è già articolato in responsabilità riconoscibili, ma tali responsabilità sono distribuite tra celle successive e dipendono dallo stato di esecuzione del notebook. La costruzione dei modelli, la compilazione, l'addestramento, la generazione delle predizioni, il calcolo delle metriche e il salvataggio degli artefatti sono eseguiti in modo sequenziale nello stesso ambiente. La progettazione modulare separa le responsabilità proprie del Forecasting Layer dalle attività di valutazione quantitativa, assegnate al successivo Monitoring Layer.

Il layer deve preservare il comportamento funzionale del prototipo originale. Non introduce quindi un nuovo approccio predittivo, ma ricostruisce in forma modulare l'architettura effettivamente utilizzata: due modelli LSTM sequence-to-sequence distinti, uno dedicato alla previsione dei consumi e uno dedicato alla previsione della produzione fotovoltaica, addestrati attraverso un processo comune e alimentati da sequenze specifiche prodotte dal Feature Engineering Layer.

19.2 Architettura predittiva ricostruita dal notebook

Nel notebook sono presenti due istanze indipendenti della stessa famiglia di modello. Il primo modello apprende la dinamica del consumo energetico; il secondo apprende la dinamica della produzione fotovoltaica. La separazione è coerente con la natura fisica e statistica dei due fenomeni: il consumo dipende dai profili di utilizzo e dalle condizioni ambientali, mentre la produzione fotovoltaica è fortemente condizionata dall'irraggiamento e dagli altri fattori meteorologici.

Entrambi i modelli utilizzano un'architettura encoder–decoder basata su LSTM. L'encoder riceve una finestra storica e comprime l'informazione temporale nei propri stati interni. Il decoder riceve le variabili future disponibili per l'orizzonte di previsione e utilizza gli stati dell'encoder come condizione iniziale. L'uscita del decoder viene trasformata, per ciascun passo temporale, in un singolo valore previsto attraverso un livello denso applicato lungo l'intera sequenza.

L'architettura comprende inoltre componenti di dropout applicati agli stati prodotti dall'encoder e alle uscite del decoder. Questa scelta riduce il rischio di sovradattamento e rende il comportamento del modello più robusto rispetto ai dati di addestramento. La compilazione utilizza l'ottimizzatore Adam e la funzione di perdita Mean Squared Error, coerentemente con la natura continua dei target previsti.

Nel flusso analizzato, le finestre storiche e future sono configurate su un orizzonte di 168 passi temporali, corrispondente a sette giorni di osservazioni orarie. Il modello mantiene tuttavia una struttura capace di ricevere sequenze temporali coerenti con le dimensioni definite dal Feature Engineering Layer. Questo consente di mantenere la configurazione delle finestre esterna alla definizione interna della rete neurale.

19.3 Modelli di consumo e produzione

Il modello di consumo riceve una sequenza encoder contenente le variabili storiche predisposte per il fenomeno di consumo e una sequenza decoder contenente le variabili future note, in particolare le informazioni meteorologiche disponibili sull'orizzonte previsionale. Il target è costituito dal consumo energetico futuro normalizzato. Al termine dell'inferenza, le previsioni vengono riportate alla scala fisica originale attraverso il servizio di inverse transform appartenente al Feature Engineering Layer.

Il modello di produzione utilizza lo stesso schema architetturale, ma opera su un insieme di sequenze specificamente costruito per la produzione fotovoltaica. La sequenza encoder integra la storia della produzione con le feature pertinenti, mentre il decoder utilizza le variabili future disponibili. Il target è la produzione fotovoltaica futura normalizzata, successivamente ricondotta alla scala originale.

La parte comune comprende la struttura encoder–decoder, la strategia di compilazione, i parametri generali di training, la gestione delle callback, il salvataggio del modello, la registrazione della history e la generazione delle predizioni necessarie alla successiva valutazione. Le responsabilità specifiche riguardano invece i tensori di ingresso, il target, gli scaler associati, la denominazione degli artefatti e l'identità del fenomeno previsto. L'architettura modulare deve quindi riutilizzare

il medesimo processo senza confondere i due percorsi previsionali.

19.4 Sequenza di training

Il processo di training ricostruito dal notebook segue una sequenza deterministica:

```
Sequenze dal Feature Engineering Layer
                  |
                  v
Validazione di forme e dimensioni
                  |
                  v
Costruzione e compilazione del modello
                  |
                  v
Configurazione delle callback
                  |
                  v
Training con dati train e validation
                  |
                  v
Predizione su train, validation e test
                  |
                  v
Consegna di valori reali e predetti al Monitoring Layer
                  |
                  v
Salvataggio del modello e della history
```

La prima fase verifica la coerenza dimensionale delle sequenze encoder, decoder e target. Questa validazione è necessaria perché il modello deve ricevere tensori tridimensionali allineati per numero di campioni e orizzonte temporale. Il Forecasting Layer non costruisce tali sequenze, ma deve rifiutare input incompatibili attraverso controlli di contratto chiari.

La seconda fase costruisce e compila una nuova istanza del modello per il fenomeno considerato. Il numero delle feature encoder e decoder viene ricavato dagli input predisposti dal Feature Engineering Layer, mentre la dimensione latente, il dropout e le altre opzioni di modellazione appartengono alla configurazione del Forecasting Layer.

La terza fase configura la callback di Early Stopping. Nel notebook il monitoraggio avviene sulla validation loss, con una pazienza pari a cinque epoche e il ripristino dei migliori pesi osservati. Il training viene avviato con un massimo di trenta epoche e batch di trentadue campioni. Questi valori rappresentano la configurazione iniziale ricostruita dal prototipo e dovranno essere resi espliciti, anziché rimanere incorporati nelle celle.

Dopo l'addestramento, il modello genera predizioni sui dataset di training, validation e test. Le predizioni vengono rese disponibili insieme ai target affinché il Monitoring Layer possa eseguire la valutazione quantitativa in scala originale. Il Forecasting Layer produce quindi gli artefatti necessari al controllo della qualità, ma non definisce né calcola le metriche di monitoraggio.

Il processo si conclude con il salvataggio del modello addestrato e della history di training. Nel notebook gli artefatti sono identificati mediante timestamp e distinti tra consumo e produzione. La modularizzazione deve preservare questa tracciabilità, rendendo però configurabili destinazione, convenzione di denominazione e metadati associati.

19.5 Struttura modulare prevista

In coerenza con l'architettura già definita nei capitoli precedenti, il Forecasting Layer sarà organizzato nella seguente struttura:

```
src/
‘-- forecasting/
    |-- lstm_model.py
    |-- trainer.py
    ‘-- predictor.py
```

I tre moduli separano rispettivamente la definizione del modello, il ciclo di addestramento e la gestione degli artefatti, e l'utilizzo operativo dei modelli addestrati. La suddivisione evita che una singola unità software debba conoscere contemporaneamente la preparazione delle feature, la struttura della rete neurale, la persistenza degli artefatti, la valutazione delle prestazioni, la generazione delle previsioni e la successiva interpretazione energetica.

19.6 Modulo `lstm_model.py`

Il modulo `lstm_model.py` ha la responsabilità esclusiva di definire e compilare l'architettura LSTM sequence-to-sequence. Esso rappresenta la traduzione modulare della funzione di costruzione del modello presente nel notebook e deve poter essere utilizzato sia per il consumo sia per la produzione, senza contenere condizioni specifiche legate al significato del target.

L'API pubblica prevista comprende un'operazione di costruzione del modello, identificabile come `build_seq2seq_model`, alla quale vengono forniti il numero di feature dell'encoder, il numero di feature del decoder e la configurazione architetturale. La compilazione può essere inclusa nella medesima operazione oppure esposta attraverso una responsabilità esplicita, purché il contratto renda visibili ottimizzatore, funzione di perdita e parametri configurabili.

Le funzioni interne devono occuparsi della creazione degli input encoder e decoder, della costruzione delle rispettive LSTM, del trasferimento degli stati tra i due blocchi, dell'applicazione del dropout e

della costruzione della testa di uscita temporale. Devono inoltre validare i parametri architetturali prima di restituire il modello.

Gli input del modulo sono esclusivamente metadati strutturali e iperparametri: dimensioni delle feature, dimensione latente, tasso di dropout e configurazione di compilazione. L'output è un modello Keras compilato e pronto per il training. Il modulo dipende da TensorFlow/Keras, ma non deve dipendere da Pandas, dagli scaler, dai dataset del Data Layer o dalla logica di bilancio energetico.

Rispetto al notebook monolitico, questo modulo consente di creare modelli in modo ripetibile, elimina la duplicazione tra consumo e produzione e rende testabile la correttezza della struttura senza avviare l'intera pipeline.

19.7 Modulo `trainer.py`

Il modulo `trainer.py` governa il ciclo di addestramento e la produzione degli artefatti di training. Riceve un modello già costruito e sequenze già preparate, applica la configurazione di training e restituisce risultati strutturati che possano essere revisionati, salvati e utilizzati dai componenti successivi.

L'API pubblica prevista comprende un'operazione generale di training, identificabile come `train_model`, utilizzabile per entrambi i fenomeni mediante una configurazione esplicita, e un servizio di persistenza degli artefatti di training. L'API deve distinguere chiaramente il modello di consumo dal modello di produzione attraverso un identificatore di target, senza replicare due implementazioni quasi identiche.

Le funzioni interne gestiscono la configurazione dell'Early Stopping, l'esecuzione del metodo di fit e la raccolta della history. Il modulo deve inoltre controllare la presenza e la compatibilità di training e validation, gestire le convenzioni di denominazione e registrare le informazioni minime necessarie a riprodurre l'addestramento.

Gli input comprendono il modello compilato, i tensori encoder e decoder per training e validation, i target corrispondenti e la configurazione di training. Gli output comprendono il modello addestrato, la history e i riferimenti agli artefatti persistiti.

Il modulo dipende da TensorFlow/Keras per il training. Non dipende dalle metriche di scikit-learn né dai servizi di inverse transform, poiché la valutazione quantitativa appartiene al Monitoring Layer. La dipendenza dal filesystem e dalla serializzazione JSON deve rimanere circoscritta alla persistenza degli artefatti.

La separazione introdotta migliora il notebook perché rende il training parametrico, ripetibile e indipendente dal contesto interattivo. Consente inoltre di verificare separatamente callback, history e convenzioni di salvataggio.

19.8 Modulo `predictor.py`

Il modulo `predictor.py` gestisce l'inferenza mediante modelli già addestrati. La sua responsabilità è ricevere input previsionali validi, invocare il modello corretto e restituire serie di forecast interpretabili, mantenendo separati il percorso del consumo e quello della produzione.

L'API pubblica prevista comprende un'operazione generale di predizione, identificabile come `predict`, e due interfacce orientate al dominio predittivo, dedicate rispettivamente al forecast del consumo e al forecast della produzione. Può inoltre includere il caricamento controllato di un modello persistito, così da evitare che l'orchestratore debba conoscere i dettagli del formato Keras utilizzato.

Le funzioni interne verificano la compatibilità tra modello e tensori, eseguono l'inferenza, normalizzano la forma dell'output, richiedono l'inverse transform al Feature Engineering Layer e associano le previsioni all'indice temporale fornito. La costruzione delle finestre storiche e future resta esterna, perché appartiene a `sequence_builder.py`; analogamente, il fit o la ricostruzione degli scaler non deve essere eseguito dal predictor.

Gli input sono il modello addestrato o il relativo riferimento persistito, le sequenze encoder e decoder già normalizzate, il servizio o artefatto di scaling del target e l'indice temporale dell'orizzonte futuro. Gli output sono due risultati distinti: la serie del consumo previsto e la serie della produzione fotovoltaica prevista, espresse nella scala fisica originale e corredate dai timestamp corrispondenti.

Il modulo dipende da TensorFlow/Keras per caricamento e inferenza, da NumPy per la gestione degli array e, se il contratto di uscita è tabellare, da Pandas per la costruzione delle serie temporali. Dipende inoltre dall'API di inverse transform del Feature Engineering Layer, ma non dai dettagli interni del preprocessing.

Rispetto al notebook, `predictor.py` elimina la dipendenza dall'ordine di esecuzione delle celle, rende espliciti gli input necessari all'inferenza e impedisce che la generazione delle previsioni venga confusa con il calcolo del bilancio energetico o con la produzione dei grafici.

19.9 Dipendenze dal Feature Engineering Layer

Il Forecasting Layer dipende dal Feature Engineering Layer attraverso un contratto preciso. Per il training riceve, per ciascun fenomeno, i tensori encoder `X_enc`, i tensori decoder `X_dec` e i target `y`, separati in training, validation e test. Per l'inferenza riceve l'ultima finestra storica, le feature future già normalizzate e l'indice temporale dell'orizzonte da prevedere.

Il Forecasting Layer non seleziona le feature, non suddivide temporalmente i dataset, non esegue il fit degli scaler e non costruisce le sequenze. Può verificare che gli input rispettino le forme attese, ma non deve correggere autonomamente dati o sequenze non conformi. Questa regola mantiene il confine tra preparazione e modellazione e consente di attribuire gli errori al componente responsabile.

L'inverse transform rappresenta un punto di collaborazione controllata. Le previsioni operative

devono essere espresse nella scala fisica originale, ma i parametri necessari alla trasformazione appartengono al Feature Engineering Layer. Il Forecasting Layer utilizza quindi il servizio pubblico di scaling senza appropriarsi della relativa responsabilità.

19.10 Contratto di ingresso e di uscita

Il contratto di ingresso richiede sequenze numeriche tridimensionali coerenti con il formato campioni, passi temporali e feature. Il numero di campioni deve essere allineato tra encoder, decoder e target; l'orizzonte del decoder deve essere coerente con quello del target; i valori devono risultare già normalizzati secondo gli scaler adattati esclusivamente sul training set. Devono inoltre essere disponibili l'identificatore del fenomeno previsto, la configurazione del modello e, quando necessario, il riferimento allo scaler del target.

Il contratto di uscita del training comprende un modello addestrato, la history delle epoche e i metadati degli artefatti salvati. Il contratto di uscita dell'inferenza comprende invece sequenze temporali di consumo previsto e produzione prevista, associate ai relativi timestamp e riportate nelle unità fisiche utilizzate dal progetto. Predizioni e target costituiscono inoltre gli input del Monitoring Layer.

I due forecast rimangono separati all'uscita del layer. La loro combinazione non è responsabilità del Forecasting Layer, ma costituisce l'input del successivo Energy Balance Layer.

19.11 Responsabilità esterne al Forecasting Layer

La modularizzazione richiede che alcune attività presenti nelle celle successive del notebook rimangano esplicitamente esterne. Il caricamento, la generazione e la pulizia dei dati appartengono al Data Layer. La costruzione delle feature, lo scaling, la suddivisione temporale e la generazione delle sequenze appartengono al Feature Engineering Layer.

Il calcolo di `net_balance`, `grid_import`, `grid_export` e dei relativi totali e cumulativi appartiene all'Energy Balance Layer. Il Forecasting Layer deve limitarsi a produrre le due grandezze previsionali necessarie, già nella scala fisica originale, senza attribuire etichette operative o interpretazioni economiche ai risultati.

Il calcolo di MAE e RMSE e l'orchestrazione del confronto quantitativo appartengono al Monitoring Layer. La rappresentazione grafica appartiene al Visualization Layer; il bilancio energetico e i KPI appartengono all'Energy Balance Layer.

Anche l'orchestrazione end-to-end, la scelta del modello da eseguire, la definizione dei percorsi applicativi e la pubblicazione dei risultati non devono essere incorporate nei moduli predittivi. Il notebook ottimizzato o un futuro componente applicativo coordineranno i layer attraverso le rispettive API pubbliche.

19.12 Miglioramenti architetturali

La nuova organizzazione elimina la duplicazione del processo di training tra consumo e produzione, centralizzando le responsabilità comuni e mantenendo configurabili gli elementi specifici. La costruzione del modello diventa indipendente dai dati, il training diventa ripetibile e l'inferenza può essere eseguita senza ricostruire manualmente lo stato del notebook.

La separazione tra `lstm_model.py`, `trainer.py` e `predictor.py` migliora testabilità e manutenibilità. La struttura del modello può essere verificata senza addestramento completo; il trainer può essere controllato su dataset ridotti; il predictor può essere validato con artefatti noti e input deterministici. Le history, le predizioni e i modelli persistiti diventano inoltre artefatti tracciabili e utilizzabili dal Monitoring Layer.

L'architettura consente infine di evolvere il sistema senza modificare i layer precedenti. Nuove configurazioni LSTM o nuovi modelli predittivi potranno essere introdotti dietro contratti compatibili, mentre il Data Layer e il Feature Engineering Layer continueranno a fornire dati e sequenze attraverso interfacce stabili.

Il Forecasting Layer è stato implementato nei moduli `lstm_model.py`, `trainer.py` e `predictor.py`, mantenendo la separazione delle responsabilità definita in questo capitolo. Il layer costituisce ora la sorgente ufficiale delle previsioni utilizzate dal Monitoring Layer e dal Visualization Layer, mentre le successive interpretazioni energetiche rimangono esterne al perimetro predittivo.

20 Monitoring Layer

20.1 Obiettivo e stato attuale

Il Monitoring Layer è un layer ufficiale e implementato dell'architettura. Ha la responsabilità di valutare quantitativamente la qualità delle previsioni prodotte dalla piattaforma. Esso si colloca a valle del Forecasting Layer e riceve valori osservati e valori predetti già espressi nella medesima scala fisica, restituendo indicatori numerici utilizzabili per verificare il comportamento dei modelli di consumo energetico e produzione fotovoltaica.

La prima versione del layer è stata completata attraverso l'implementazione dei moduli `metrics.py` ed `evaluation.py`, collocati nella cartella `07_projects/forecasting_energy/src/monitoring`. L'implementazione trasferisce in componenti Python riutilizzabili la logica di valutazione presente nel notebook originale, mantenendo un perimetro intenzionalmente limitato alle funzionalità effettivamente realizzate.

Il Monitoring Layer iniziale calcola Mean Absolute Error e Root Mean Squared Error e orchestra il confronto tra valori reali e predetti. Non include visualizzazioni, analisi del drift, monitoraggio operativo, gestione degli scaler o attività collegate al ciclo di addestramento. Questa delimitazione mantiene coerente il principio di separazione delle responsabilità adottato nei layer precedenti.

20.2 Architettura implementata

La struttura attualmente disponibile è la seguente:

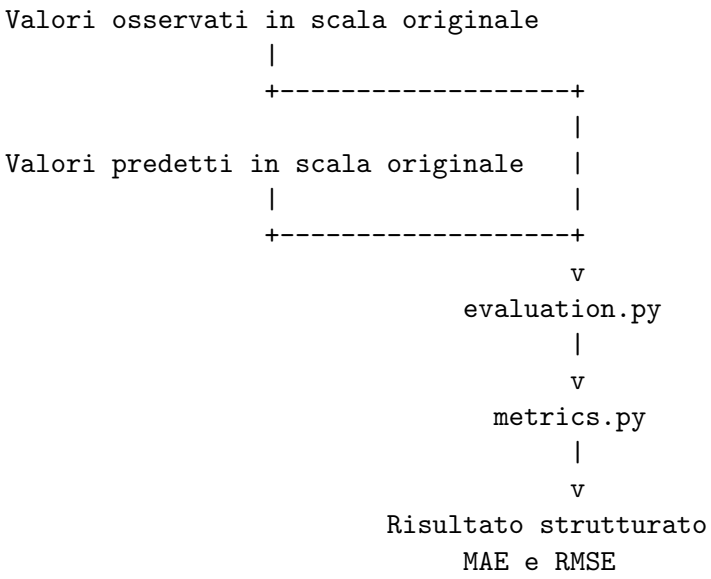
```
07_projects/forecasting_energy/src/
‘-- monitoring/
    |-- metrics.py
    ‘-- evaluation.py
```

I due moduli svolgono compiti complementari. `metrics.py` contiene esclusivamente il calcolo delle metriche quantitative. `evaluation.py` coordina la valutazione, riceve valori osservati e predetti, richiama le metriche disponibili e restituisce il risultato in forma strutturata.

La dipendenza interna è unidirezionale: `evaluation.py` utilizza i servizi esposti da `metrics.py`, mentre `metrics.py` non conosce il processo di valutazione complessivo. Nessuno dei due moduli dipende dalla struttura interna dei modelli di forecasting o dalle modalità con cui i dati sono stati preparati.

20.3 Pipeline operativa

La pipeline implementata può essere rappresentata nel seguente modo:



La pipeline richiede che osservazioni e predizioni siano già confrontabili. Il Monitoring Layer non esegue trasformazioni preliminari, non ricostruisce gli scaler e non modifica la scala dei dati. `evaluation.py` governa il confronto e delega a `metrics.py` il calcolo matematico; il risultato contiene gli indicatori quantitativi prodotti dalla valutazione.

Lo stesso percorso è riutilizzabile per entrambi i fenomeni energetici. Il layer non incorpora formule specifiche per il consumo o per la produzione fotovoltaica: la distinzione dipende dai valori forniti in ingresso, mentre il processo di valutazione rimane comune.

20.4 Modulo `metrics.py`

Il modulo `metrics.py` implementa le operazioni numeriche elementari del Monitoring Layer. La sua responsabilità è limitata al calcolo di MAE e RMSE a partire da valori osservati e predetti già correttamente allineati e riportati alla scala fisica originale.

Il Mean Absolute Error misura la media del valore assoluto degli scostamenti tra osservazioni e previsioni. L'indicatore restituisce un errore espresso nella stessa unità della grandezza energetica analizzata e fornisce una misura immediatamente interpretabile dell'ampiezza media degli scostamenti.

Il Root Mean Squared Error calcola la radice quadrata della media degli errori quadratici. Rispetto al MAE attribuisce un peso maggiore agli scostamenti più elevati e permette quindi di evidenziare maggiormente la presenza di errori previsionali rilevanti.

L'API pubblica del modulo espone le funzionalità necessarie al calcolo delle due metriche. Gli input sono le sequenze dei valori reali e dei valori predetti; gli output sono i corrispondenti valori numerici di MAE e RMSE. Le operazioni interne restano circoscritte alla validazione numerica minima necessaria, alla preparazione richiesta dal calcolo e all'applicazione delle formule metriche.

Il modulo non effettua scaling o inverse scaling, non carica dati, non costruisce feature, non addestra modelli e non genera previsioni. Non produce grafici e non interpreta le metriche attraverso soglie operative. Questa indipendenza rende il componente riutilizzabile sia per il consumo sia per la produzione fotovoltaica.

Rispetto al notebook monolitico, `metrics.py` centralizza le formule in un unico punto, elimina la duplicazione e permette di verificare separatamente la correttezza degli indicatori. Il modulo costituisce il livello quantitativo di base su cui opera `evaluation.py`.

20.5 Modulo `evaluation.py`

Il modulo `evaluation.py` implementa l'orchestrazione della valutazione quantitativa. Riceve i valori osservati e predetti, ne coordina il confronto, utilizza `metrics.py` per il calcolo degli indicatori e restituisce in forma strutturata i risultati della valutazione.

L'API pubblica del modulo espone il servizio di valutazione utilizzabile dai componenti che dispongono di una coppia di serie osservate e previste. Gli input sono esclusivamente i valori reali e predetti già espressi nella stessa scala; l'output raccoglie MAE e RMSE in una struttura unica, evitando che ogni chiamante debba invocare e organizzare separatamente le metriche.

Le responsabilità interne riguardano il coordinamento del confronto e la composizione del risultato. Il modulo non contiene una seconda implementazione delle formule: il calcolo resta delegato a `metrics.py`. Questa relazione mantiene separato il livello matematico dal livello di orchestrazione.

La stessa valutazione può essere applicata al modello di consumo e al modello di produzione fotovoltaica. `evaluation.py` non contiene logiche di business specifiche per i due fenomeni e non combina i risultati in un indicatore unico. Il chiamante mantiene l'identità del target valutato e utilizza il medesimo servizio per ottenere metriche confrontabili all'interno del relativo contesto.

Il modulo non richiama direttamente i modelli di Machine Learning, non esegue training o prediction e non gestisce la persistenza degli artefatti. Riceve risultati già disponibili e restituisce esclusivamente la valutazione quantitativa prevista dall'implementazione attuale.

Rispetto al notebook, `evaluation.py` elimina l'orchestrazione manuale distribuita tra celle, rende uniforme il confronto tra valori reali e predetti e fornisce un contratto di uscita riutilizzabile dai componenti successivi.

20.6 Contratto sulla scala dei valori

MAE e RMSE devono essere calcolati esclusivamente su valori espressi nella scala fisica originale. Questa regola preserva il comportamento corretto già adottato nel notebook e garantisce che gli errori siano interpretabili nell'unità energetica del progetto.

Il Monitoring Layer assume quindi come precondizione che valori osservati e predetti siano già stati riportati alla scala corretta. Lo scaling e l'inverse scaling appartengono al Feature Engineering Layer e ai contratti utilizzati dal Forecasting Layer per restituire output interpretabili. `metrics.py` ed `evaluation.py` non ricevono la responsabilità di adattare, salvare o ricostruire scaler.

Separare il ripristino della scala dal calcolo delle metriche riduce l'accoppiamento e impedisce che la valutazione dipenda da dettagli interni della preparazione dei dati. Il chiamante deve garantire la coerenza della scala prima di invocare il servizio di valutazione.

20.7 Relazioni con gli altri layer

Il Data Layer rimane responsabile del caricamento, della generazione, della validazione e della pulizia dei dati. Il Monitoring Layer non utilizza direttamente i moduli interni del Data Layer e non replica i controlli strutturali già centralizzati in `loader.py`, `validator.py` e `synthetic_data.py`.

Il Feature Engineering Layer mantiene la responsabilità di preprocessing, scaling, inverse scaling e costruzione delle sequenze. Il Monitoring Layer riceve soltanto valori già disponibili nella scala fisica richiesta e non interviene sul processo che li ha prodotti.

Il Forecasting Layer mantiene costruzione, compilazione e addestramento dei modelli, gestione delle predizioni e produzione dei forecast. Il Monitoring Layer non conosce l'architettura LSTM e non

invoca direttamente trainer o predictor; valuta valori reali e predetti forniti attraverso il contratto applicativo.

Il Visualization Layer rimane responsabile dei grafici e delle rappresentazioni visuali. I risultati strutturati prodotti da `evaluation.py` costituiscono uno dei suoi input, ma i moduli di monitoring non generano artefatti grafici. L'Energy Balance Layer utilizza invece le previsioni per produrre bilanci e KPI.

20.8 Responsabilità escluse dall'implementazione iniziale

L'implementazione corrente non comprende preparazione dei dati, feature engineering, scaling, inverse scaling, training, prediction, visualizzazione o calcolo del bilancio energetico. Non comprende inoltre salvataggio storico delle metriche, definizione di soglie, generazione di alert, schedulazione delle valutazioni o attivazione automatica del retraining.

Il layer non interpreta MAE e RMSE come decisioni di accettazione o rifiuto del modello. Produce indicatori quantitativi; eventuali criteri di qualità, policy di rilascio o regole operative dovranno essere definiti in componenti o fasi successive.

Questa delimitazione evita di attribuire a `metrics.py` ed `evaluation.py` responsabilità non presenti nel codice e mantiene l'implementazione iniziale coerente con il principio di modularizzazione incrementale.

20.9 Componenti futuri

Il modulo `drift_monitor.py` non fa parte del Monitoring Layer implementato. La rilevazione del drift richiederà in futuro previsioni archiviate, valori reali acquisiti dopo l'esercizio, una baseline di riferimento, finestre temporali di confronto, soglie approvate e regole di escalation. Fino alla disponibilità di tali elementi, il drift rimane una possibile evoluzione e non una funzionalità corrente.

Anche eventuali componenti di monitoring operativo restano futuri. Registrazione continuativa delle prestazioni, alert, dashboard operative, schedulazione delle verifiche e integrazione con processi di retraining non sono inclusi nei due moduli disponibili.

Il Visualization Layer e l'Energy Balance Layer sono componenti distinti dal Monitoring Layer. Il primo rappresenta risultati già calcolati; il secondo dispone di un contratto definitivo per applicare regole energetiche ai forecast. Nessuno dei due trasferisce responsabilità nei moduli di monitoring.

20.10 Stato di implementazione

Il Monitoring Layer iniziale è stato completato con successo per il perimetro definito. Sono disponibili `metrics.py`, dedicato esclusivamente al calcolo di MAE e RMSE, ed `evaluation.py`, dedicato

all'orchestrazione del confronto tra valori osservati e predetti e alla restituzione strutturata delle metriche.

Il layer non dispone di una Issue numerata dedicata nel backlog storico. La sua implementazione è documentata come attività non numerata soltanto per preservare la sequenza delle Issue esistenti; dal punto di vista architetturale e implementativo il Monitoring Layer ha lo stesso status ufficiale degli altri layer presenti in `src`.

I componenti implementati sono riutilizzabili per la valutazione del consumo energetico e della produzione fotovoltaica e operano esclusivamente su valori già riportati alla scala originale. Il progetto dispone quindi di una prima separazione concreta tra produzione delle previsioni e misurazione quantitativa della loro qualità.

Restano futuri `drift_monitor.py`, il monitoraggio operativo, la storicizzazione delle prestazioni, gli alert e le eventuali politiche di retraining. Energy Balance e Visualization risultano invece implementati nei rispettivi moduli.

20.11 Benefici architetturali

L'implementazione del Monitoring Layer separa il calcolo delle metriche dal Forecasting Layer e dalla visualizzazione. Il modulo `metrics.py` centralizza le formule quantitative. Il modulo `evaluation.py` fornisce invece un unico punto di accesso per il confronto tra osservazioni e previsioni.

La struttura riduce la duplicazione, migliora la riusabilità e permette di testare indipendentemente calcolo e orchestrazione. La stessa logica può essere applicata ai due modelli senza introdurre dipendenze dalla loro architettura interna.

Il layer costituisce inoltre una base controllata per evoluzioni successive. Nuove metriche, visualizzazioni o capacità di monitoring operativo potranno essere aggiunte in modo incrementale, senza modificare le responsabilità già implementate e senza trasferire nel Monitoring Layer funzioni appartenenti agli altri componenti dell'architettura.

21 Energy Balance Layer

21.1 Obiettivo e stato architetturale

L'Energy Balance Layer ha la responsabilità di trasformare le previsioni di consumo energetico e produzione fotovoltaica in grandezze fisiche utili a descrivere lo scambio netto con la rete. Il layer non produce forecast e non valuta la qualità dei modelli: applica esclusivamente regole deterministiche a due serie previsionali già disponibili, allineate e riportate alla scala fisica originale.

La struttura architetturale definitiva prevede i moduli `balance_calculator.py` e `kpi.py` nella

cartella 07_projects/forecasting_energy/src/energy_balance. Questo capitolo costituisce il contratto di riferimento per la successiva implementazione tramite Codex. Le responsabilità descritte derivano dalla logica osservata nel notebook Previsione Fabbisogno v05_all.ipynb, ma vengono riorganizzate secondo la separazione dei compiti stabilita dall'architettura.

21.2 Struttura prevista

```
src/
‘-- energy_balance/
    |-- balance_calculator.py
    ‘-- kpi.py
```

Il modulo balance_calculator.py è responsabile dei calcoli elementari eseguiti per ciascun intervallo temporale. Il modulo kpi.py è responsabile delle aggregazioni totali e cumulative costruite a partire dai risultati del calcolatore. I due moduli non devono contenere caricamento o salvataggio di file, visualizzazione, metriche di forecasting o logiche decisionali ulteriori.

21.3 Derivazione dal notebook originale

Nel notebook originale la logica del bilancio compare dopo la generazione delle previsioni future. Una prima elaborazione costruisce la colonna bilancio come produzione prevista meno consumo previsto e aggiunge una colonna testuale etichetta, valorizzata come "rilascio" quando il bilancio è positivo e come "acquisto" negli altri casi. Questa seconda operazione classifica implicitamente anche il valore zero come acquisto.

Una cella successiva ripete il calcolo con la colonna net_balance e separa i flussi attraverso to_grid e to_buy. Il notebook calcola inoltre la somma dei due flussi sull'orizzonte di sette giorni. Nella sezione grafica vengono infine create le colonne cumulative_grid e cumulative_buy mediante somma cumulativa.

La nuova architettura conserva il significato matematico di queste operazioni, ma adotta una nomenclatura unica e separa calcolo, KPI, visualizzazione e persistenza. La corrispondenza è la seguente:

Notebook	**Naming canonico**	**Destinazione**
bilancio, net_balance	net_balance	balance_calculator.py
to_grid	grid_export	balance_calculator.py
to_buy	grid_import	balance_calculator.py
somma di to_grid	total_grid_export	kpi.py
somma di to_buy	total_grid_import	kpi.py
cumulative_grid	cumulative_grid_export	kpi.py
cumulative_buy	cumulative_grid_import	kpi.py
etichetta	nessun campo obbligatorio	esclusa dal contratto

Il salvataggio di `bilancio_energetico.csv`, presente nel notebook, documenta il comportamento del prototipo ma non costituisce una responsabilità da trasferire nei due moduli del layer.

21.4 Contratto di ingresso

Il layer riceve due serie temporali: `consumption` e `production`. Nel flusso corrente esse rappresentano rispettivamente il consumo previsto e la produzione fotovoltaica prevista per ciascuna ora dell'orizzonte futuro.

Le due serie devono rispettare le seguenti precondizioni:

- stessa lunghezza, stesso ordine e identico indice temporale;
- valori numerici confrontabili e privi di trasformazioni di scaling residue;
- stessa unità di misura e stessa granularità temporale;
- timestamp già validati e ordinati dai componenti precedenti;
- assenza di operazioni implicite di join, interpolazione, riempimento o ricampionamento nel layer.

Il contratto non autorizza il layer a correggere silenziosamente gli input. In particolare, valori negativi di consumo o produzione non devono essere trasformati automaticamente in zero, sostituiti con valori assoluti o sottoposti ad altro clipping. Eventuali regole di plausibilità fisica appartengono alla validazione a monte o a una futura decisione esplicita; l'Energy Balance Layer deve preservare i valori ricevuti oppure segnalare una violazione del contratto, senza modificarli implicitamente.

21.5 Formule canoniche

Per ogni intervallo temporale t, il bilancio netto è definito come:

$$\texttt{net_balance}_t = \texttt{production}_t - \texttt{consumption}_t$$

Un valore positivo di `net_balance` indica che la produzione supera il consumo nell'intervallo considerato. Un valore negativo indica invece che il consumo supera la produzione. Il valore zero indica equilibrio esatto e non richiede alcuna classificazione testuale obbligatoria.

Il flusso verso la rete è definito come:

$$\texttt{grid_export}_t = \max(\texttt{net_balance}_t, 0)$$

`grid_export` rappresenta quindi la quota non negativa del bilancio associata a un'eccedenza di produzione. Quando il bilancio è nullo o negativo, il valore è zero.

Il flusso prelevato dalla rete è definito come:

$$\texttt{grid_import}_t = \max(-\texttt{net_balance}_t, 0)$$

`grid_import` rappresenta la quota non negativa necessaria a coprire la parte di consumo non soddisfatta dalla produzione. Quando il bilancio è nullo o positivo, il valore è zero.

L'uso della funzione `max` in queste due formule non costituisce clipping degli input. È esclusivamente la decomposizione matematica del bilancio netto nei due flussi mutuamente esclusivi di importazione ed esportazione.

21.6 Contratto di uscita

L'output elementare del layer conserva l'indice temporale ricevuto e rende disponibili almeno le seguenti grandezze:

- `consumption`, valore di consumo utilizzato nel calcolo;
- `production`, valore di produzione utilizzato nel calcolo;
- `net_balance`, differenza con segno tra produzione e consumo;
- `grid_export`, eccedenza non negativa verso la rete;
- `grid_import`, fabbisogno non negativo prelevato dalla rete.

Il contratto non prevede una colonna obbligatoria di stato. In particolare, il caso `net_balance = 0` non deve essere forzato nelle categorie acquisto o rilascio. Un'eventuale classificazione descrittiva potrà essere aggiunta in futuro da un componente di presentazione o analisi operativa, purché non modifichi le grandezze canoniche.

21.7 Granularità temporale, unità di misura e scala fisica

Il notebook opera su una granularità oraria e presenta consumo, produzione, importazione ed esportazione in `kWh`. Nel contratto corrente ogni valore rappresenta quindi energia riferita al singolo intervallo di un'ora, mentre totali e cumulativi mantengono la stessa unità energetica.

Il layer non converte potenza in energia, non modifica la frequenza temporale e non deduce autonomamente l'unità di misura. Se in futuro gli input utilizzeranno granularità o unità differenti, la conversione dovrà essere definita esplicitamente prima dell'invocazione del layer.

Consumo e produzione devono essere già riportati alla scala fisica originale. Scaling e inverse scaling appartengono al Feature Engineering Layer e al contratto di uscita del Forecasting Layer. L'Energy Balance Layer non deve conoscere gli scaler né operare su valori normalizzati.

21.8 Responsabilità di `balance_calculator.py`

`balance_calculator.py` deve implementare esclusivamente il calcolo vettoriale o temporale di `net_balance`, `grid_export` e `grid_import`. Deve preservare indice, ordine, scala e unità degli input e deve rendere espliciti gli errori di allineamento o incompatibilità strutturale.

Il modulo non deve:

- caricare o salvare file CSV, Excel o altri formati;
- eseguire preprocessing, scaling, inverse scaling o imputazione;
- applicare clipping automatico a consumo o produzione;
- creare una classificazione obbligatoria per acquisto, rilascio o bilancio nullo;
- calcolare MAE, RMSE o altre metriche previsionali;
- addestrare modelli o generare predizioni;
- calcolare costi, ricavi, tariffe o logiche di ottimizzazione;
- produrre grafici, report o dashboard.

21.9 Responsabilità di `kpi.py`

`kpi.py` riceve i flussi canonici già calcolati e produce le aggregazioni minime formalmente previste. I KPI obbligatori sono:

$$\texttt{total_grid_export} = \sum_t \texttt{grid_export}_t$$

$$\texttt{total_grid_import} = \sum_t \texttt{grid_import}_t$$

$$\texttt{cumulative_grid_export}_t = \mathrm{cumsum}(\texttt{grid_export})_t$$

$$\texttt{cumulative_grid_import}_t = \mathrm{cumsum}(\texttt{grid_import})_t$$

I due totali sono valori scalari riferiti all'intero orizzonte analizzato. Le due grandezze cumulative sono serie temporali allineate all'indice di ingresso e descrivono l'accumulo progressivo dell'energia esportata e importata.

Il modulo non deve ricalcolare il bilancio a partire dai forecast, alterare i flussi ricevuti o introdurre KPI ulteriori non approvati. Costi, ricavi, tariffe, batterie, emissioni di CO_2, curtailment, demand response e ottimizzazione non appartengono al contratto corrente e possono essere considerati soltanto come future evoluzioni formalmente autorizzate.

21.10 Confini con gli altri layer

Il Forecasting Layer produce le serie previsionali di consumo e produzione. Deve consegnarle nella scala fisica originale, ma non deve calcolare `net_balance`, `grid_import` o `grid_export`.

Il Monitoring Layer confronta valori osservati e predetti e calcola MAE e RMSE. Non partecipa al calcolo del bilancio energetico e l'Energy Balance Layer non dipende dai risultati di valutazione per applicare le proprie formule.

Il Visualization Layer può rappresentare consumo, produzione e le grandezze canoniche del bilancio. Queste comprendono `net_balance`, `grid_import`, `grid_export`, totali e cumulativi. Il layer visuale non deve ricalcolare tali valori, creare formule alternative o introdurre nomi divergenti.

Il Data Layer e il Feature Engineering Layer restano responsabili rispettivamente della qualità dei dati e della preparazione numerica. L'Energy Balance Layer non duplica validazione dei dataset, trasformazioni o gestione degli scaler.

21.11 CSV, file I/O e orchestrazione

Nel notebook il dataframe finale viene salvato come `bilancio_energetico.csv`. Questa operazione appartiene al comportamento del prototipo, ma viene esclusa da `balance_calculator.py` e `kpi.py`. I moduli devono ricevere oggetti già disponibili in memoria e restituire risultati in memoria.

L'eventuale esportazione CSV deve essere gestita dal notebook ottimizzato, da un orchestratore applicativo o da un futuro componente dedicato a reporting e persistenza. Lo stesso principio vale per la scelta dei percorsi, la creazione delle directory, il logging dell'esportazione e la serializzazione degli output.

21.12 Operational Analysis ed evoluzioni future

Operational Analysis rimane un concetto architetturale più ampio dell'Energy Balance Layer. Può utilizzare forecast, metriche, bilanci, KPI e visualizzazioni per supportare decisioni, ma non corrisponde a una responsabilità aggiuntiva dei due moduli qui definiti.

Qualsiasi futura estensione relativa a tariffe, costi, ricavi, batterie, autoconsumo, emissioni, curtailment, demand response, soglie decisionali o ottimizzazione dovrà essere formalizzata separatamente. Nessuna di queste funzionalità è implicita nelle formule canoniche o deve essere anticipata durante l'implementazione iniziale.

21.13 Criteri di completamento architetturale

L'implementazione potrà essere considerata coerente con questo capitolo quando i due moduli applicheranno il naming canonico, preserveranno scala e allineamento temporale, produrranno esclusivamente i calcoli e i KPI definiti, non classificheranno obbligatoriamente il caso zero, non applicheranno clipping automatico agli input e non conterranno file I/O.

Il notebook originale rimane la prova della logica funzionale di partenza. La nuova architettura ne conserva le formule essenziali, sostituisce `to_grid` e `to_buy` con `grid_export` e `grid_import`, unifica `bilancio` e `net_balance` nel nome `net_balance` e assegna totali e cumulativi a `kpi.py`. In questa forma il capitolo fornisce a Codex un contratto non ambiguo per la successiva fase di implementazione.

22 Visualization Layer

22.1 Obiettivo, perimetro e stato

Il Visualization Layer è il componente responsabile della rappresentazione grafica di risultati già prodotti dagli altri layer della piattaforma. Il suo obiettivo è rendere leggibili forecast, confronti tra valori osservati e predetti, diagnostica del training, metriche quantitative e risultati energetici, senza modificare i valori ricevuti e senza duplicare le logiche che li hanno generati.

Il materiale sorgente fornito per la v15 contiene un'implementazione di `plotting.py` e il sorgente della suite `tests/test_visualization_plotting.py`, coerenti con le sette API descritte in questo capitolo. La v14 riportava la GitHub Issue #5 – *Create Visualization Layer* come aperta; lo stato amministrativo attuale, l'eventuale Pull Request, la review e il merge non sono verificabili dai materiali disponibili. Il capitolo resta il contratto architetturale di riferimento e le specifiche delle API non vengono modificate sulla sola base della presenza del codice.

La struttura prevista è intenzionalmente essenziale:

```
src/
‘-- visualization/
    ‘-- plotting.py
```

Dashboard, servizi web, composizione automatica di report e pubblicazione degli artefatti rimangono evoluzioni future. La Issue #5 riguarda esclusivamente funzioni grafiche riutilizzabili e in-

dipendenti dal notebook.

22.2 Posizione architetturale

Il Visualization Layer è un consumer finale di risultati provenienti da più rami della piattaforma. Le dipendenze corrette sono:

```
Forecasting Layer --> Monitoring Layer
Forecasting Layer --> Energy Balance Layer
Forecasting Layer --> Visualization Layer
Monitoring Layer  --> Visualization Layer
Energy Balance Layer --> Visualization Layer
```

Il Monitoring Layer non è un prerequisito dell'Energy Balance Layer e l'Energy Balance Layer non è un prerequisito del Monitoring Layer. Entrambi utilizzano output del Forecasting Layer per finalità differenti. Il Visualization Layer può ricevere direttamente previsioni, valutazioni quantitative e risultati energetici e rappresentarli separatamente.

22.3 Derivazione dal notebook originale

Il notebook `Previsione Fabbisogno v05_all.ipynb` contiene le seguenti famiglie di visualizzazione:

1. grafici delle serie contenute nei dataset e matrice di correlazione, prodotti dalla funzione esplorativa `analisi_statistica`;
2. curve di training loss e validation loss per i modelli di consumo e produzione;
3. confronto tra valori reali e predetti per una singola finestra temporale;
4. distribuzione degli errori mediante istogramma e curva KDE;
5. grafico combinato del forecast futuro di consumo e produzione;
6. confronto orario tra energia importata ed esportata;
7. andamento cumulativo dei flussi di importazione ed esportazione.

La distinzione architetturale separa visualizzazioni operative, diagnostiche ed esplorative. Le curve di loss sono diagnostiche del processo di training ma rientrano nella Issue #5 perché sono già presenti nel notebook e non calcolano nuove metriche. I grafici reale–predetto, distribuzione degli errori, forecast futuro, scambio con la rete e cumulativi sono visualizzazioni obbligatorie del modulo.

I grafici generici dei dataset e la matrice di correlazione rimangono visualizzazioni esplorative del notebook. Non fanno parte dell'implementazione iniziale della Issue #5, perché richiederebbero di trasferire in `plotting.py` una funzione ibrida che combina statistiche descrittive, correlazioni e rappresentazione grafica. Potranno essere formalizzati in futuro come strumenti diagnostici separati.

Il requisito corretto relativo agli errori è la distribuzione statistica mediante istogramma e KDE, coerente con il notebook. Un grafico temporale degli errori non è obbligatorio nella Issue #5 e rimane una possibile evoluzione futura.

Il forecast futuro di consumo e produzione deve essere rappresentato nello stesso grafico, come nel notebook. La separazione tra consumo e produzione riguarda invece i confronti reale–predetto: la stessa funzione generica viene invocata una volta per il consumo e una volta per la produzione.

22.4 Principi dell'API pubblica

`plotting.py` deve esporre esclusivamente le sette funzioni pubbliche definite in questo capitolo. Le funzioni ricevono dati già calcolati, costruiscono la figura e restituiscono una coppia composta da `matplotlib.figure.Figure` e `matplotlib.axes.Axes`.

Le funzioni non devono chiamare `show()`, non devono salvare file e non devono chiudere la figura. Il notebook ottimizzato o l'orchestratore decidono se mostrare, salvare o chiudere l'oggetto ricevuto. Questo contratto rende le funzioni utilizzabili sia in modalità interattiva sia nei test automatici.

Le firme riportate di seguito sono contratti architetturali e non costituiscono implementazione Python completa.

22.5 API `plot_training_history`

```
plot_training_history(
    history,
    *,
    title=None,
    figsize=(8, 5),
    colors=None,
    show_grid=True
) -> tuple[Figure, Axes]
```

La funzione rappresenta training loss e validation loss in funzione delle epoche. `history` è un mapping con le chiavi obbligatorie `loss` e `val_loss`; i valori sono sequenze numeriche non vuote della stessa lunghezza. Il modulo non deve dipendere da TensorFlow o dal tipo Keras `History`: l'orchestratore passa il relativo mapping numerico.

L'asse orizzontale rappresenta le epoche e l'asse verticale la loss, identificata come MSE nel flusso

corrente. La loss è diagnostica del training e non è espressa necessariamente in kWh. colors, quando specificato, contiene i due colori delle curve.

La funzione solleva TypeError per strutture non compatibili e ValueError per chiavi mancanti, sequenze vuote, valori non numerici o lunghezze differenti.

22.6 API plot_prediction_window

```
plot_prediction_window(
    y_true,
    y_pred,
    *,
    target_name,
    window_index=0,
    timestamps=None,
    unit="kWh",
    title=None,
    figsize=(10, 5),
    colors=None,
    show_grid=True
) -> tuple[Figure, Axes]
```

La funzione confronta valori reali e predetti per una singola finestra. y_true e y_pred sono array NumPy numerici con forma (n_windows, horizon) oppure (n_windows, horizon, 1) e devono avere la stessa forma. I dati devono essere già riportati alla scala fisica originale.

window_index seleziona la finestra da mostrare. timestamps è opzionale. Quando è presente, deve essere un oggetto DatetimeIndex di Pandas con lunghezza pari all'orizzonte della finestra; in caso contrario l'asse orizzontale utilizza lo step temporale relativo. target_name identifica consumo o produzione e viene utilizzato nel titolo e nella legenda.

La funzione solleva TypeError per tipi o valori non numerici e ValueError per array vuoti, forme differenti, dimensionalità non supportata, indice della finestra fuori intervallo o timestamp di lunghezza incompatibile.

22.7 API plot_error_distribution

```
plot_error_distribution(
    errors,
    *,
    target_name,
    bins=50,
    kde=True,
    unit="kWh",
```

```
    title=None,
    figsize=(8, 5),
    color=None,
    show_grid=True
) -> tuple[Figure, Axes]
```

La funzione rappresenta la distribuzione degli errori mediante istogramma e, quando `kde=True`, curva KDE. `errors` è una sequenza numerica monodimensionale o un array NumPy appiattibile, già calcolato come valore osservato meno valore predetto e già espresso nella scala fisica originale.

Il Visualization Layer non calcola gli errori: la serie viene preparata dall'orchestratore a partire da valori già allineati oppure resa disponibile da un componente responsabile. La figura include una linea verticale in corrispondenza dello zero. `bins` deve essere un intero positivo.

La funzione solleva `TypeError` per input non numerici e `ValueError` per serie vuote, valori non finiti o numero di bin non valido.

22.8 API `plot_evaluation_metrics`

```
plot_evaluation_metrics(
    metrics,
    *,
    target_name,
    unit="kWh",
    title=None,
    figsize=(6, 4),
    colors=None,
    show_grid=False
) -> tuple[Figure, Axes]
```

La funzione produce un riepilogo grafico a barre dei valori MAE e RMSE già calcolati dal Monitoring Layer. Per evitare dipendenze da una classe concreta non ancora formalizzata, `metrics` è un mapping con chiavi obbligatorie `mae` e `rmse` e valori numerici finiti e non negativi.

Il modulo non deve importare o invocare `metrics.py` o `evaluation.py`; riceve esclusivamente il mapping prodotto o adattato dall'orchestratore. La funzione solleva `TypeError` per strutture incompatibili e `ValueError` per chiavi mancanti, valori non numerici, non finiti o negativi.

22.9 API `plot_future_forecast`

```
plot_future_forecast(
    consumption,
    production,
```

```
    *,
    unit="kWh",
    title=None,
    figsize=(14, 5),
    colors=None,
    show_grid=True
) -> tuple[Figure, Axes]
```

La funzione rappresenta nello stesso grafico il consumo previsto e la produzione prevista sull'orizzonte futuro. `consumption` e `production` sono `pandas.Series` numeriche, non vuote, dotate dello stesso `DatetimeIndex`, con uguale lunghezza e valori già nella scala fisica originale.

Il flusso corrente utilizza una granularità oraria e un orizzonte di 168 ore. La funzione non deve tuttavia ricampionare o imporre tale lunghezza: conserva l'indice ricevuto e segnala serie disallineate. Solleva `TypeError` per tipi o valori non numerici e `ValueError` per input vuoti, indici non temporali, duplicati, non ordinati o differenti.

22.10 API `plot_grid_exchange`

```
plot_grid_exchange(
    grid_import,
    grid_export,
    *,
    unit="kWh",
    title=None,
    figsize=(14, 5),
    colors=None,
    show_grid=True
) -> tuple[Figure, Axes]
```

La funzione rappresenta nello stesso grafico `grid_import` e `grid_export` per ciascun intervallo temporale. Gli input sono `pandas.Series` numeriche prodotte dall'Energy Balance Layer, con identico `DatetimeIndex`, stessa lunghezza, stessa granularità e stessa unità.

La funzione non calcola `net_balance`, non deriva i due flussi e non applica clipping. Solleva `TypeError` per tipi o valori non numerici e `ValueError` per serie vuote, valori non finiti, indici duplicati, non ordinati o disallineati.

22.11 API `plot_cumulative_grid_exchange`

```
plot_cumulative_grid_exchange(
    cumulative_grid_import,
    cumulative_grid_export,
```

```
    *,
    unit="kWh",
    title=None,
    figsize=(14, 5),
    colors=None,
    show_grid=True
) -> tuple[Figure, Axes]
```

La funzione rappresenta i cumulativi di importazione ed esportazione già calcolati da `kpi.py`. Gli input sono `pandas.Series` numeriche, non vuote, con indice temporale identico, ordinato e privo di duplicati.

La funzione non deve applicare `cumsum` e non deve ricostruire i KPI. Solleva `TypeError` per tipi o valori non numerici e `ValueError` per input vuoti, valori non finiti o disallineamento temporale.

22.12 Contratti comuni sui dati

Le API basate su serie temporali accettano esclusivamente `pandas.Series`. L'indice deve essere un `DatetimeIndex` monotono crescente, privo di duplicati e identico tra le serie confrontate. Il modulo non deve eseguire join, interpolazione, riempimento, ordinamento, ricampionamento o conversione di frequenza.

Le API basate sulle finestre del modello accettano array NumPy nelle forme esplicitamente definite. Il modulo può selezionare e appiattire la finestra richiesta esclusivamente per la presentazione, senza alterare gli array ricevuti.

Tutti i dati energetici devono essere già espressi nella scala fisica originale. Nel flusso corrente consumo, produzione, `grid_import`, `grid_export`, `net_balance` e cumulativi sono espressi in `kWh`. MAE e RMSE utilizzano la stessa unità del target. La loss mantiene invece l'unità e la scala utilizzate dal processo di training e deve essere etichettata come MSE, senza conversioni nel layer visuale.

Le funzioni non devono modificare gli oggetti ricevuti. Eventuali copie, conversioni temporanee o selezioni devono rimanere interne e non produrre effetti collaterali sugli input.

Tutte le sequenze numeriche devono contenere esclusivamente valori finiti. Valori `NaN`, infiniti o non numerici costituiscono una violazione del contratto e devono produrre `ValueError` o `TypeError` secondo la natura dell'errore.

22.13 Configurazione grafica ammessa

La configurazione rimane limitata agli aspetti necessari al progetto. Tutte le API possono accettare titolo, dimensione della figura, colori e attivazione della griglia. Le etichette degli assi e le legende possiedono valori predefiniti coerenti con la funzione e con il parametro `unit`.

`title` è una stringa opzionale; `figsize` è una coppia di numeri positivi; `show_grid` è un valore booleano; `unit` è una stringa non vuota. `colors` è una sequenza opzionale di colori Matplotlib con un elemento per ciascuna serie o barra rappresentata. Quando non viene fornita, la funzione utilizza colori locali predefiniti. `color` identifica invece il singolo colore della distribuzione degli errori.

`plot_prediction_window` espone `window_index`; `plot_error_distribution` espone `bins` e `kde`. Non sono previsti temi grafici globali, plugin, configurazioni esterne o sistemi generici di styling. La scelta di font, marker e stili di linea può utilizzare default locali semplici e coerenti con il notebook.

22.14 Dipendenze ammesse

Il codice fornito usa inoltre i moduli standard Python `collections`, `numbers` e `typing`.

Il Visualization Layer può dipendere esclusivamente da:

- Matplotlib, per la creazione di figure, assi, linee, barre e istogrammi;
- Pandas, per le serie temporali e il relativo `DatetimeIndex`;
- NumPy, per gli array delle finestre e le verifiche numeriche;
- Seaborn, limitatamente alla rappresentazione dell'istogramma con KDE degli errori.

Il modulo non deve dipendere da TensorFlow, Keras, scikit-learn, dai moduli interni di forecasting, da `metrics.py`, da `evaluation.py`, da `balance_calculator.py` o da `kpi.py`. I risultati di tali componenti vengono passati come dati attraverso l'orchestratore.

22.15 File I/O e orchestrazione

`plotting.py` non deve creare directory, costruire percorsi, salvare PNG, PDF o altri file, pubblicare dashboard o produrre documenti di reporting. Restituisce esclusivamente oggetti Matplotlib in memoria.

L'orchestratore o il notebook ottimizzato è responsabile della selezione delle visualizzazioni da produrre, dell'adattamento degli output dei layer ai contratti definiti, della chiamata a `show()`, del salvataggio tramite gli oggetti restituiti, della chiusura delle figure e dell'eventuale inserimento in report.

22.16 Confini con gli altri layer

Il Forecasting Layer costruisce, addestra e utilizza i modelli e produce previsioni. Il Visualization Layer non esegue prediction, non applica scaling o inverse scaling e non conosce l'architettura LSTM.

Il Monitoring Layer calcola MAE e RMSE e orchestra la valutazione. Il Visualization Layer può rappresentare un mapping contenente tali valori e una serie di errori già preparata, ma non calcola metriche e non decide se un modello è accettabile.

L'Energy Balance Layer calcola `net_balance`, `grid_import`, `grid_export`, totali e cumulativi. Il Visualization Layer non ricalcola, rinomina o modifica queste grandezze e non applica classificazioni operative.

La composizione tra previsioni, metriche, bilanci e KPI appartiene all'orchestratore. Il Visualization Layer riceve soltanto gli input richiesti dalla singola funzione e non deve accedere direttamente alle sorgenti dati o agli artefatti dei modelli.

22.17 Visualizzazioni non incluse e sviluppi futuri

La Issue #5 non include grafici generici dei dataset, matrici di correlazione, statistiche descrittive, andamento temporale degli errori, visualizzazione separata del forecast futuro di consumo e produzione, grafico autonomo di `net_balance`, intervalli di confidenza, dashboard, report automatici o visualizzazioni interattive.

Questi elementi non sono vietati in assoluto, ma richiedono una futura decisione documentata. Eventuali intervalli previsionali potranno essere rappresentati soltanto quando un componente responsabile ne formalizzerà calcolo e contratto.

22.18 Criteri di test e accettazione

La suite specifica fornita per il Visualization Layer verifica:

- restituzione di una coppia `Figure/Axes`;
- presenza del numero atteso di linee, barre o distribuzioni grafiche;
- presenza di titolo, etichette degli assi e legenda quando richiesti;
- gestione corretta di input validi senza modifica degli oggetti ricevuti;
- eccezione coerente per input vuoti, tipi non supportati, valori non numerici o non finiti;
- rilevazione di forme incompatibili, finestre fuori intervallo e indici temporali disallineati;

- assenza di file creati e assenza di chiamate obbligatorie a visualizzazione interattiva;
- assenza di ricalcolo di previsioni, metriche, bilanci o KPI;
- funzionamento con backend Matplotlib non interattivo, necessario per la Continuous Integration.

In particolare, il sorgente del test controlla i nomi, l'ordine dei parametri e i default delle sette API, i tipi `Figure`/`Axes`, elementi grafici essenziali, input validi e categorie di input invalidi, allineamento temporale, configurazioni ammesse, mancata modifica degli input e assenza di dipendenze o operazioni incompatibili con i confini del layer. È stato comunicato il risultato di 18 test superati in 5,14 secondi con backend Agg; è stato inoltre comunicato che `git diff --check` non ha rilevato errori, salvo avvisi Git di conversione LF/CRLF. Questi esiti sono riportati come informazioni comunicate, non come verifiche rieseguite durante la redazione.

La validazione descritta nel codice comprende mapping e chiavi richiesti, dimensioni compatibili, input non vuoti, valori numerici finiti, indici temporali ordinati e senza duplicati, timestamp coerenti con l'orizzonte, numero di bin positivo, metriche non negative e configurazioni di dimensione, titolo, griglia e colori. Nei campioni a errore costante il codice disabilita la KDE, comportamento coperto dal test, per evitare di richiedere una stima della densità su dati a varianza nulla. Le verifiche della suite coprono casi selezionati e non ogni combinazione possibile.

La copertura non è esaustiva per ogni combinazione di input e non dimostra la qualità estetica, l'integrazione end-to-end, il collegamento dal notebook ottimizzato o l'assenza assoluta di difetti. La presenza dei sorgenti nel materiale fornito non dimostra da sola che essi siano stati integrati nel repository ufficiale.

22.19 Criteri di completamento della Issue #5

Nel materiale sorgente esaminato, `plotting.py` espone le sette API documentate, restituisce figure e assi Matplotlib, usa le dipendenze previste e non contiene file I/O, chiamate obbligatorie a `show()` o ricalcoli di forecast, errori, metriche, bilanci e KPI. Il sorgente dei test copre questi confini in parte con controlli di comportamento e in parte con verifiche statiche; il risultato comunicato è di 18 test superati con backend Agg.

Questo consente di registrare come disponibili l'implementazione descritta e la verifica automatica specifica, ma non di dichiarare la Issue amministrativamente chiusa o il lavoro integrato. Restano da verificare la corrispondenza dei sorgenti con il branch ufficiale, la review umana, l'eventuale Pull Request/merge e l'effettivo richiamo dal notebook ottimizzato. Dashboard, reporting avanzato e visualizzazioni esplorative restano fuori dal perimetro iniziale.

Le specifiche del capitolo restano il contratto architetturale. Il materiale fornito documenta l'implementazione e una verifica automatica specifica superata secondo il risultato comunicato. La conformità del branch ufficiale, la review umana e l'integrazione definitiva non sono attestate da queste fonti.

50 Notes, Glossario e Riferimenti Metodologici

50.1 Introduzione

Questa sezione raccoglie definizioni, concetti e terminologia utilizzati nella documentazione del progetto *Energy Forecasting Platform – AI Operational Stack*. L'obiettivo è fornire al lettore un riferimento sintetico per interpretare correttamente le scelte architetturali, il backlog operativo, le specifiche tecniche e il ruolo degli strumenti coinvolti.

Le note non sostituiscono i capitoli precedenti, ma li completano. Esse chiariscono il significato dei termini ricorrenti e rendono più accessibile il collegamento tra gestione del progetto, sviluppo software, machine learning e documentazione tecnica.

50.2 Glossario dei principali concetti

Stack Operativo. Uno Stack Operativo è l'insieme coordinato di strumenti, processi, ruoli e pratiche utilizzati per trasformare un'idea progettuale in un sistema funzionante. Nel contesto di questo progetto, lo stack non include solo tecnologie software, ma anche strumenti di analisi, documentazione, versionamento e automazione.

Stack Operativo Ottimizzato. Lo Stack Operativo Ottimizzato è la configurazione integrata di ChatGPT, Codex, Prism e GitHub descritta in questa documentazione. Il suo valore risiede nella distribuzione chiara delle responsabilità: progettazione e analisi, sviluppo assistito, gestione della conoscenza e tracciabilità del codice.

Architecture Decision Record (ADR). Un ADR è un documento breve che registra una decisione architetturale rilevante, il contesto in cui è stata presa, le alternative considerate e le conseguenze attese. Gli ADR permettono di preservare la memoria tecnica del progetto e di spiegare perché una certa soluzione è stata preferita ad altre.

RFC (Request for Comments). Una RFC è una proposta tecnica sottoposta a discussione prima di essere adottata. Può essere usata per descrivere un cambiamento significativo, raccogliere osservazioni e ridurre il rischio di decisioni affrettate. Nel progetto può precedere un ADR o una modifica importante alla roadmap.

Issue. Una Issue è un'unità di lavoro tracciabile all'interno di GitHub. Descrive un'attività, un problema, una richiesta di miglioramento o un obiettivo tecnico. Nel backlog del progetto, le Issue rappresentano il punto di ingresso operativo per le attività affidate a Codex.

Ticket. Un ticket è una richiesta di lavoro gestita in un sistema di tracciamento. Nel linguaggio operativo può essere usato come sinonimo di Issue, anche se in alcuni contesti indica richieste più orientate al supporto, alla manutenzione o alla gestione di anomalie.

Commit. Un commit è una registrazione puntuale di modifiche al codice o alla documentazione in un repository Git. Ogni commit dovrebbe rappresentare un cambiamento coerente e comprensibile,

accompagnato da un messaggio che ne descrive il contenuto.

Branch. Un branch è una linea di sviluppo separata dal ramo principale del repository. Permette di lavorare su una funzionalità, una correzione o un esperimento senza modificare direttamente la versione stabile del progetto.

Pull Request. Una Pull Request è una proposta di integrazione di modifiche da un branch verso un altro. Consente di revisionare il codice, discutere le scelte implementative, eseguire controlli automatici e approvare il merge nel ramo principale.

Pipeline CI/CD. Una pipeline CI/CD è una sequenza automatizzata di controlli, build, test e distribuzione. La Continuous Integration verifica che le modifiche siano compatibili con il progetto; la Continuous Delivery o Deployment prepara o automatizza il rilascio.

Executive Summary. Un Executive Summary è una sintesi ad alto livello pensata per lettori decisionali. Evidenzia obiettivi, risultati, rischi e implicazioni principali senza entrare nei dettagli tecnici completi.

Bug Fixing. Il Bug Fixing è l'attività di identificazione e correzione di errori nel comportamento del software. Una buona correzione non si limita a eliminare il sintomo, ma cerca di intervenire sulla causa effettiva del problema.

Refactoring. Il Refactoring è la modifica della struttura interna del codice senza alterarne il comportamento funzionale atteso. Serve a migliorare leggibilità, modularità, testabilità e manutenibilità.

Troubleshooting. Il Troubleshooting è il processo sistematico di analisi di un problema operativo o tecnico. Include raccolta di evidenze, formulazione di ipotesi, verifica delle cause e individuazione di una soluzione.

Root Cause Analysis. La Root Cause Analysis è l'analisi della causa radice di un problema. L'obiettivo è evitare correzioni superficiali e individuare il fattore principale che ha generato l'anomalia, così da prevenirne la ricorrenza.

50.3 Concetti di Machine Learning utilizzati nel progetto

Forecasting. Il forecasting è l'attività di previsione di valori futuri a partire da dati storici e variabili esplicative. Nel progetto riguarda la previsione dei consumi energetici e della produzione fotovoltaica su un orizzonte temporale definito.

Serie Temporali. Una serie temporale è una sequenza di osservazioni ordinate nel tempo. Nel dominio energetico, esempi tipici sono il consumo orario, la produzione fotovoltaica e le misure meteorologiche rilevate a intervalli regolari.

Multi-Series Forecasting. Il Multi-Series Forecasting riguarda la previsione coordinata di più serie temporali. In questo progetto le serie principali sono consumo energetico e produzione foto-

voltaica, trattate con modelli separati ma integrate successivamente nel bilancio energetico.

Feature Engineering. Il Feature Engineering è il processo di costruzione, selezione e trasformazione delle variabili utilizzate dai modelli. Include feature temporali, normalizzazione, finestre storiche e variabili meteorologiche utili alla previsione.

Training. Il training è la fase in cui un modello apprende dai dati storici. Durante questa fase il modello modifica i propri parametri interni per ridurre l'errore tra valori previsti e valori osservati.

Validation. La validation è la fase di controllo delle prestazioni del modello su dati non usati direttamente per l'apprendimento. Serve a stimare la capacità del modello di generalizzare e a ridurre il rischio di overfitting.

Inference. L'inference è la fase in cui un modello già addestrato viene utilizzato per produrre previsioni su nuovi dati. Nel progetto corrisponde alla generazione dei forecast futuri di consumo e produzione.

Retraining. Il retraining è il riaddestramento periodico o straordinario di un modello con dati aggiornati. È necessario quando cambiano i pattern di consumo, le condizioni operative o la qualità delle previsioni decade nel tempo.

Drift del Modello. Il drift del modello indica la perdita progressiva di accuratezza dovuta al cambiamento delle distribuzioni dei dati o delle relazioni tra variabili. Nel forecasting energetico può dipendere da nuove abitudini di consumo, modifiche impiantistiche o variazioni climatiche.

50.4 Rete neurale LSTM Sequence-to-Sequence

La rete neurale LSTM, acronimo di *Long Short-Term Memory*, è un tipo di rete neurale ricorrente progettata per trattare sequenze di dati. A differenza di modelli che considerano osservazioni isolate, una LSTM mantiene una memoria interna che le consente di utilizzare informazioni provenienti da istanti precedenti.

Questa caratteristica rende le LSTM adatte alle serie temporali, dove il valore futuro dipende spesso da pattern passati. Nel caso energetico, consumi e produzione possono mostrare cicli giornalieri, settimanali, stagionali e dipendenze legate al meteo o al comportamento degli utenti.

Un modello Sequence-to-Sequence riceve in ingresso una sequenza storica e produce in uscita una sequenza futura. Questo approccio è utile quando non si vuole prevedere un singolo valore, ma un intero orizzonte temporale, ad esempio le successive 168 ore.

Rispetto a modelli tradizionali, come regressioni o metodi statistici più semplici, una LSTM può apprendere relazioni non lineari e dipendenze temporali complesse. Questo la rende interessante per il forecasting energetico, soprattutto quando sono disponibili dati storici sufficienti e variabili esogene come temperatura, irraggiamento o calendario.

I vantaggi principali sono la capacità di modellare sequenze, catturare pattern ricorrenti e gestire

orizzonti previsionali articolati. I limiti riguardano la maggiore complessità computazionale, la necessità di dati ben preparati, la minore interpretabilità rispetto a modelli più semplici e il rischio di overfitting se training e validation non sono gestiti correttamente.

50.5 Ruolo degli strumenti dello Stack

ChatGPT. ChatGPT supporta l'analisi dei requisiti, la progettazione concettuale, la qualità della documentazione e il troubleshooting. Nel progetto aiuta a chiarire obiettivi, strutturare decisioni, formulare prompt per Codex e verificare la coerenza tra architettura, backlog e specifiche.

Codex. Codex è lo strumento orientato allo sviluppo software. Il suo ruolo riguarda implementazione, refactoring, bug fixing e generazione o aggiornamento di test automatici. A partire dalle Issue GitHub, Codex potrà modificare il repository e proporre Pull Request coerenti con i contratti tecnici definiti.

Prism. Prism agisce come laboratorio documentale e spazio di gestione della conoscenza. Supporta ADR, documentazione tecnica, documentazione cliente e continuità tra decisioni progettuali e implementazione. Il suo ruolo è preservare il contesto e rendere leggibile l'evoluzione del progetto.

GitHub. GitHub fornisce versionamento, collaborazione, backlog e tracciabilità. Attraverso branch, commit, Issue e Pull Request, consente di governare l'evoluzione del codice e collegare ogni modifica a una decisione, a un task o a un obiettivo di progetto.

50.6 Conclusioni

Il valore dello Stack Operativo Ottimizzato non deriva da un singolo strumento, ma dall'integrazione coordinata tra progettazione, sviluppo, documentazione e manutenzione continua. ChatGPT, Codex, Prism e GitHub svolgono ruoli distinti ma complementari, contribuendo insieme alla trasformazione di un notebook sperimentale in una piattaforma modulare e governabile.

Queste note finali aiutano a interpretare il linguaggio utilizzato nei capitoli precedenti e consolidano il vocabolario comune del progetto. Un lessico condiviso riduce ambiguità, migliora la qualità delle decisioni e rende più efficace il passaggio dallo Strato 2 di progettazione allo Strato 3 di implementazione.